เขียนโดย AI เพื่อ AI

$ RELIC-SCAN RUN

โอเพนซอร์ส
AGPL v3



# Relic Scan

1,009
บรรทัด · 4 ไฟล์

144
sessions

4
months

3.2k
rows

คู่มือทีละขั้น จากทรานสคริปต์ของเอเจนต์ ไปถึงดัชนีที่ถามได้

ทุกคำสั่งรันจริง ทุกภาพคือแอปจริง บนบ้านเดโมที่ทุกอย่างเป็นของสมมติ ยกเว้นภาพปกที่ AI วาด — 2026-10-26

Claude (AI, ไม่ใช่คน) กับ Nat

Soul-Brews-Studio/relic4-scan · relic-scan 26.10.26-alpha.516 · AGPL v3

# สารบัญ

คำนำ ..... 2

บทที่ 1: ติดตั้งและรันครั้งแรก ..... 4

บทที่ 2: รันซ้ำ อ่านแค่ของใหม่ ..... 15

บทที่ 3: เปิดดูในเว็บ ..... 24

บทที่ 4: ดิสก์ ห้องนิรภัย และการตั้งค่า ..... 32

บทที่ 5: ในเทอร์มินัล ..... 39

บทที่ 6: ถามดัชนีด้วย DuckDB ..... 50

บทที่ 7: เดือนที่ปิดผนึก และสำเนาไป S3 ..... 62

บทที่ 8: ทางเดินของข้อมูลข้ามเครื่อง ..... 70

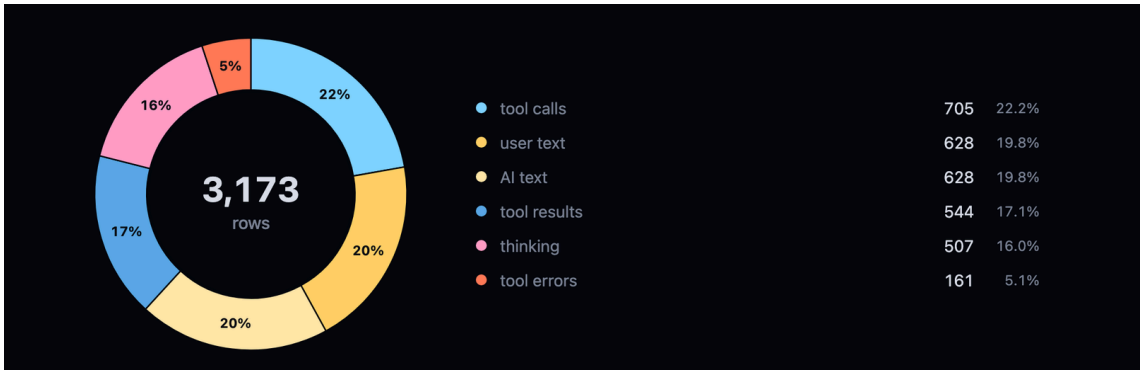
บทที่ 9: ความเป็นส่วนตัว และข้อควรระวัง ..... 77

บทที่ 10: รูปแบบของ transcript ที่ relic-scan อ่าน ..... 83

ภาคผนวก: โค้ดที่เลือก ..... 96

# คำนำ

ARRA Relic Scan เป็นเครื่องมือที่ทำงานเดียว ค้นหาทรานสคริปต์ของเอเจนต์ AI ทุกไฟล์ในเครื่อง แล้วเก็บเป็นดัชนีที่ถามได้ ไม่ว่าจะเป็น Claude Code, Codex, omp หรือตัวอื่น ผลลัพธ์คือไฟล์ Parquet หนึ่งไฟล์ต่อหนึ่งเดือน อยู่ใน `~/.relic-scan` และเครื่องมือเปิดดูได้สามทาง ได้แก่ หน้าเว็บในเครื่อง เทอร์มินัล และ DuckDB




รูปที่ 1: หน้า Smart Scan ของบ้านเดโม: โดนัทแสดงแถว 3,173 แถว แยกเป็น tool calls, user text, AI text, tool results, thinking และ tool errors

ภาพบนคือตัวอย่างของสิ่งที่ได้ เป็นหน้า Smart Scan ของบ้านเดโม แถวทั้งหมดในดัชนีแยกตามชนิด เล่มนี้ไม่มีภาพที่แต่งขึ้นนอกจากภาพปก

เล่มนี้พาเดินทีละขั้น จากติดตั้งไปจนถึงถามว่าเดือนนี้โมเดลไหนตอบไปกี่ครั้ง ใช้โทเคนเท่าไร ทุกคำสั่งในเล่มรันจริง ทุกภาพคือหน้าจอจริงของแอป

## ภาพมาจากไหน

ทรานสคริปต์จริงเป็นงานส่วนตัว จึงไม่มีสักบรรทัดอยู่ในเล่มนี้ ทุกภาพถ่ายจาก **บ้านเดโม** ที่สร้างด้วยสคริปต์ `make-demo-home.py` ในโฟลเดอร์ `book/` ข้างเล่ม เป็นทรานสคริปต์ 144 ไฟล์ของ repo สมมติหกตัว (tea-cart, starfish-api, harbor-bot, lantern-app, kite-docs, moss-cli) ข้อความ คำถาม คำตอบ และโมเดลทุกตัวแต่งขึ้น สคริปต์ใช้ seed คงที่ ใครรันก็ได้บ้านเดโมหน้าตาเดิม

ในแคปเจอร์ `~` จึงหมายถึงบ้านเดโมนั้น ไม่ใช่โฟลเดอร์ของใคร ส่วนภาพหน้าเว็บถ่ายจาก `relic-scan serve` ที่เปิดดัชนีเดโม ผ่าน Chrome แบบ headless ขนาด 1280 × 860

## วิธีอ่าน

- ภาค 1 ใช้งาน (บท 1–5): รันดัชนี เปิดหน้าเว็บ เปิดในเทอร์มินัล
- ภาค 2 อ่านดัชนี (บท 6–7): ถามด้วย DuckDB และเรื่องเดือนที่ปิดผนึก
- ภาค 3 ข้างใน (บท 8–9): chain ข้ามเครื่อง ความเป็นส่วนตัว
- ภาคผนวก (บท 10–11): รูปแบบของ transcript และโค้ดที่เลือก

แต่ละขั้นมีคำสั่งหรือหน้าจอ แล้วมีภาพของผลจริงอยู่ใต้มัน อ่านแค่ตัวหนังสือก็ทำตามได้ ภาพไว้ยืนยันว่าเห็นเหมือนกัน

## ใครเขียน

เล่มนี้เขียนโดย Claude ซึ่งเป็น AI กับ Nat คนที่เป็นเจ้าของโครงการ ภาพปกเป็นภาพเดียวที่ AI วาด ที่เหลือเป็นของจริงทั้งหมด

relic-scan เป็นโอเพนซอร์ส (AGPL v3) โค้ดอยู่ที่ https://github.com/Soul-Brews-Studio/relic4-scan หน้าเว็บของโครงการอยู่ที่ https://relic.buildwithoracle.com/scan/ เวอร์ชันที่ใช้ถ่ายทั้งเล่มคือ `26.10.26-alpha.516`

# บทที่ 1: ติดตั้งและรันครั้งแรก

เครื่องของคนที่ใช้ AI เขียนโค้ดทุกวัน มี transcript กองอยู่หลายที่ ทั้ง Claude Code, Codex, omp แต่ละตัวเก็บบทสนทนาเป็นไฟล์ `.jsonl` ไว้ในโฟลเดอร์ซ่อนของตัวเอง พอผ่านไปสามเดือนก็ไม่มีใครรู้ว่าอะไรอยู่ตรงไหน บทนี้พาติดตั้ง `relic-scan` แล้วรันครั้งแรก ให้เห็นกับตาว่ามันหาของเจอได้ยังไง และเขียนอะไรลงดิสก์บ้าง

## 1.1 relic-scan คืออะไร และบ้านเดโมที่ใช้ตลอดเล่ม

`relic-scan` เป็นโปรแกรมฟรี (AGPL) ของ Soul-Brews-Studio ทำงานเดียวคือหา transcript ของเอเจนต์ทุกตัวบนเครื่อง แล้วจัดเป็นดัชนีเดียว ดัชนีนี้เป็นไฟล์ Parquet หนึ่งไฟล์ต่อหนึ่งเดือน (นับเดือนตามเวลากรุงเทพ) อยู่ใต้ `~/.relic-scan` จากนั้นเปิดดูได้สามทาง คือหน้าเว็บบน 127.0.0.1 หน้าจอในเทอร์มินัล และ DuckDB ที่ถามเป็น SQL

มีกติกาข้อหนึ่งที่ต้องรู้ก่อนเริ่ม **transcript อ่านอย่างเดียว** โปรแกรมไม่แก้ ไม่ลบ ไม่ย้ายไฟล์ของเอเจนต์ และเขียนเฉพาะใน `~/.relic-scan` เท่านั้น หน้าเว็บก็ไม่เรียกเน็ต ที่ออกเน็ตได้มีสองกรณี คือ `export --to s3://…` ที่เราสั่งเอง กับ `ops` ที่ GET สถานะของ hub หนึ่งครั้ง

เหตุที่ต้องมีสามทางเปิดดู เพราะคนใช้ต่างกัน บางคนอยากกวาดตาดูภาพรวมในหน้าเว็บ บางคนอยู่ใน ssh และต้องการแค่เทอร์มินัล ส่วนคนที่ชอบ SQL ก็อยากถามตรง ๆ ทั้งสามทางอ่านดัชนีชุดเดียวกัน เลขที่ได้จึงตรงกันเสมอ ไม่ต้องกลัวว่าทางไหนให้คำตอบต่างจากทางอื่น

ตลอดเล่มนี้ทุกคำสั่งรันจริงเมื่อ 2026-10-26 บนเวอร์ชัน `26.10.26-alpha.516` และรันบน **บ้านเดโม** ที่สร้างจากสคริปต์ `make-demo-home.py` มี transcript 144 ไฟล์ของ repo สมมติ ไม่มีของจริงปนเลย ดังนั้น `~` ในทุกภาพคือบ้านเดโม ไม่ใช่บ้านของผู้เขียน

### ขั้นที่ 1 — ติดตั้ง

ติดตั้งจากซอร์สด้วยสคริปต์บรรทัดเดียว ต้องมี Rust 1.89 ขึ้นไปในเครื่อง **สคริปต์ build ราว 30 วินาที**แล้ววางไฟล์ไว้ที่ `~/.local/bin/relic-scan` ถ้าไม่อยากคอมไพล์เอง ก็ไปโหลดแอปเดสก์ท็อปจากหน้า Releases ของ GitHub ได้

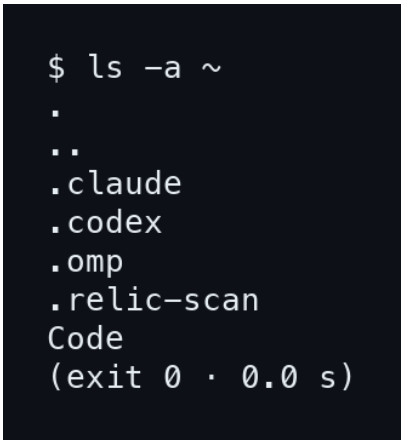
```
curl -fsSL https://raw.githubusercontent.com/Soul-Brews-Studio/relic4-scan/main/install.sh | sh
```

คำสั่งนี้ไม่ได้มาจากการรันในบ้านเดโม จึงไม่มีภาพประกอบ เป็นคำสั่งจากไฟล์ AGENTS.md ของโปรเจกต์ ถ้าคุณมีไบนารีอยู่แล้วก็ข้ามไปขั้นต่อไปได้เลย ตรวจแค่ว่า `~/.local/bin` อยู่ใน `PATH`

### ขั้นที่ 2 — ดูบ้านเดโมก่อนรัน

ก่อนสั่งอะไรกับ relic-scan ลองดูว่าในบ้านมีอะไร สั่ง `ls -a ~` เพื่อให้เห็นโฟลเดอร์ซ่อนด้วย

```
ls -a ~
```




รูปที่ 2: ผล ls -a ของบ้านเดโม มี .claude .codex .omp .relic-scan และ Code

สามโฟลเดอร์แรกคือ **บ้านของเอเจนต์** `.claude` เป็นของ Claude Code, `.codex` ของ Codex, `.omp` ของ omp ส่วน `Code` เป็นที่เก็บ repo และมี ψ vault ซ่อนอยู่ในบางโปรเจกต์ ส่วน `.relic-scan` คือโฟลเดอร์ผลลัพธ์ ตอนนี้ยังว่าง เดี๋ยวจะเห็นมันถูกเติมในหัวข้อ 1.5

## 1.2 ดูคำสั่งทั้งหมดและเวอร์ชัน

relic-scan มีคำสั่งย่อยเยอะ แต่ที่ใช้จริงบ่อยมีไม่กี่ตัว ลองไล่ดูจาก help แล้วจะเห็นว่าแต่ละตัวอยู่ตรงไหนของเล่มนี้

### ขั้นที่ 1 — อ่าน help

สั่ง `relic-scan --help` แล้วดูรายการ Commands ทีละบรรทัด

```
relic-scan --help
```

```
$ relic-scan --help
Find and index every AI-agent transcript on this machine

Usage: relic-scan [COMMAND]

Commands:
  scan    Find every agent home (.claude, .claude_bak, .claudebak, .claude2, .codex …) and index …
  … its .jsonl files
  run     Incremental: scan, then parse only new/grown/rewritten files from where they were left, …
  … then meta + map
  up      One command: `run`, then serve the map and the tree on 127.0.0.1 …
  … (http://127.0.0.1:PORT/map · /tree)
  serve   Serve OUT/meta/map.html on 127.0.0.1: /map (star map) and /tree (homes -> repos -> …
  … sessions)
  export  Copy the sealed months and their seals to an S3-compatible store (AWS S3, Garage, …
  … MinIO); re-runnable, nothing is replaced unless --overwrite
  ops     What the relic chain is doing right now: the pass in flight with its stages and ETA, …
  … running processes, the hub, the S3 job
  seals   Closed months and their seals (read-only files with the time, rows and key hash they …
  … were sealed at); --verify checks each again
  log     The scan log: every index run and check (dry run) with what it found; --diff N lists …
  … one entry's files
  tui     Browse the sessions, repos and every .jsonl in the terminal (reads OUT/meta/*.parquet)
  meta    The month files + _state.tsv -> meta/files.parquet + meta/sessions.parquet (`run` does …
  … this as its last step)
  map     meta/sessions.parquet + meta/files.parquet -> meta/map.html: every session as a star, …
  … queryable in the browser
  parse   Parse every indexed file -> one Parquet file per Bangkok month (truth schema, copies …
  … folded)
  help    Print this message or the help of the given subcommand(s)

Options:
  -h, --help     Print help
  -V, --version  Print version
(exit 0 · 0.0 s)
```

รูปที่ 3: ผลของ relic-scan --help แสดงคำสั่ง scan run up serve export ops seals log tui meta map parse

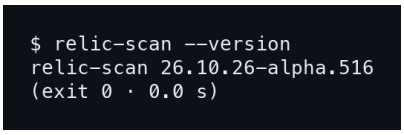
คำสั่งที่ต้องจำมีสี่ตัว **`run` คือคำสั่งหลัก** มันสแกน อ่านเฉพาะส่วนที่ใหม่ แล้วสร้าง meta กับหน้า map ให้ครบ `up` คือ `run` ตามด้วย serve หน้าเว็บ `tui` เปิดดูในเทอร์มินัล และ `log` ดูประวัติการรัน ส่วน `scan`, `parse`, `meta`, `map` เป็นขั้นย่อยที่ `run` เรียกต่อกันให้อยู่แล้ว ไม่ต้องสั่งเอง

สังเกตบรรทัดของ `run` ใน help ว่าเขียนไว้ว่า Incremental คือรันซ้ำได้ ไม่ต้องกลัวเสียเวลา บทที่ 2 จะพิสูจน์ข้อนี้ด้วยตัวเลข ส่วน `export` และ `seals` อยู่บทที่ 7

### ขั้นที่ 2 — ดูเวอร์ชัน

เวอร์ชันสำคัญตอนรายงานปัญหา และตอนเทียบผลกับเล่มนี้ ถ้าเลขต่างกัน ตัวเลขในผลที่คุณเห็นอาจไม่ตรงกับที่พิมพ์ไว้

```
relic-scan --version
```




รูปที่ 4: ผลของ relic-scan --version คือ relic-scan 26.10.26-alpha.516

```
relic-scan 26.10.26-alpha.516
```

เลขเวอร์ชันเป็นแบบ CalVer คือปี เดือน วัน ตามด้วยเลข alpha ของรุ่น เล่มนี้ใช้ **26.10.26-alpha.516** ตลอด

## 1.3 run --dry-run: ดูก่อนว่าจะอ่านอะไร

ครั้งแรกที่รันบนเครื่องจริง โปรแกรมจะไล่อ่าน transcript ทุกไฟล์ใต้โฟลเดอร์บ้าน และเขียนดัชนีลง `~/.relic-scan` AGENTS.md ของโปรเจกต์บอกให้ถามเจ้าของเครื่องก่อนรันครั้งแรกเสมอ เพราะบนเครื่องจริงมันกินเวลาและพื้นที่ดิสก์ ทางที่ปลอดภัยคือสั่ง dry run ก่อน

### ขั้นที่ 1 — สั่ง dry run

เติม `--dry-run` ต่อท้าย `run` ตัวโปรแกรมจะสแกนหาไฟล์เหมือนจริง แต่หยุดก่อนอ่านเนื้อหา และไม่เขียนอะไรลงดิสก์

```
relic-scan run --dry-run
```




รูปที่ 5: ผล dry run: new 144, 0.00 GB to read และกล่อง RELIC·SCAN บอกว่า nothing written

บรรทัดแรกคือสรุปทั้งหมด อ่านเป็นคู่ ๆ ตามนี้

```
run: scan 0.2s · new 144 · grown 0 · rewritten 0 · unchanged 0 · gone 0 …
```

แต่ละคำคือ **สถานะของไฟล์เทียบกับครั้งก่อน** `new` คือไฟล์ที่ไม่เคยเห็น `grown` คือไฟล์ที่เคยอ่านแล้วแต่มีบรรทัดต่อท้าย `rewritten` คือไฟล์ที่เนื้อหาเดิมเปลี่ยน `unchanged` คือไม่เปลี่ยน และ `gone` คือไฟล์ที่เคยมีแต่หายไป ตอนนี้ยังไม่เคยรัน เลยเป็น new 144 ทั้งหมด

ตัวเลือกของ `run` ดูได้จาก `relic-scan run --help`

```
relic-scan run --help
```

```
$ relic-scan run --help
Incremental: scan, then parse only new/grown/rewritten files from where they were left, then meta …
… + map

Usage: relic-scan run [OPTIONS]

Options:
      --out <OUT>          output directory: YYYY-MM.parquet per month, _state.tsv, meta/ …
      … [default: ~/.relic-s
      --root <ROOTS>
      --depth <DEPTH>      [default: 8]
      --threads <THREADS>  [default: 0]
      --exclude <EXCLUDE>
      --full               content-sniff NEW files (unchanged ones keep their verdict from the …
      … light scan)
      --agent <AGENT>
      --jt-compat
      --only <ONLY>        only files whose path contains this text (repeatable)
      --kind <KIND>        only homes of this kind: claude, codex, omp, omx, thclaws, gemini, pi …
      … (repeatable);
      --month <MONTH>      only rows of this Bangkok month, YYYY-MM. Without --out it writes …
      … ~/.relic-scan/<ki
      --day <DAY>          only rows of this Bangkok day, YYYY-MM-DD (same folder rule as --month)
      --from <FROM>        only rows on or after this Bangkok day, YYYY-MM-DD or YYYY-MM (same …
      … folder rule as
      --to <TO>            only rows on or before this Bangkok day, YYYY-MM-DD or YYYY-MM (same …
      … folder rule as
      --dry-run            only scan and compare with the state: print what an index would read …
      … (new, grown, r
  -h, --help               Print help
(exit 0 · 0.0 s)
```

รูปที่ 6: ผล relic-scan run --help: รายการตัวเลือก --out --root --depth \--threads --exclude --full --kind --month --dry-run

ในภาพมี `--out` กำหนดโฟลเดอร์ผลลัพธ์ `--root` เพิ่มโฟลเดอร์ที่จะสแกน `--kind` เลือกเฉพาะบ้านของเอเจนต์ชนิดเดียว `--month` เลือกเฉพาะเดือน และ `--dry-run` ที่เพิ่งใช้ไป คำอธิบายบางบรรทัดถูกตัดตรงขอบภาพ ส่วนบทที่ 2 จะใช้ `--kind` กับ `--month` จริง

## ขั้นที่ 2 — อ่านกล่อง RELIC·SCAN

ท้ายผลลัพธ์มีกล่องเล็ก ๆ ที่โปรแกรมวาดไว้ ทุกคำสั่งที่รันจริงจะจบด้วยกล่องแบบนี้ รูปร่างเดียวกัน เปลี่ยนแค่ข้อความข้างใน

```
| dry run · an index would read 144 files, 0.00 GB       |
| scan 0.2s · nothing written                            |
| → ~/.relic-scan                                        |
```

บรรทัดที่ควรอ่านทุกครั้งคือ **nothing written** กับลูกศรที่ชี้ปลายทาง ลูกศรบอกว่าถ้ารันจริงจะเขียนที่ไหน คือ `~/.relic-scan` ส่วน `0.00 GB` ไม่ใช่ศูนย์จริง แต่ transcript เดโมเล็กมาก ทั้งหมดรวมกันแค่ราว 1.4 MB (ตัวเลขนี้ไปเห็นอีกทีในบทที่ 2) ปัดเป็น GB เลยออกมาเป็นศูนย์

เทียบให้เห็นสเกล ในเครื่อง Mac จริงเครื่องหนึ่งที่ AGENTS.md ยกตัวอย่างไว้ มี 87,000 ไฟล์ รวม 75 GB ได้ 5.4 ล้านแถว และ index ครั้งแรกใช้ราวหนึ่งนาที เล่มนี้ไม่อ้างตัวเลขเกินไปกว่านี้

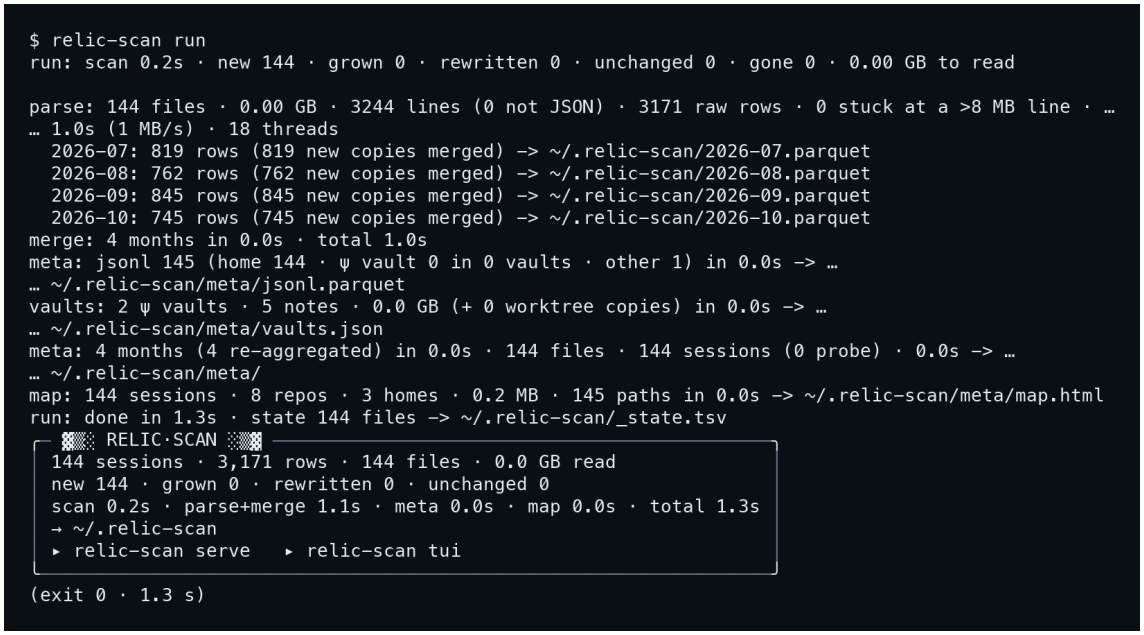
## 1.4 run ครั้งแรก: อ่านผลทีละบรรทัด

พอ dry run บอกว่าเห็นไฟล์ 144 ไฟล์ ก็ถึงเวลาลงมือจริง คำสั่งเดียวกัน ตัดแค่ `--dry-run` ออก ผลลัพธ์ยาวหน่อย แต่แต่ละบรรทัดขึ้นต้นด้วยชื่อขั้นตอน อ่านตามลำดับได้เลย

### ขั้นที่ 1 — สั่ง run

สั่ง `relic-scan run` บนบ้านเดโม ใช้เวลารวม 1.3 วินาที

```
relic-scan run
```




รูปที่ 7: ผล relic-scan run ครั้งแรก ไล่ขั้น scan, parse, merge, meta, vaults, map, done และกล่องสรุป

**ไล่ทีละขั้นตามที่โปรแกรมพิมพ์** ขั้น `scan` คือบรรทัดแรก เหมือนกับ dry run ทุกตัวอักษร เจอ 144 ไฟล์ใหม่ ขั้นถัดมา `parse` อ่านเนื้อไฟล์จริง

```
parse: 144 files · 0.00 GB · 3244 lines (0 not JSON) · 3171 raw rows · …
```

144 ไฟล์มี 3244 บรรทัด ไม่มีบรรทัดไหนที่ไม่ใช่ JSON พอแปลงแล้วได้ 3171 แถวดิบ ตัวเลขสองตัวนี้ไม่เท่ากันเพราะบรรทัดหนึ่งอาจให้หลายแถว หรือไม่ให้เลยก็ได้ เช่นบรรทัดที่เป็นแค่ข้อมูลกำกับ บทที่ 10 จะอธิบายว่าบรรทัดหนึ่งกลายเป็นแถวได้ยังไง

### ขั้นที่ 2 — ดู month ที่ถูกเขียน

ใต้บรรทัด parse มีสี่บรรทัดที่เยื้องเข้าไป หนึ่งบรรทัดต่อหนึ่งเดือน ตรงนี้คือจุดที่ relic-scan จัดแถวลงไฟล์ตามเวลากรุงเทพ

```
  2026-07: 819 rows (819 new copies merged) -> ~/.relic-
scan/2026-07.parquet
  2026-08: 762 rows (762 new copies merged) -> ~/.relic-
scan/2026-08.parquet
  2026-09: 845 rows (845 new copies merged) -> ~/.relic-
scan/2026-09.parquet
  2026-10: 745 rows (745 new copies merged) -> ~/.relic-
scan/2026-10.parquet
```

ไฟล์เดือนที่เขียนแล้วดูได้ด้วย `ls -l`

```
ls -l ~/.relic-scan/*.parquet
```

```
$ ls -l ~/.relic-scan/*.parquet
-r--r--r--@ 34153 ~/.relic-scan/2026-07.parquet
-r--r--r--@ 32337 ~/.relic-scan/2026-08.parquet
-r--r--r--@ 34805 ~/.relic-scan/2026-09.parquet
-rw-r--r--@ 31950 ~/.relic-scan/2026-10.parquet
(exit 0 · 0.0 s)
```

รูปที่ 8: ผล ls -l ของไฟล์ parquet สี่เดือน สามเดือนแรกเป็น -r--r--r-- และเดือน 2026-10 เป็น -rw-r--r--

สามเดือนแรก (2026-07 ถึง 2026-09) ขึ้นต้นด้วย `-r--r--r--` คืออ่านได้อย่างเดียว เพราะเป็นเดือนที่ผนึกแล้ว ส่วน 2026-10 เป็น `-rw-r--r--` ยังเขียนต่อได้เพราะเป็นเดือนที่เปิดอยู่ (เรื่องผนึกอยู่ในบทที่ 7)

บวกสี่เดือนได้ 3171 พอดี คือ 819 + 762 + 845 + 745 วงเล็บ `new copies merged` บอกว่ามีกี่แถวที่เข้าไฟล์ใหม่ ถ้าไฟล์เดียวกันมีสำเนาในหลายบ้าน (เช่น mirror หรือ backup) มันจะถูก **พับเป็นแถวเดียว** ในเดโมไม่มีสำเนาซ้ำ ตัวเลขสองฝั่งเลยเท่ากัน

### ขั้นที่ 3 — meta, vaults, map

หลังจาก merge มีสามบรรทัดที่สร้างของเสริมสำหรับหน้าเว็บ ไล่อ่านทีละบรรทัดแล้วจะเห็นว่า relic-scan เก็บอะไรไว้นอกเหนือจากแถวข้อความ

```
meta: jsonl 145 (home 144 · ψ vault 0 in 0 vaults · other 1) in 0.0s -> …
vaults: 2 ψ vaults · 5 notes · 0.0 GB (+ 0 worktree copies) in 0.0s -> …
map: 144 sessions · 8 repos · 3 homes · 0.2 MB · 145 paths in 0.0s -> …
```

บรรทัด meta นับไฟล์ `.jsonl` ที่เห็นทั้งหมด 145 ไฟล์ เป็นของบ้านเอเจนต์ 144 ไฟล์ อีก 1 ไฟล์เป็น other คือ `.jsonl` ที่ไม่ใช่ transcript ถูกเก็บไว้ในรายการแต่ไม่นับเป็น session บรรทัด vaults พบ ψ vault 2 แห่ง รวม 5 โน้ต ส่วน map สร้างหน้าเว็บ **144 sessions จาก 8 repos ใน 3 บ้าน** ตัวเลขเหล่านี้จะกลับมาอีกในหน้าเว็บบทที่ 3

### ขั้นที่ 4 — อ่านกล่องสรุป

สุดท้ายคือบรรทัด `run: done` กับกล่องสรุป กล่องนี้คือที่ที่ควรมองก่อนอย่างอื่น เพราะมันตอบคำถามว่า รอบนี้ได้อะไรมา และใช้เวลาไปกับขั้นไหน

```
| 144 sessions · 3,171 rows · 144 files · 0.0 GB read              |
| new 144 · grown 0 · rewritten 0 · unchanged 0                    |
| scan 0.2s · parse+merge 1.1s · meta 0.0s · map 0.0s · total 1.3s |
```

**3,171 แถว จาก 144 session** คือเลขที่ควรจำ มันจะตรงกับตัวเลขในหน้า Smart Scan หลังบทที่ 2 (ที่มีแถวเพิ่มอีก 2) เวลาทั้งหมด 1.3 วินาที ส่วนใหญ่หมดไปกับ scan และ parse ท้ายกล่องมีสองบรรทัดชี้ทางต่อ คือ `relic-scan serve` กับ `relic-scan tui` ทั้งสองคือทางเปิดดูผลที่เพิ่งทำเสร็จ

## 1.5 อะไรอยู่ใน ~/.relic-scan

รันเสร็จแล้ว มาดูว่าโปรแกรมวางอะไรไว้ในโฟลเดอร์ผลลัพธ์ ใช้ `ls` ชี้ทั้งโฟลเดอร์หลักและ `meta` ในคำสั่งเดียว

### ขั้นที่ 1 — ไล่ดูโฟลเดอร์

สั่ง `ls` สองโฟลเดอร์พร้อมกัน ผลจะแยกเป็นสองกลุ่มโดยมีบรรทัดว่างคั่น

```
ls ~/.relic-scan ~/.relic-scan/meta
```



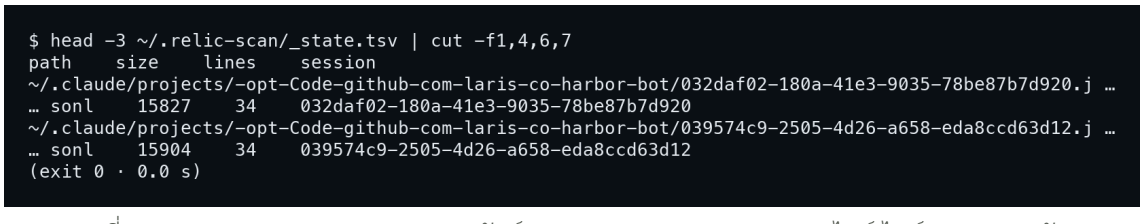

รูปที่ 9: รายการไฟล์ใน ~/.relic-scan และ ~/.relic-scan/meta

กลุ่มแรกคือโฟลเดอร์หลัก **ไฟล์ขึ้นต้นด้วย `_` เป็นสถานะของโปรแกรม** ส่วนที่เหลือคือข้อมูล เรียงตามหน้าที่ได้ดังนี้

- `2026-07.parquet` ถึง `2026-10.parquet` คือ **แถวของข้อความทั้งหมด** หนึ่งไฟล์ต่อเดือน ถามด้วย DuckDB ได้ตรง ๆ (บทที่ 6)
- `_state.tsv` จำว่าไฟล์ transcript แต่ละไฟล์อ่านไปถึงตรงไหนแล้ว นี่คือเหตุผลที่รันซ้ำเร็ว (ดูภาพด้านล่าง)
- `_ledger.tsv` เป็นสมุดบัญชีที่ต่อท้ายทุกครั้งที่ index ไม่เคยถูกเขียนทับ
- `_scans.jsonl` คือบันทึกการรัน ที่ `relic-scan log` อ่านออกมาแสดง
- `_config.json` เก็บค่าตั้ง เช่น โฟลเดอร์ที่เดินสแกน (บทที่ 4)

`_state.tsv` เปิดดูได้ตรง ๆ เพราะเป็นข้อความคั่นด้วย tab

```
head -3 ~/.relic-scan/_state.tsv | cut -f1,4,6,7
```




รูปที่ 10: ผล head ของ _state.tsv คอลัมน์ path size lines session สองไฟล์ ไฟล์ละ 34 บรรทัด

แต่ละแถวคือหนึ่งไฟล์ transcript มี path, ขนาดเป็นไบต์, จำนวนบรรทัด และ id ของ session ในภาพทั้งสองไฟล์มี 34 บรรทัด ขนาด 15827 กับ 15904 ไบต์

### ขั้นที่ 2 — ดูใน meta

กลุ่มที่สองคือ `meta/` ซึ่งเก็บ **สรุปรายไฟล์และราย session** แยกจากแถวข้อความ ทำให้หน้าเว็บเปิดเร็วโดยไม่ต้องกวาดข้อความทั้งหมด

- `sessions.parquet` หนึ่งแถวต่อหนึ่ง session มี repo, เวลาแรกและสุดท้าย, จำนวนแถว
- `files.parquet` และ `jsonl.parquet` หนึ่งแถวต่อหนึ่งไฟล์ transcript
- `months.json` ข้อมูลของแต่ละไฟล์เดือน รวมถึงผนึกของเดือนที่ปิดแล้ว
- `vaults.json` กับโฟลเดอร์ `vaults/` คือรายการ ψ vault และโน้ตข้างใน
- `map.html` คือหน้าเว็บทั้งหน้า สร้างไว้เป็นไฟล์เดียวที่ `serve` ส่งออกไป
- `paths.json` และ `unreadable.json` รายการ path ที่เห็น และที่อ่านไม่ได้

```
ls -l ~/.relic-scan/meta
```




รูปที่ 11: ผล ls -l ของ meta: files.parquet jsonl.parquet map.html months.json paths.json sessions.parquet unreadable.json vaults.json และโฟลเดอร์ cache กับ vaults

ในภาพ `map.html` ใหญ่สุด 217325 ไบต์ เพราะเป็นหน้าเว็บทั้งหน้า ส่วน `sessions.parquet` 26517 ไบต์ และ `months.json` 2063 ไบต์ มี `cache` กับ `vaults` เป็นโฟลเดอร์

ลองสังเกตว่าชื่อไฟล์เดือนเรียงตามตัวอักษรก็เรียงตามเวลาไปในตัว ทำให้เวลาใช้ DuckDB อ่านด้วย `*.parquet` ก็ได้ข้อมูลครบทุกเดือนในคำเดียว ส่วนไฟล์ใน `meta/` ไม่ใช่แถวข้อความ จึงไม่ปนเข้าไปในกลุ่มนั้น

จุดที่ควรจำคือทุกอย่างอยู่ใต้ `~/.relic-scan` ลบโฟลเดอร์นี้ทิ้งเมื่อไหร่ก็ได้ transcript ต้นทางไม่เสียอะไร แล้วสั่ง `relic-scan run` ใหม่ก็ได้ดัชนีกลับมา AGENTS.md แนะนำวิธีนี้ด้วยซ้ำ เมื่อต้องการดัชนีรูปแบบล่าสุดจากศูนย์ ให้ย้ายโฟลเดอร์เก่าไปไว้ข้างแล้วรันใหม่

## ต่อไป

ตอนนี้ดัชนีเดโมมี 3,171 แถวนอนอยู่ใน `~/.relic-scan` แล้ว คำถามถัดไปคือ ถ้ารันซ้ำทันที มันจะอ่านทุกอย่างใหม่ไหม และถ้า transcript ไฟล์หนึ่งโตขึ้นสองบรรทัด จะเกิดอะไรขึ้น บทที่ 2 ตอบสองข้อนี้ด้วยตัวเลขจากการรันจริง

# บทที่ 2: รันซ้ำ อ่านแค่ของใหม่

ดัชนีที่สร้างครั้งแรกใช้ 1.3 วินาทีบนบ้านเดโม แต่บนเครื่องจริงที่มีข้อมูลหลายแสนบรรทัด มันกินเวลาเป็นนาที ถ้าทุกครั้งที่อยากอัปเดตต้องอ่านใหม่หมด ก็คงไม่มีใครอยากรัน relic-scan บ่อย ๆ บทนี้เลยทดลองสามอย่าง คือรันซ้ำทันทีโดยไม่มีอะไรเปลี่ยน รันหลังจาก transcript ไฟล์หนึ่งโตขึ้น และรันเฉพาะบางส่วนของเครื่อง ทั้งหมดดูจากตัวเลขในผลลัพธ์จริง

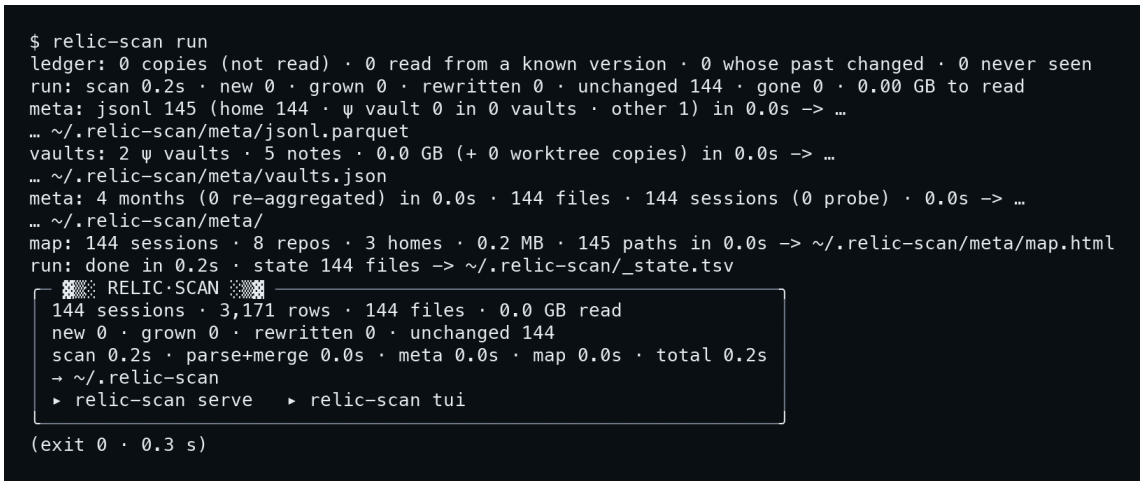
## 2.1 รันซ้ำทันที: ไม่อ่านอะไรเลย

หลักการของ `run` คือ **จำว่าอ่านไฟล์ไหนไปถึงไหนแล้ว** ไฟล์ `_state.tsv` ที่เจอในบทที่ 1 เก็บตำแหน่งนั้นไว้ พอรันรอบใหม่ โปรแกรมสแกนหาไฟล์อีกรอบเพื่อเทียบ ถ้าไฟล์ไหนเหมือนเดิมก็ข้าม เราจะพิสูจน์ข้อนี้ด้วยการสั่งซ้ำโดยยังไม่แตะอะไรเลย

### ขั้นที่ 1 — สั่ง run อีกรอบ

พิมพ์คำสั่งเดียวกับบทที่ 1 ทุกตัวอักษร ไม่ต้องใส่อะไรเพิ่ม ระหว่างรอบแรกกับรอบนี้ยังไม่มี transcript ใหม่เกิดขึ้น

```
relic-scan run
```




รูปที่ 12: ผล run รอบที่สอง: new 0, unchanged 144, total 0.3s ไม่มีบรรทัด parse

เทียบกับรอบแรก จุดที่ต่างมีสามจุด จุดแรกคือบรรทัดบนสุดที่รอบแรกไม่มี เป็นบรรทัดของ ledger ที่เป็นศูนย์ทั้งสี่ช่อง

```
ledger: 0 copies (not read) · 0 read from a known version · 0 whose past
…
run: scan 0.2s · new 0 · grown 0 · rewritten 0 · unchanged 144 · gone 0 …
```

จุดที่สองคือบรรทัด scan ซึ่งตอนนี้เป็น **new 0 และ unchanged 144** ทั้งหมด ไฟล์ทุกไฟล์ถูกรู้จักแล้วและไม่มีไฟล์ไหนเปลี่ยน จุดที่สามคือหายไปทั้งก้อน ไม่มีบรรทัด `parse:` ไม่มีบรรทัดรายเดือน ไม่มี `merge:` เพราะไม่มีอะไรให้อ่าน

### ขั้นที่ 2 — ดูว่าขั้นหลังรันซ้ำอย่างไร

บรรทัด meta กับ map ยังโผล่มาให้เห็นเหมือนเดิม แต่ดูตัวเลขข้างในดีดี

```
meta: 4 months (0 re-aggregated) in 0.0s · 144 files · 144 sessions …
run: done in 0.3s · state 144 files -> ~/.relic-scan/_state.tsv
```

รอบแรกเขียนไว้ว่า `4 months (4 re-aggregated)` คือสรุปทั้งสี่เดือนใหม่ รอบนี้เป็น **0 re-aggregated** ไม่มีเดือนไหนต้องสรุปซ้ำ ส่วนแถวสุดท้ายในกล่องยังเป็น 3,171 แถวเท่าเดิม และเวลารวมเหลือ 0.3 วินาที จากที่รอบแรกใช้ 1.3 วินาที เวลาที่เหลือเกือบทั้งหมดคือ scan 0.2 วินาที คือโปรแกรมเดินดูว่าไฟล์ไหนเปลี่ยนบ้าง

บรรทัด `map:` ยังถูกสร้างใหม่ทุกครั้ง แต่ใช้เวลา 0.0 วินาที เพราะไม่ต้องอ่านข้อมูลก้อนใหญ่ ราคาของการรันซ้ำจึงเท่ากับราคาของการเดินดูไฟล์ เท่านั้นเอง

ข้อนี้สำคัญตอนตั้งให้รันอัตโนมัติ เช่นทุกคืนผ่าน cron หรือ launchd ถ้ารันซ้ำถูกแบบนี้ ก็ปล่อยให้รันบ่อยแค่ไหนก็ได้ โดยไม่ต้องห่วงว่าจะไปแย่งซีพียูกับงานของเรา และผลลัพธ์ไม่เปลี่ยนไม่ว่าจะรันกี่รอบก็ตามถ้าต้นทางไม่เปลี่ยน

## 2.2 transcript โตขึ้นหนึ่งไฟล์

รันซ้ำตอนไม่มีอะไรเปลี่ยนเป็นกรณีง่าย กรณีจริงคือเอเจนต์ยังเขียนอยู่ transcript ที่อ่านไปแล้วจะยาวขึ้นเรื่อย ๆ ตอนนั้น relic-scan ต้องอ่านเฉพาะส่วนท้ายที่เพิ่มมา ไม่อ่านทั้งไฟล์ซ้ำ ในเดโมเราต่อท้ายไฟล์ transcript ของ Claude Code หนึ่งไฟล์ด้วยสองบรรทัด แล้วรันอีกรอบ

### ขั้นที่ 1 — รันหลังไฟล์โต

คำสั่งยังเป็นตัวเดิมอีกแล้ว คนรันไม่ต้องบอกว่าไฟล์ไหนโต โปรแกรมหาเอง

```
relic-scan run
```




รูปที่ 13: ผล run หลัง transcript โตหนึ่งไฟล์: grown 1, unchanged 143, parse 1 files, 2 raw rows

บรรทัด scan บอกสถานะใหม่ทันที

```
run: scan 0.2s · new 0 · grown 1 · rewritten 0 · unchanged 143 · gone 0 …
```

**grown 1** คือไฟล์ที่เคยอ่านแล้วและยาวขึ้น ที่เหลืออีก 143 ไฟล์ยัง unchanged ขอให้สังเกตว่าไม่ใช่ new และไม่ใช่ rewritten ด้วย โปรแกรมแยกสามกรณีนี้ออกจากกัน เพราะวิธีจัดการต่างกัน ไฟล์ใหม่อ่านทั้งไฟล์ ไฟล์ที่โตอ่านต่อจากเดิม ส่วนไฟล์ที่ถูกเขียนทับต้องอ่านใหม่

ถ้าอยากลองเอง ให้เปิด transcript ของเดโมสักไฟล์ ต่อท้ายด้วยบรรทัด JSON ที่ถูกรูปแบบสองบรรทัด แล้วสั่ง run ดู ผลจะออกมาแบบเดียวกับภาพข้างบน แต่ตัวเลขแถวอาจต่างตามสิ่งที่คุณต่อท้ายลงไป

### ขั้นที่ 2 — อ่านบรรทัด parse

ทีนี้บรรทัด `parse:` กลับมา แต่ตัวเลขเล็กมากเทียบกับรอบแรก

```
parse: 1 files · 0.00 GB · 2 lines (0 not JSON) · 2 raw rows · …
  2026-10: 747 rows (2 new copies merged) -> ~/.relic-
scan/2026-10.parquet
merge: 1 months in 0.0s · total 0.0s
```

รอบแรก parse 144 ไฟล์ 3244 บรรทัด รอบนี้ parse **1 ไฟล์ 2 บรรทัด** ได้ 2 แถวดิบ ซึ่งตรงกับที่เราต่อท้ายไป และมีบรรทัดเดือนบรรทัดเดียวคือ 2026-10 เพราะสองบรรทัดนั้น

ลงวันที่ในเดือนตุลาคม จำนวนแถวของเดือนนั้นขยับจาก 745 เป็น 747 ส่วนสามเดือนก่อนหน้าไม่ถูกแตะเลย

### ขั้นที่ 3 — ตรวจยอดรวมและส่วนที่สรุปใหม่

ลงมาที่บรรทัด meta และกล่องสรุป ตัวเลขสองจุดนี้ยืนยันว่าอ่านแค่ของใหม่จริง

```
meta: 4 months (1 re-aggregated) in 0.0s · 144 files · 144 sessions …
| 144 sessions · 3,173 rows · 144 files · 0.0 GB read                |
| new 0 · grown 1 · rewritten 0 · unchanged 143                      |
```

สรุปใหม่ **1 เดือนจากสี่** คือเดือนที่ได้แถวใหม่ ส่วนยอดรวมขยับจาก 3,171 เป็น 3,173 ตรงกับสองแถวที่เพิ่ม และจำนวน session ยังเป็น 144 เพราะบรรทัดที่ต่อไปเป็นของ session เดิม ตัวเลข 3,173 นี้คือยอดที่หน้าเว็บทุกหน้าในบทถัดไปแสดง

บรรทัด ledger ด้านบนยังเป็นศูนย์ทั้งหมดในรอบนี้ ตรงกับที่ปรากฏในผลลัพธ์ ledger เป็นสมุดของ “เวอร์ชันที่เคยอ่านมาแล้ว” ซึ่งจะทำงานเมื่อมีไฟล์ใหม่ที่เนื้อหาตรงกับของเก่า เช่น โฟลเดอร์ที่ย้ายที่หรือสำเนาจากอีกเครื่อง ไฟล์ที่โตขึ้นในที่เดิมใช้ตำแหน่งที่จำไว้ใน `_state.tsv` อยู่แล้ว

ตัวสมุดคือ `_ledger.tsv` เปิดดูหัวไฟล์ได้

```
head -4 ~/.relic-scan/_ledger.tsv | cut -f2,3,5,9
```




รูปที่ 14: ผล head ของ _ledger.tsv คอลัมน์ lines bytes session how สามแถว ทุกแถว how เป็น read

หนึ่งแถวคือหนึ่งเวอร์ชันของหนึ่ง transcript ในภาพสามแถวแรกมี 17, 28 และ 19 บรรทัด ขนาด 7856, 12970 และ 8751 ไบต์ และคอลัมน์ `how` เป็น `read` ทั้งหมด คืออ่านจริงจากไฟล์ ไม่ใช่สำเนา

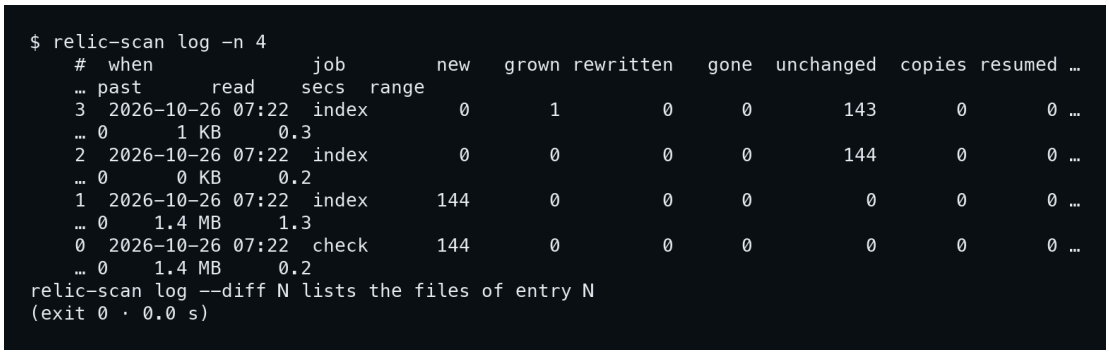
## 2.3 relic-scan log และหน้า Scan log

สี่รอบที่ผ่านมา (รอบ dry run, รอบแรก, รอบซ้ำ, รอบไฟล์โต) ทุกครั้งโปรแกรมต่อบรรทัดหนึ่งลงไฟล์ `_scans.jsonl` ไม่เคยเขียนทับ คำสั่ง `log` อ่านไฟล์นั้นออกมาเป็นตาราง ดูย้อนหลังได้ว่าแต่ละรอบเกิดอะไรขึ้น โดยไม่ต้องเลื่อนหาในเทอร์มินัล

### ขั้นที่ 1 — ดูสี่รอบล่าสุด

ใส่ `-n 4` เพื่อจำกัดว่าเอากี่รายการล่าสุด ในเดโมมีพอดีสี่รายการ

```
relic-scan log -n 4
```




รูปที่ 15: ตาราง relic-scan log -n 4 สี่แถว: check 144 ใหม่, index 144 ใหม่, index 144 ไม่เปลี่ยน, index grown 1

ตารางเรียงใหม่ไปเก่า เลขแถว `#` เริ่มที่ 0 สำหรับรายการแรกสุด อ่านจากล่างขึ้นบนจะได้เรื่องราวตามลำดับเวลาที่เราทำ

```
#  when              job        new   grown rewritten   gone  unch …
3  2026-10-26 07:22  index        0       1         0      0       …
2  2026-10-26 07:22  index        0       0         0      0       …
1  2026-10-26 07:22  index      144       0         0      0       …
```

ตัดคอลัมน์ท้ายออกเพื่อให้พอดีบรรทัด ต้นฉบับกว้างกว่านี้ (ภาพมีครบ) รายการ 0 คือ dry run ซึ่งขึ้นเป็น job `check` ส่วนอีกสามเป็น job `index` ให้ดูคอลัมน์ **read** รอบแรกอ่าน 1.4 MB รอบที่สองอ่าน 0 KB รอบที่สามอ่านแค่ 1 KB และมันบอกด้วยว่าบนเดโมนี้ transcript ทั้งหมดรวมกัน 1.4 MB ซึ่งเป็นสาเหตุที่ GB ในกล่องออกมาเป็น 0.00

### ขั้นที่ 2 — ดูไฟล์ในรายการหนึ่ง

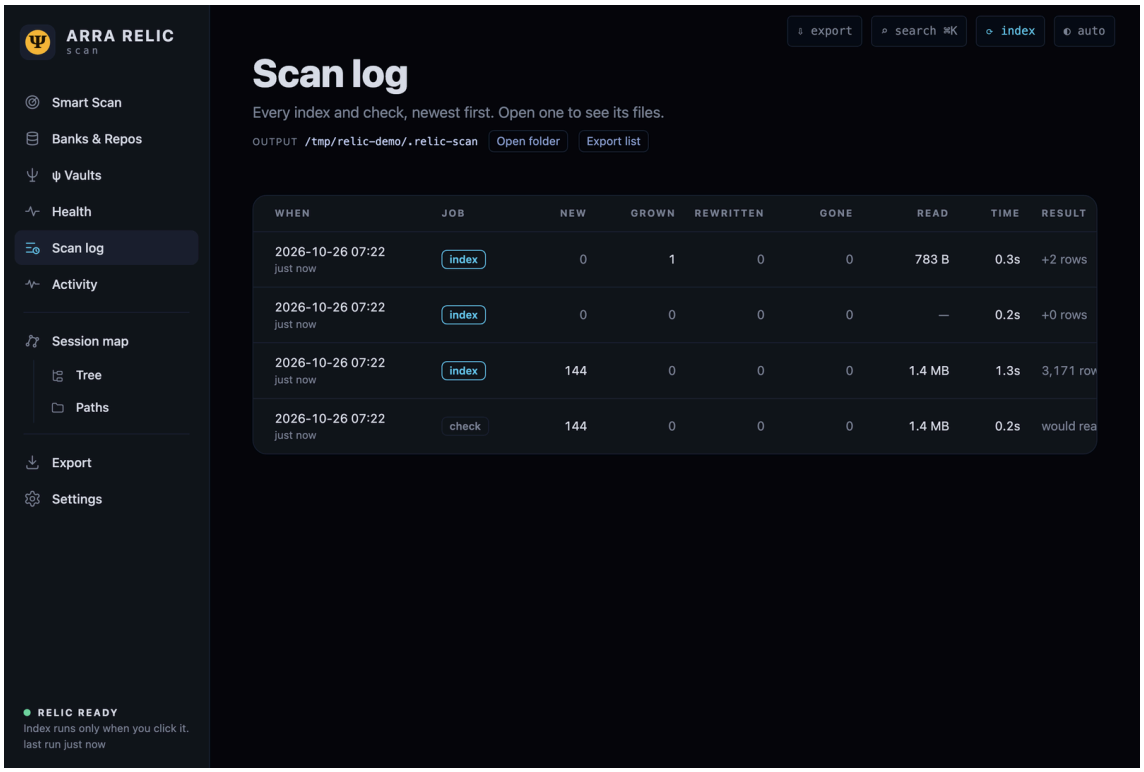
บรรทัดท้ายสุดของผลลัพธ์บอก **วิธีเจาะลึกต่อด้วย `--diff`**

```
relic-scan log --diff N lists the files of entry N
```

ใส่เลขแถวแทน N เช่นของรอบที่ไฟล์โตก็จะได้รายชื่อไฟล์ที่มีสถานะ grown พร้อมขนาดที่เพิ่ม ใช้ตอนอยากรู้ว่าไฟล์ไหนทำให้เลขเปลี่ยน ในเล่มนี้ไม่ได้รัน `--diff` จึงไม่มีผลลัพธ์ให้ดู

### ขั้นที่ 3 — เปิดหน้า Scan log บนเว็บ

ข้อมูลชุดเดียวกันมีให้ดูในเว็บด้วย เปิด server ด้วย `relic-scan serve` หรือ `relic-scan up` ซึ่งใช้พอร์ต 8787 ถ้าไม่ระบุ (ภาพในเล่มใช้ `--port 8791`) แล้วเปิดหน้า `/scans` ในเบราว์เซอร์ คือ `http://127.0.0.1:8787/scans`




รูปที่ 16: หน้า Scan log ที่ /scans: ตารางสี่รายการ คอลัมน์ WHEN JOB NEW GROWN REWRITTEN GONE READ TIME RESULT

หน้านี้เป็นตารางเดียวกับคำสั่ง log แต่คอลัมน์ท้ายสุดคือ **RESULT** ซึ่งสรุปผลเป็นประโยคสั้น ๆ เช่น `+2 rows` สำหรับรอบที่ไฟล์โต `+0 rows` สำหรับรอบซ้ำ `3,171 rows` สำหรับรอบแรก และ `would read…` สำหรับรอบ dry run ที่ไม่ได้อ่านจริง ถ้าอยากเช็กว่าช่วงนี้ nightly job ทำงานหรือเปล่า ดูหน้านี้เร็วกว่าเปิดไฟล์ jsonl เอง

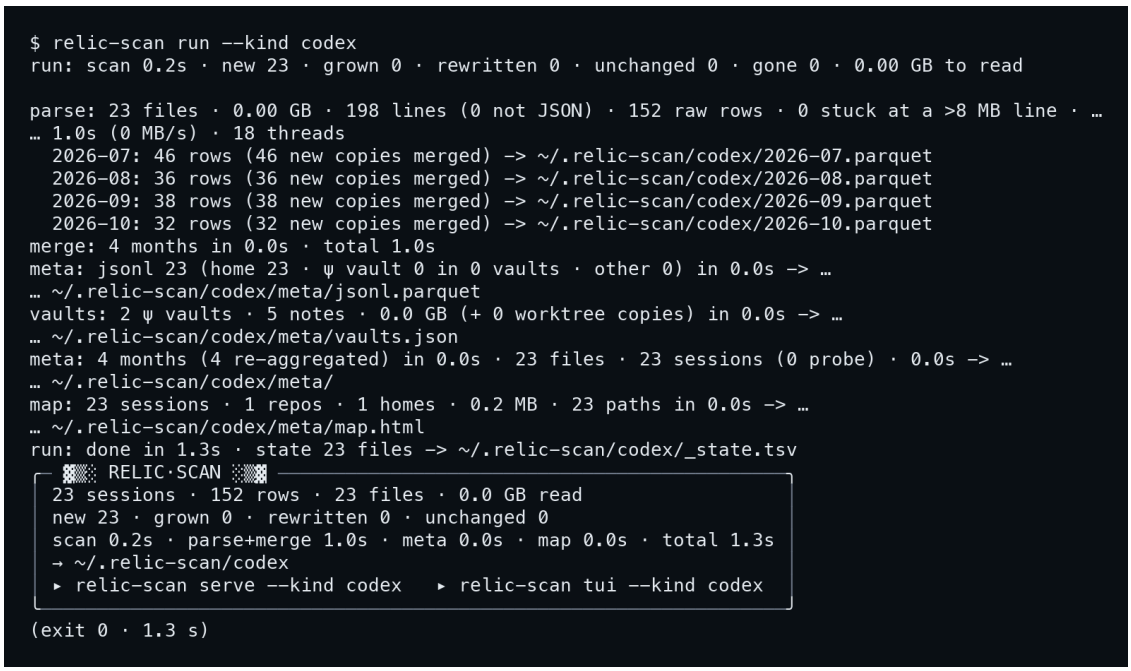
## 2.4 รันแค่บางส่วน: --kind และ --month

ปกติ `run` เก็บทุกเอเจนต์ทุกเดือนไว้ในดัชนีเดียว แต่บางครั้งอยากได้ชิ้นเล็กกว่า เช่นดูเฉพาะ Codex หรือเฉพาะเดือนนี้ มีสองตัวเลือกให้ ที่เด่นคือทั้งสองแบบ **เขียนลงโฟลเดอร์แยก** ไม่ปนกับดัชนีหลัก และโฟลเดอร์แยกแต่ละอันมีไฟล์ตั้งค่า `_config.json` ของตัวเอง (ในเดโมคัดลอกจากดัชนีหลัก เพื่อให้สแกนแค่ `~`)

### ขั้นที่ 1 — รันเฉพาะ Codex

ใส่ `--kind codex` ต่อท้าย `run` โปรแกรมจะกรองเอาเฉพาะ transcript ของ Codex

```
relic-scan run --kind codex
```




รูปที่ 17: ผล run --kind codex: new 23 ไฟล์, สี่เดือน 2026-07 ถึง 2026-10, ปลายทาง ~/.relic-scan/codex

บรรทัดแรกบอกว่าพบ 23 ไฟล์ใหม่ ซึ่งเป็นของ Codex ล้วน ๆ จากทั้งหมด 144 ไฟล์ในบ้านเดโม พอ parse เสร็จผลลัพธ์กระจายลงสี่เดือน ทุกบรรทัดเขียนลงโฟลเดอร์ `codex/` ใต้ดัชนีหลัก

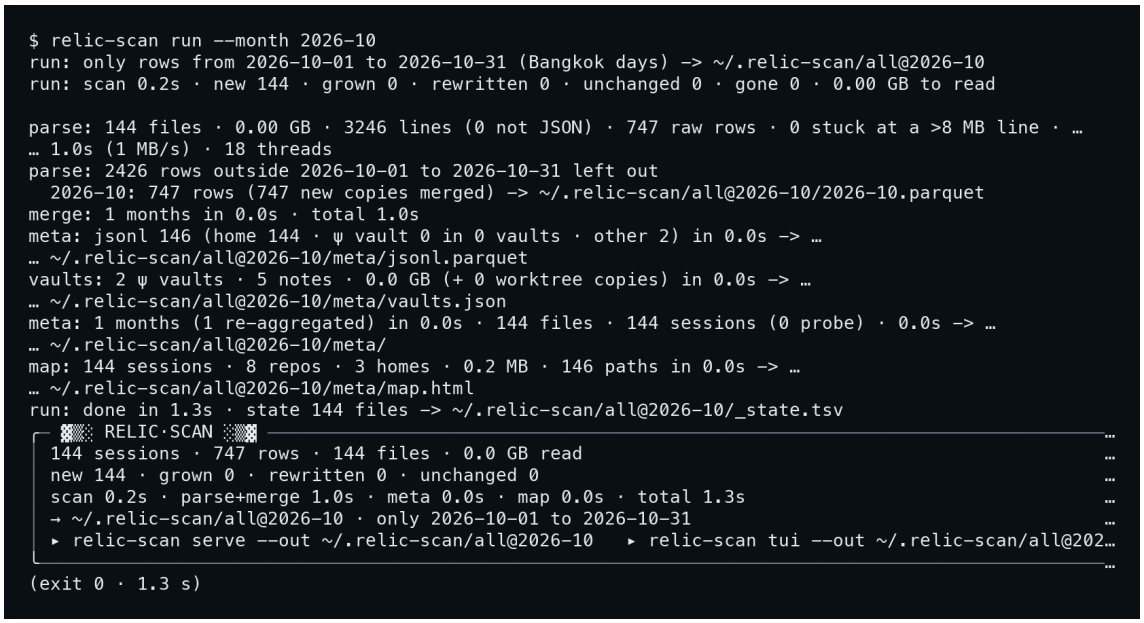
```
  2026-07: 46 rows (46 new copies merged) -> ~/.relic-scan/codex/2026-0…
| 23 sessions · 152 rows · 23 files · 0.0 GB read                       |
| → ~/.relic-scan/codex                                                 |
| ▸ relic-scan serve --kind codex   ▸ relic-scan tui --kind codex       |
```

มีสองอย่างน่าดู อย่างแรก **ปลายทางคือ `~/.relic-scan/codex`** ไม่แตะ `2026-10.parquet` ของดัชนีหลัก อย่างที่สองคือบรรทัดล่างสุดที่บอกวิธีเปิดดูต่อ ต้องใส่ `--kind codex` ให้ `serve` และ `tui` ด้วย ไม่เช่นนั้นมันจะไปเปิดดัชนีหลัก

### ขั้นที่ 2 — รันเฉพาะเดือนตุลาคม

อีกแกนหนึ่งคือเวลา ใส่ `--month 2026-10` เพื่อเอาแถวที่ลงวันที่ในเดือนนั้น (นับตามวันของกรุงเทพ) เท่านั้น

```
relic-scan run --month 2026-10
```




รูปที่ 18: ผล run --month 2026-10: อ่าน 144 ไฟล์ ทิ้ง 2426 แถวนอกช่วง ปลายทาง all@2026-10

บรรทัดแรกของผลลัพธ์บอกช่วงและปลายทางตั้งแต่ต้น

```
run: only rows from 2026-10-01 to 2026-10-31 (Bangkok days) -> …
```

ชื่อโฟลเดอร์ปลายทางคือ **`all@2026-10`** แปลว่าทุกเอเจนต์ (all) เฉพาะเดือน 2026-10 ถ้าใช้ `--kind` กับ `--month` ร่วมกัน ก็คาดได้ว่าชื่อสะท้อนทั้งสองอย่าง แต่เล่มนี้ไม่ได้รันคู่กัน จึงไม่ยืนยัน

## ขั้นที่ 3 — ดูว่าอะไรถูกทิ้ง

ตัวเลือก `--month` เปิดอ่านทุกไฟล์ แต่ทิ้งแถวที่ตกนอกช่วง

```
parse: 2426 rows outside 2026-10-01 to 2026-10-31 left out
  2026-10: 747 rows (747 new copies merged) -> …/
all@2026-10/2026-10.parquet
| 144 sessions · 747 rows · 144 files · 0.0 GB read                    |
```

เลข 747 แถว ตรงกับแถวของเดือน 2026-10 ในดัชนีหลักหลังไฟล์โต (745 บวก 2) และ 2426 ที่ถูกทิ้งก็คือ 3,173 ลบ 747 พอดี สองตัวเลขนี้ยืนยันว่าตัวเลือก `--month` ตัดถูกที่ ส่วนจำนวน session ยังเป็น 144 เพราะทั้ง 144 ไฟล์มีแถวในเดือนนี้อย่างน้อยหนึ่งแถว

ผลที่ได้จะเปิดดูด้วย `relic-scan serve --out ~/.relic-scan/all@2026-10` ตามที่กล่องบอกท้ายผลลัพธ์ ใช้ได้ดีเมื่ออยากส่งหรือดูเฉพาะเดือนหนึ่งโดยไม่ปนกับทั้งก้อน

## ต่อไป

ถึงตรงนี้ดัชนีเดโมมีครบ 3,173 แถว และรู้แล้วว่ารันซ้ำถูกแค่ไหน ยังไม่ได้ดูผลด้วยตาเลย บทที่ 3 เปิดหน้าเว็บบน 127.0.0.1 ให้เห็นแถวทั้งหมดเป็นรูปโดนัท เป็นแผนผังต้นไม้ และเป็นแผนที่ดาว

# บทที่ 3: เปิดดูในเว็บ

สองบทแรกอยู่ในเทอร์มินัลล้วน ๆ บทนี้ย้ายมาดูเป็นหน้าเว็บ relic-scan มีหน้าเว็บในตัวเสิร์ฟบน 127.0.0.1 เท่านั้น เปิดแล้วเห็นทั้งดัชนีเป็นภาพ ตั้งแต่ยอดรวม ต้นไม้ของ session ไปจนถึงแผนที่ดาว ทุกภาพในบทนี้คือหน้าจริงของบ้านเดโมที่เราสร้างไว้ตั้งแต่บทแรก ไม่มีข้อมูลจริงของใครปนอยู่

## เปิดเซิร์ฟเวอร์

### ขั้นที่ 1 — เปิดหน้าเว็บด้วย `up` หรือ `serve`

มีสองคำสั่งให้เลือก `relic-scan up` คือ `run` ก่อนแล้วค่อย `serve` ต่อ ส่วน `relic-scan serve` เสิร์ฟดัชนีที่มีอยู่เลยโดยไม่สแกนใหม่ พอ index ไว้แล้วตั้งแต่บทที่ 1 ใช้ `serve` ก็พอ ค่าเริ่มต้นคือพอร์ต 8787 เปลี่ยนได้ด้วย `--port`

```
relic-scan up                  # index แล้วเสิร์ฟ http://127.0.0.1:8787
relic-scan serve               # เสิร์ฟดัชนีที่มีอยู่
relic-scan serve --port 8791   # เปลี่ยนพอร์ต
```

ภาพทุกใบในเล่มนี้ถ่ายจากพอร์ต 8791 ส่วนคนที่ลองตามด้วยค่าเริ่มต้น ให้ใช้ 8787 แทน ที่อยู่ของแต่ละหน้าคือ `http://127.0.0.1:8787/<view>` เช่น `/scan` `/tree` `/map` `/paths` เซิร์ฟเวอร์ผูกกับ 127.0.0.1 เท่านั้น และตัวหน้าเว็บ **ไม่เรียกเน็ตเลย** ทุกอย่างอ่านจากดัชนีในเครื่อง

### ขั้นที่ 2 — รัน `run` ที่อื่นแล้ว reload

หน้าเว็บอ่านดัชนีทุกครั้งที่โหลด ถ้าเปิดเซิร์ฟเวอร์ค้างไว้แล้วไป `run` ในอีกเทอร์มินัล พอกลับมากด reload ก็เห็นของใหม่ทันที ไม่ต้องรีสตาร์ต นอกจากนั้นในหน้ายังมีปุ่ม ? **index** ที่สั่ง run แบบเพิ่มเฉพาะส่วนใหม่จากในเซิร์ฟเวอร์เองได้ ถ้ายังไม่เคย index เลย ทุก view จะขึ้นหน้าต้อนรับพร้อมปุ่มเดียวกัน

## Smart Scan — ภาพรวมทั้งหมด

### ขั้นที่ 1 — เปิดหน้าแรก `/scan`

เปิด `http://127.0.0.1:8787/scan` ซึ่งเป็นหน้าแรกของแอป (เปิดแค่ `/` ก็ได้) ฝั่งซ้ายคือเมนู เรียงจากบนลงล่างเป็น Smart Scan, Banks & Repos, ψ Vaults, Health, Scan log, Activity แล้วกลุ่ม Session map ที่มี Tree, Paths, Disk อยู่ข้างใน ท้ายสุดคือ Export กับ Settings กดเมนูแล้วสลับ view โดยไม่ต้องโหลดหน้าใหม่

กลางหน้ามีโดนัทใหญ่ ตรงกลางโดนัทเป็นยอดรวมของแถวทั้งหมด คือ **3,173 แถว** ใต้โดนัทมีแผ่นตัวเลขห้าอัน ROWS, SESSIONS, FILES INDEXED, NEWEST ROW และ SPAN




รูปที่ 19: หน้า Smart Scan: โดนัทแถวแยกตามชนิด ยอดรวม 3,173 และแผ่นตัวเลขด้านล่าง

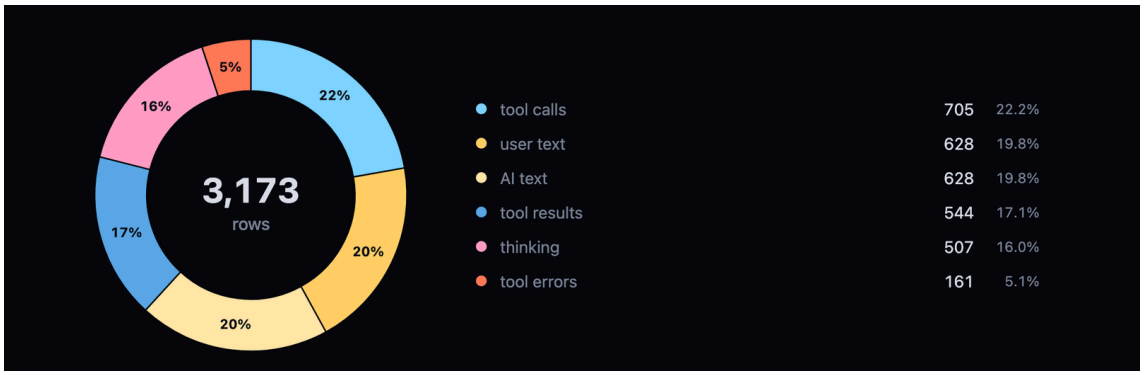
ตัวเลขชุดนี้คือชุดเดียวกับที่ DuckDB นับได้ในบทที่ 6 ถ้าสองที่นับไม่ตรงกันแปลว่ามีอะไรผิด เป็นวิธีเช็กที่เร็วดี

### ขั้นที่ 2 — อ่านโดนัทและคำอธิบายสี

โดนัทแบ่งแถวตามว่าแถวนั้นคืออะไร มีหกส่วน ได้แก่ tool calls, user text และ AI text ต่อด้วย tool results, thinking และ tool errors แต่ละส่วนมีจำนวนแถวกับเปอร์เซ็นต์กำ

กับ **หนึ่งแถวคือหนึ่งข้อความ หนึ่ง thinking block หนึ่งการเรียกเครื่องมือ หรือหนึ่งผลลัพธ์** ไม่ใช่หนึ่งบรรทัดของไฟล์

ภาพนี้ตัดมาเฉพาะโดนัทกับคำอธิบาย ขยายให้อ่านตัวเลขง่ายขึ้น




รูปที่ 20: โดนัทที่ตัดมาเฉพาะวงกับคำอธิบายสีของแต่ละชนิดแถว

### ขั้นที่ 3 — ปุ่ม Index now และ Check changes

หน้านี้มีสองปุ่มที่ควรรู้จัก **Check changes** คือการสแกนแบบ dry run เดินดูโฟลเดอร์เทียบขนาดกับ `_state.tsv` แล้วบอกว่าจะมีไฟล์ใหม่ ไฟล์โต ไฟล์ถูกเขียนใหม่ และไฟล์หายกี่ไฟล์ โดยไม่เขียนอะไรเลย เหมือน `run --dry-run` ในบทที่ 1

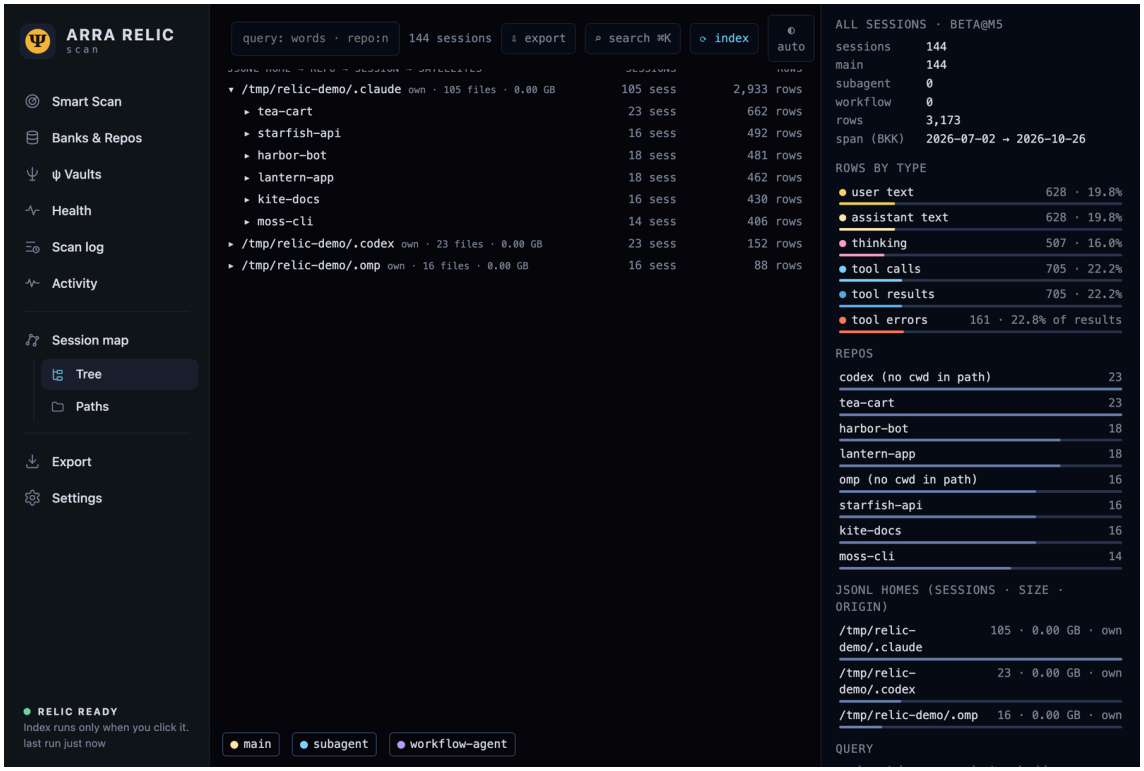
ส่วน Index now คือ `run` ที่ทำอยู่ในเซิร์ฟเวอร์ ระหว่างทำมีแถบความคืบหน้ากับปุ่ม Stop ถ้ากด Stop ตอนยังเดินดูโฟลเดอร์หรือยังอ่านอยู่ จะไม่มีอะไรถูกเขียน รอบหน้าก็อ่านไบต์ชุดเดิมซ้ำ แต่ถ้าเข้าขั้น merge แล้วมันจะทำจนจบ

### ขั้นที่ 4 — ค้นด้วย ⌘K

กด ⌘K หรือ `/` ได้จากทุกหน้า ช่องค้นหาเดียวครอบคลุม view, action, session (ค้นด้วย id, repo หรือ project) พาธของ `.jsonl` และรายการใน scan log พิมพ์ว่างไว้ก็ได้ มันโชว์ session ล่าสุดให้ เลือกผลลัพธ์แล้วเปิด view ที่เกี่ยวข้อง ช่องนี้เร็วกว่าไล่คลิกเมนูมาก

# Tree — บ้าน, repo, session

## ขั้นที่ 1 — เปิด `/tree`




รูปที่ 21: หน้า Tree: บ้าน .claude ที่มี 6 repo, ตามด้วย .codex และ .omp, แผงขวาสรุปยอด

เปิด `http://127.0.0.1:8787/tree` หน้านี้เรียงเป็นชั้น บ้าน (home) ของเอเจนต์ แล้ว repo แล้ว session บ้านแรกคือ `~/.claude` มี 6 repo ได้แก่ tea-cart, starfish-api, harbor-bot, lantern-app, kite-docs และ moss-cli ถัดลงไปเป็นบ้าน `.codex` กับ `.omp` **ทุกชั้นมีจำนวน session กับจำนวนแถวกำกับ** และมีแท่งแยกชนิดแถว

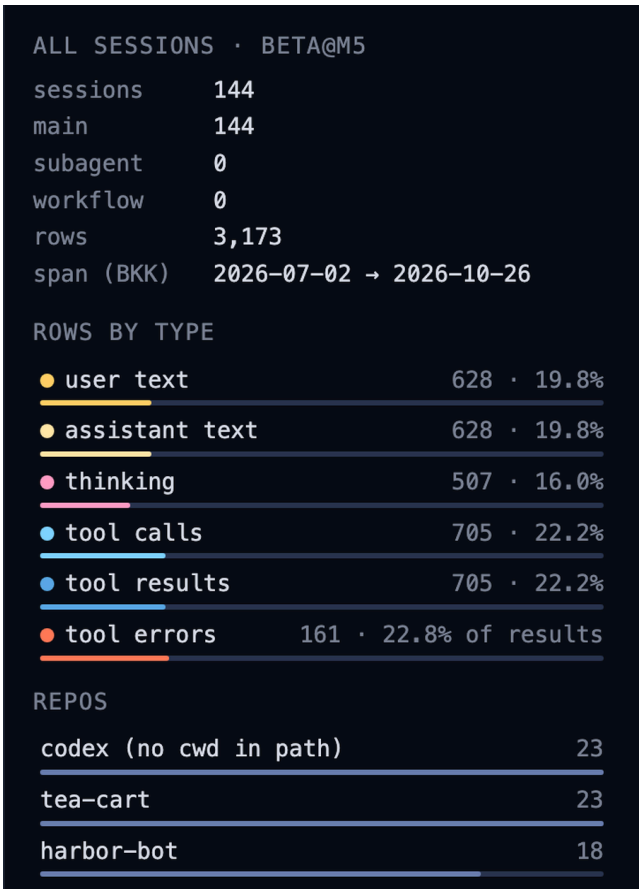
ฝั่งขวาคือแผงสรุป มีหัวข้อ ALL SESSIONS (ยอดรวม) ROWS BY TYPE (แท่งแยกตามชนิดแถว) REPOS (รายชื่อ repo) และ JSONL HOMES

session หนึ่งอาจอยู่ในหลายบ้าน ถ้ามีสำเนาในบ้านอื่น มันจะปรากฏใต้ทุกบ้านที่เก็บสำเนา แต่ในดัชนีนับเป็นแถวเดียว ตามที่ AGENTS.md บอกว่าสำเนาถูกพับรวมเป็นหนึ่งแถว

## ขั้นที่ 2 — อ่านแผงขวา ROWS BY TYPE

แท่ง ROWS BY TYPE คือโดนัทของ Smart Scan ในรูปแท่ง แต่ต่างกันตรงที่ **แผงนี้เปลี่ยนตามสิ่งที่เลือกอยู่** คลิก repo หนึ่งแล้วแผงก็นับเฉพาะ repo นั้น Tree กับ Map ใช้กล่องค้นหาและแผงข้างร่วมกัน คำค้นที่พิมพ์ไว้จึงตามไปอีกหน้าได้

ภาพนี้ตัดแผงขวามาให้ดูใกล้ ๆ




รูปที่ 22: แผงขวาของ Tree ที่ตัดมา: แท่งแถวแยกตามชนิดของ session ที่เลือก

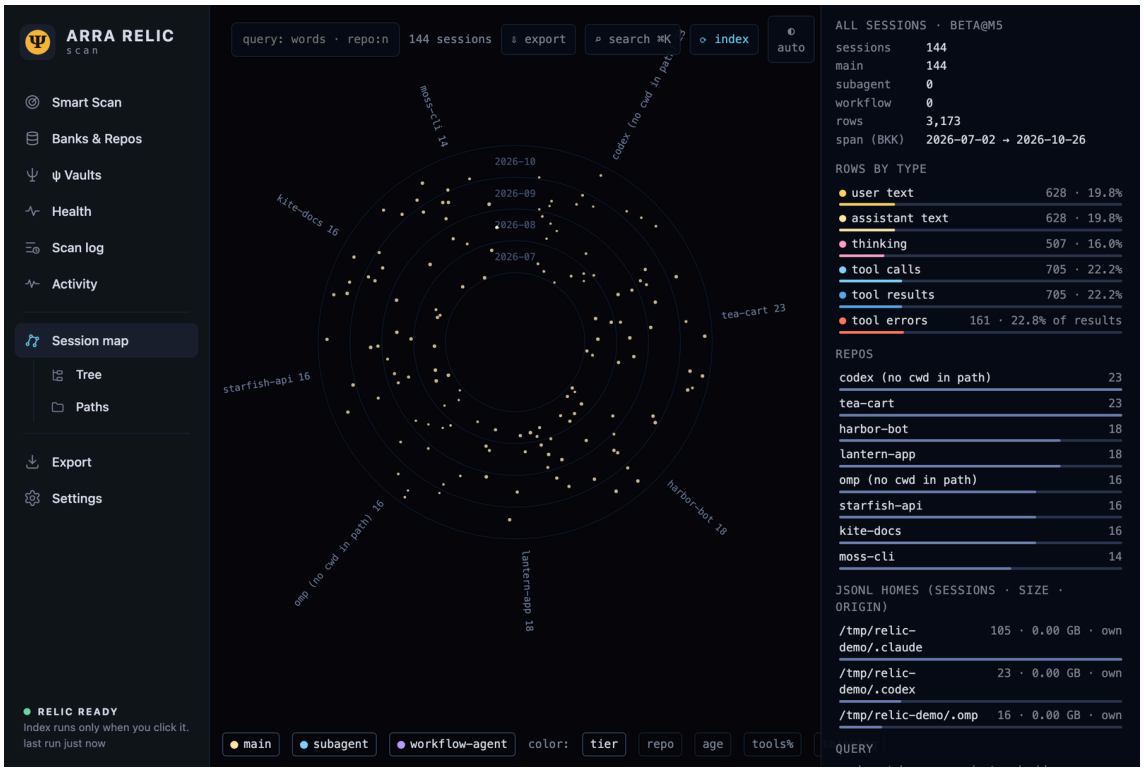
### ขั้นที่ 3 — กรองด้วยคำค้น

รายการใน REPOS และ JSONL HOMES ของแผงกดได้ กดแล้วคำค้นแคบลงทันที ถ้าอยากพิมพ์เอง คำค้นของ Map และ Tree รองรับตัวกรองอย่าง `repo:` `tier:main|sub|wf` `month:2026-07` และ `rows>1000` คำนำหน้า `-` กลับความหมาย และ **ทุกคำต้องตรงพร้อมกัน** (AND)

## Map — หนึ่งดาว หนึ่ง session

### ขั้นที่ 1 — เปิด `/map`

เปิด `http://127.0.0.1:8787/map` เห็นแผนที่ดาวกลม ๆ ทุก session คือจุดหนึ่งจุด วงแหวนคือเดือน นับจาก 2026-07 ถึง 2026-10 โดยเดือนเก่าสุดอยู่วงในสุด ส่วนซี่ที่แผ่ออกมาคือ repo ในเดโมมี moss-cli, codex, tea-cart, harbor-bot, lantern-app, omp, starfish-api และ kite-docs ปลายซี่แต่ละอันมีจำนวน session กำกับ




รูปที่ 23: หน้า Map: แผนที่ดาวกลม วงแหวนเป็นเดือน ซี่เป็น repo ปุ่มเลือกสีจุดอยู่ด้านล่าง

อ่านอย่างนี้ **ตำแหน่งบอกว่า session เริ่มเมื่อไรและอยู่ใน repo ไหน** ขนาดของดาวขึ้นกับจำนวนแถว (สเกล log) ซี่ที่กว้างกว่ามี session มากกว่า ส่วน subagent กับ workflow agent จะวนอยู่รอบ session แม่ของมัน ดาวที่อยู่ไกลจากศูนย์กลางจึงใหม่กว่าดาวที่อยู่วงใน

### ขั้นที่ 2 — เปลี่ยนสีจุด

แถบปุ่มด้านล่าง **เลือกสีของจุด** ได้สี่แบบ คือ tier, repo, age และ tools% แต่ละแบบตอบคำถามต่างกัน tier แยก session หลักออกจาก subagent, repo ให้สีตามซี่, age ไล่ตามอายุ และ tools% ชี้ว่า session ไหนใช้เครื่องมือหนักกว่าคุยธรรมดา

### ขั้นที่ 3 — คลิกดาวดูรายละเอียด

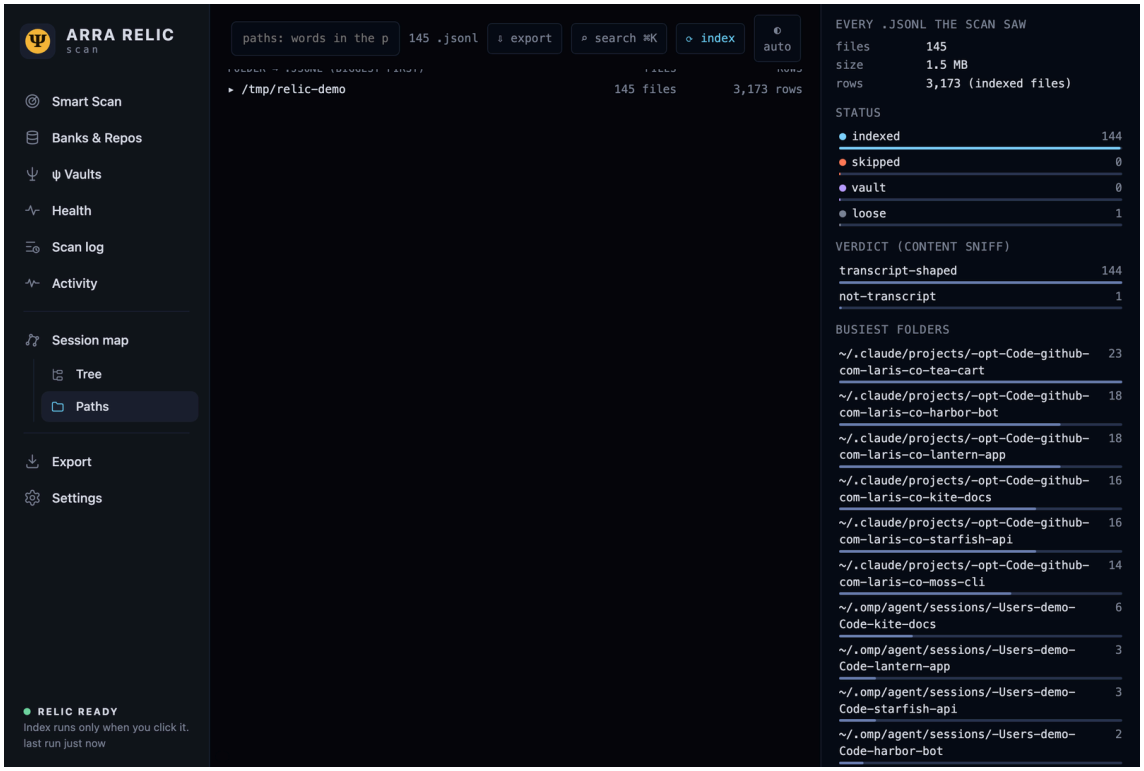
คลิกดาวหนึ่งดวงแล้วแผงข้างจะโชว์ session นั้น **ครบทั้งตัว** มี sid, repo, เวลา, แถวแยกตามชนิด, จำนวนสำเนา และปุ่มคัดลอก sid หรือบรรทัด `claude --resume` ไปวางต่อได้เลย นี่คือทางลัดจากการเห็นภาพ ไปสู่การกลับเข้า session จริง

## Paths — ทุก .jsonl ที่สแกนเห็น

### ขั้นที่ 1 — เปิด `/paths`

เปิด `http://127.0.0.1:8787/paths` หน้านี้ตอบคำถามที่ Tree ตอบไม่ได้ว่า ไฟล์ไหนที่สแกนเห็นแต่ **ไม่ได้ถูกอ่าน** เพราะอะไร เป็นต้นไม้ของโฟลเดอร์ ทุกโฟลเดอร์มีจำนวนไฟล์ จำนวนแถว ขนาด และแถบสถานะ โฟลเดอร์ที่มีทางเดียวจะถูกพับรวมเป็นแถวเดียว และ `~` แทนโฟลเดอร์บ้าน

ในเดโม โฟลเดอร์ที่พับอยู่มี 145 ไฟล์ 3,173 แถว แผงขวามีหัวข้อ STATUS, VERDICT และ BUSIEST FOLDERS




รูปที่ 24: หน้า Paths: ต้นไม้โฟลเดอร์ 145 ไฟล์ และแผงขวา STATUS, VERDICT, BUSIEST FOLDERS

### ขั้นที่ 2 — อ่าน STATUS กับ VERDICT

สองกล่องนี้ดูคล้ายกันแต่ **วัดคนละอย่าง** STATUS คือสิ่งที่ relic-scan ทำกับไฟล์ ในเดโมได้ indexed 144, skipped 0, vault 0 และ loose 1 โดย indexed คือไฟล์ที่ถูกอ่านเข้าเดือน, skipped คือไฟล์ในบ้านเอเจนต์ที่ไม่ใช่ transcript, vault คือไฟล์ใน ψ vault ซึ่งแค่แสดงรายการ, และ loose คือไฟล์ที่อยู่ที่อื่นใต้ root ก็แค่แสดงรายการเช่นกัน

VERDICT คือผลการดมเนื้อหา (content sniff) ว่าหน้าตาเป็น transcript หรือไม่ ได้ transcript-shaped 144 กับ not-transcript 1 รวมกันเป็น 145 ตรงกับจำนวนไฟล์ที่เห็นข้างบน

### ขั้นที่ 3 — ใช้ตัวกรองหาไฟล์ที่ถูกข้าม

ตัวกรองของหน้านี้ใช้ได้เช่น `is:indexed|skipped|vault|loose` และ `verdict:not-transcript` ยังมี `name:agent-` `size>1e9` `rows=0` และ `after:` กับ `before:` ซึ่งนับตามเวลาแก้ไขไฟล์ ลองพิมพ์ `verdict:not-transcript` ดู จะเห็นไฟล์ที่ไม่ใช่ transcript โผล่ขึ้นมา **คลิกไฟล์เพื่อดูสถานะ ขนาด จำนวนแถว** และปุ่มคัดลอกพาธ ถ้าไฟล์ที่คิดว่าควรถูกอ่านหายไป หน้านี้คือที่แรกที่ควรมาดู

คำค้นของ Paths ไม่ได้สร้างอะไรใหม่ มันแค่กรองรายการที่หน้าโหลดมาแล้ว เปลี่ยนคำค้นกี่รอบก็ไม่กระทบดัชนี ทุกอย่างบนหน้าเว็บเป็นการอ่าน ปุ่มเดียวที่เขียนจริงคือ Index now ซึ่งเขียนลงโฟลเดอร์ดัชนีของ relic-scan เอง ไม่แตะ transcript

หน้าเว็บที่เหลือคือการ์ดของแต่ละบ้าน แผนภูมิดิสก์ ห้องนิรภัยของ oracle หน้า Health และ Settings บทหน้าไล่ต่อให้ครบ

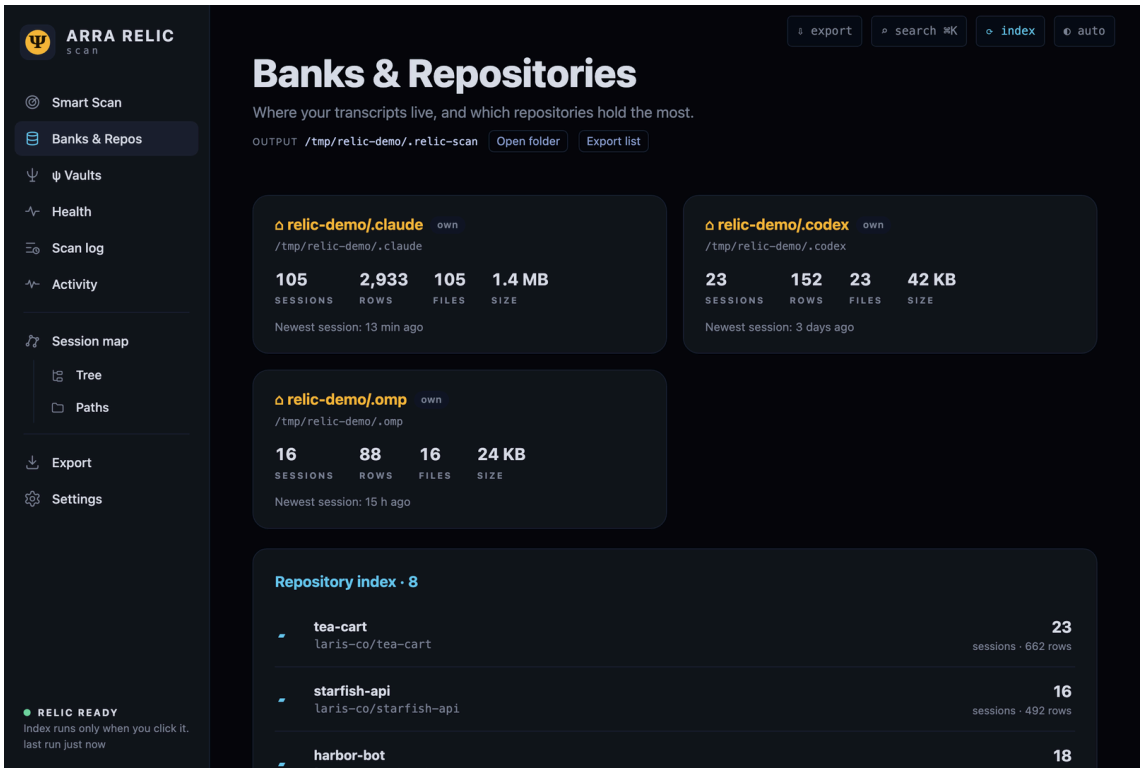
# บทที่ 4: ดิสก์ ห้องนิรภัย และการตั้งค่า

บทที่แล้วเราดูดัชนีจากมุมของแถวและ session บทนี้ดูจากมุมของ **ที่ที่ข้อมูลอยู่** บ้านของเอเจนต์แต่ละที่ ไฟล์ที่กินดิสก์ ห้องนิรภัย ψ ของ oracle แล้วปิดท้ายด้วยหน้า Health ที่บอกว่าทั้งหมดปกติดีไหม และ Settings ที่ปรับสิ่งที่สแกนเดินดู เซิร์ฟเวอร์ยังเป็นตัวเดิม ที่อยู่ยังเป็น `http://127.0.0.1:8787/<view>` ส่วนภาพในบทนี้ถ่ายจากพอร์ต 8791 เหมือนบทที่แล้ว

## Banks & Repos — การ์ดต่อหนึ่งบ้าน

### ขั้นที่ 1 — เปิด `/banks`

เปิด `http://127.0.0.1:8787/banks` หรือกด Banks & Repos ในเมนูซ้าย ด้านบนเป็นการ์ดหนึ่งใบต่อหนึ่ง **บ้านของเอเจนต์** (agent home) ในเดโมมีสามใบ คือ `.claude`, `.codex` และ `.omp` แต่ละการ์ดบอก sessions, rows, files, ขนาด และ session ล่าสุด




รูปที่ 25: หน้า Banks & Repos: การ์ดของ .claude, .codex, .omp และตาราง Repository index 8 repo

ใต้การ์ดเป็น Repository index มี 8 repo แต่ละบรรทัดบอกจำนวน session กับจำนวนแถวของ repo นั้น

### ขั้นที่ 2 — กดการ์ดหรือบรรทัดเพื่อไปต่อ

หน้านี้เป็นทางเข้าไปยังหน้าอื่น **กดการ์ดแล้วเปิด Tree ที่บ้านนั้น** ส่วนกดบรรทัดใน Repository index แล้วเปิด Map ที่ repo นั้น ส่วนตัวเลขในการ์ดคือยอดของบ้านนั้นทั้งบ้าน

README บอกว่าการ์ดมีช่อง origin ด้วย ซึ่งมีประโยชน์ตอนที่เครื่องมี mirror หรือ backup ของ `.claude` หลายที่ relic-scan พับแถวซ้ำให้เป็นแถวเดียว แต่ยังแสดงให้เห็นว่าบ้านไหนถือไฟล์อยู่

ถ้าอยากรู้ว่าเครื่องนี้มีเอเจนต์ตัวไหนทิ้งประวัติไว้มากที่สุด ดูที่ rows ของแต่ละการ์ดแล้วเทียบกัน จบในหน้าเดียว ไม่ต้องไปเขียน query

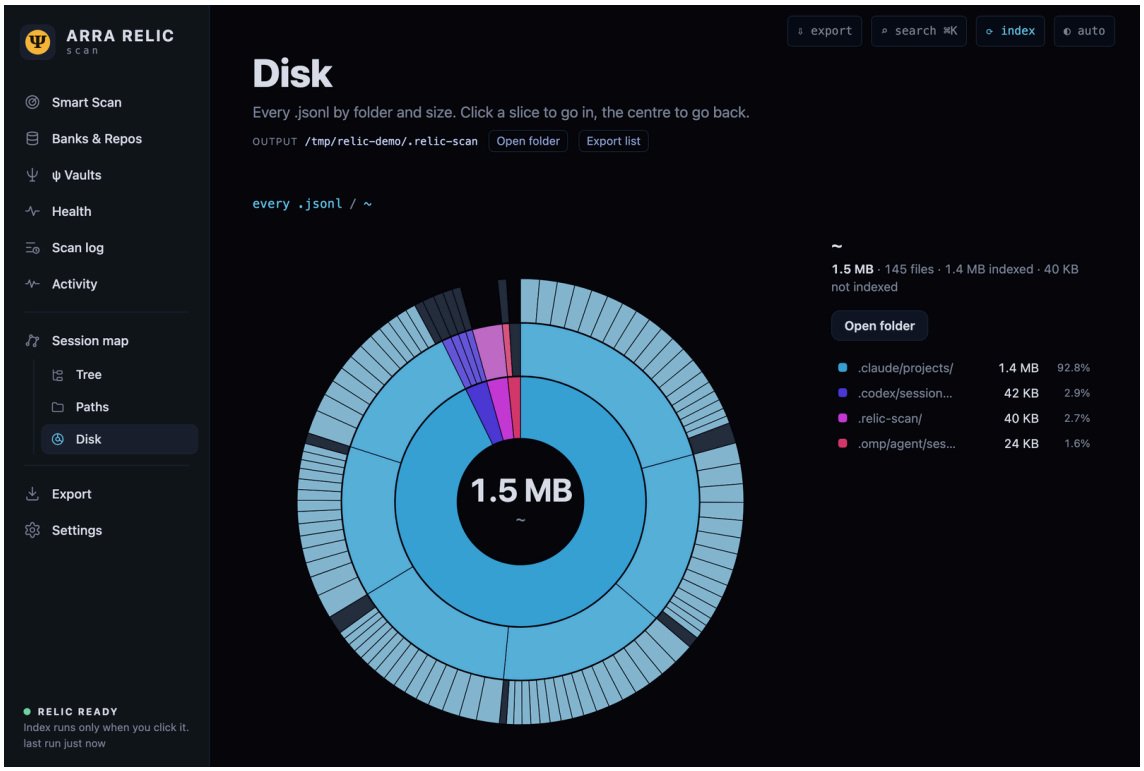
## Disk — ไฟล์ไหนกินที่

### ขั้นที่ 1 — เปิดตัวเลือก Disk ก่อน

README เรียก Disk ว่า gem คือ **ซ่อนไว้จนกว่าจะเปิดเอง** ใน Settings > Gems และเปิดเฉพาะเบราว์เซอร์นั้น ถ้าไม่เห็นเมนู Disk ใต้ Session map ให้ไปที่หน้า Settings (อยู่ท้ายบทนี้) แล้วเปิดตรงนั้นก่อน ภาพด้านล่างถ่ายจากการเปิดที่อยู่ `/disk` ตรง ๆ

### ขั้นที่ 2 — เปิด `/disk`

เปิด `http://127.0.0.1:8787/disk` ได้แผนภูมิวงกลมหลายชั้น (sunburst) ของ `.jsonl` ทุกไฟล์ จัดตามโฟลเดอร์และขนาด ตรงกลางวงคือยอดรวม **1.5 MB** วงในแทนโฟลเดอร์ใหญ่ วงที่ออกไปแตกเป็นโฟลเดอร์ย่อย มุมของแต่ละชิ้นคือสัดส่วนของโฟลเดอร์แม่




รูปที่ 26: หน้า Disk: แผนภูมิ sunburst ของ .jsonl ทุกไฟล์ ตรงกลาง 1.5 MB พร้อมคำอธิบายสีของแต่ละโฟลเดอร์

ใช้งานง่ายมาก คลิกชิ้นไหนก็ซูมเข้าไปในโฟลเดอร์นั้น คลิกตรงกลางเพื่อถอยออกมา ชิ้นเล็กมากที่แต่ละชิ้นไม่ถึง 0.5% ของตรงกลาง จะถูกรวมเป็นชิ้นสีเทา “smaller objects”

### ขั้นที่ 3 — อ่านคำอธิบายสี

คำอธิบายด้านข้างเรียงตามสัดส่วนในเดโม

```
.claude/projects   92.8%
.codex/sessions     2.9%
.relic-scan         2.7%
.omp                1.6%
```

ส่วนใหญ่อยู่ใน `.claude/projects` ซึ่งสมเหตุสมผล เพราะ transcript ของ Claude Code เป็น 144 ไฟล์จากทั้งหมด 145 ที่เห็น ที่น่าสนใจคือ `.relic-scan` ก็ติดมาในรายการ เพราะเป็นโฟลเดอร์ที่มี `.jsonl` อยู่ในนั้น ดังนั้นตัวเก็บดัชนีเองก็กินที่ให้เห็นในภาพเหมือนกัน

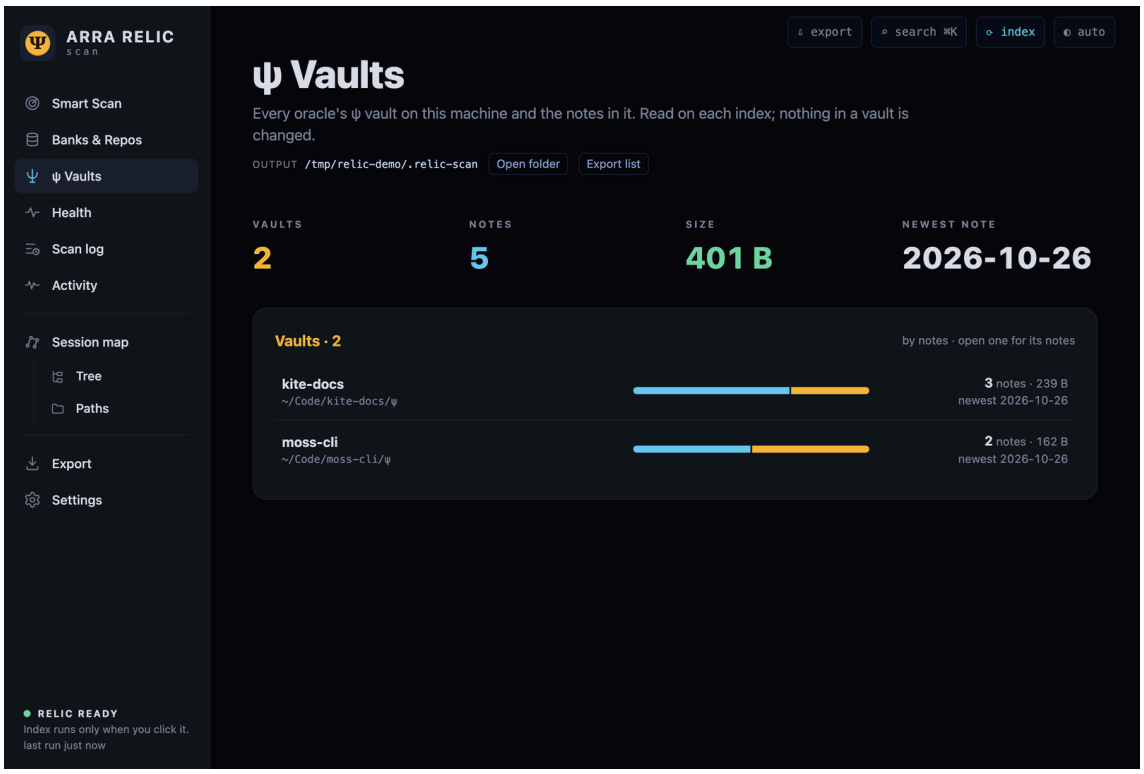
เครื่องจริงกว่านี้มาก AGENTS.md ยกตัวอย่างเครื่อง Mac เครื่องหนึ่งที่เจอ 87,000 ไฟล์ 75 GB ในเครื่องขนาดนั้น การเห็นเป็นภาพว่าโฟลเดอร์ไหนกินที่มากสุดช่วยได้เยอะ ก่อนจะลงมือจัดการอะไรก็ตาม

## ψ Vaults — ห้องนิรภัยของ oracle

### ขั้นที่ 1 — เปิด `/vaults`

ψ vault คือโฟลเดอร์ความจำของ oracle ที่อยู่ใน repo เช่น `~/Code/kite-docs/ψ` relic-scan หาเจอแล้วทำรายการให้ แต่ **อ่านอย่างเดียว** ไม่แก้และไม่ย้ายอะไรเลย เปิด `http://127.0.0.1:8787/vaults` ด้านบนเป็นแผ่นตัวเลข VAULTS 2, NOTES 5, SIZE 401 B และ NEWEST NOTE 2026-10-26

ใต้แผ่นตัวเลขเป็นรายชื่อ vault สองอัน คือ kite-docs มี 3 โน้ต และ moss-cli มี 2 โน้ต แต่ละอันมีแท่งสีและพาธ `~/Code/kite-docs/ψ`




รูปที่ 27: หน้า ψ Vaults: แผ่นตัวเลข VAULTS 2 NOTES 5 และรายการ kite-docs กับ moss-cli

### ขั้นที่ 2 — ดูข้อมูลชุดเดียวกันเป็น JSON

หน้าเว็บนี้อ่านจากไฟล์ `meta/vaults.json` ในโฟลเดอร์ดัชนี เลยถามด้วย `jq` ได้เลย คำสั่งนี้ดึงจำนวนโน้ต ชื่อ และพาธของแต่ละ vault ออกมาเป็นตาราง

```
$ jq -r '.vaults[] | [.notes, .name, .path] | @tsv' \
    ~/.relic-scan/meta/vaults.json
```

```
$ jq -r '.vaults[] | [.notes, .name, .path] | @tsv' ~/.relic-scan/meta/vaults.json
3    kite-docs    ~/Code/kite-docs/ψ
2    moss-cli    ~/Code/moss-cli/ψ
(exit 0 · 0.0 s)
```

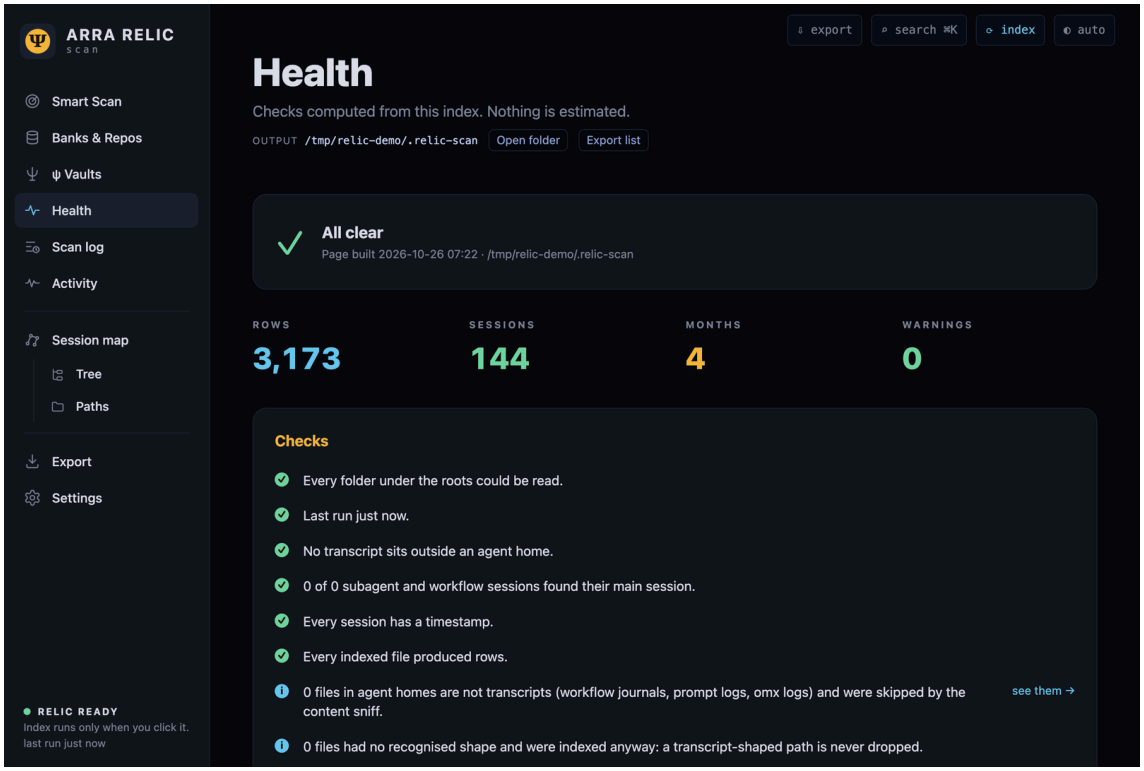
รูปที่ 28: ผลของ jq: kite-docs 3 โน้ต และ moss-cli 2 โน้ต พร้อมพาธ

ตัวเลขตรงกับหน้าเว็บ 3 บวก 2 ได้ 5 โน้ตตามแผ่น NOTES ถ้าอยากได้โน้ตรายตัว ไฟล์ `meta/vaults/<id>.json` เก็บแต่ละโน้ตเป็น path, section, title, ขนาด และเวลาแก้ไข

## Health — ปกติดีไหม

### ขั้นที่ 1 — เปิด `/health`

เปิด `http://127.0.0.1:8787/health` ในเดโมหัวหน้าขึ้นว่า All clear ใต้นั้นเป็นแผ่นตัวเลข ROWS 3,173, SESSIONS 144, MONTHS 4 และ WARNINGS 0 **WARNINGS 0 คือตัวที่ต้องดูก่อน** ถ้ามีเลขโผล่ แปลว่ามีข้อใดข้อหนึ่งด้านล่างไม่ผ่าน




รูปที่ 29: หน้า Health: All clear แผ่นตัวเลข 3,173 แถว 144 session 4 เดือน 0 คำเตือน และรายการเช็กที่ติ๊กเขียว

### ขั้นที่ 2 — อ่านรายการ Checks

ใต้แผ่นเป็นรายการเช็กที่คำนวณจากดัชนี แต่ละข้อติ๊กเขียวเมื่อผ่าน ในเดโม **ผ่านหมดทุกข้อ** คือ อ่านทุกโฟลเดอร์ได้, รอบล่าสุดเพิ่งรันเมื่อครู่, ไม่มี transcript อยู่นอกบ้านเอเจนต์, ทุก session มีเวลา และทุกไฟล์ที่ index ให้แถวออกมา

ข้อมูลที่ README อธิบายไว้ เช็กอื่นที่หน้านี้ดูให้ได้อีก มีอายุของรอบล่าสุด, subagent ที่ไม่มี session หลัก, ไฟล์ที่ถูกข้ามเพราะไม่ใช่ transcript, รูปทรงที่ไม่รู้จัก, mirror ที่ถูกพับ และ tool errors แต่ละข้อมีลิงก์ไปยัง view ที่เกี่ยวข้อง พอมีข้อไม่ผ่าน กดตามไปก็ถึงไฟล์หรือ session ที่เป็นต้นเรื่อง

แผ่น MONTHS 4 **ตรงกับ**สี่เดือนที่เห็นบนวงแหวนของ Map คือ 2026-07 ถึง 2026-10 และ SESSIONS 144 ตรงกับจำนวน transcript ที่ Paths นับเป็น indexed สามหน้าที่นับจากดัชนีเดียวกันจึงต้องตรงกัน ถ้าเลขไม่ตรง ให้สงสัยว่าเปิดคนละโฟลเดอร์ดัชนี เช่น ลืม `--out` หรือ `--kind`

ข้อที่ผ่านแต่มีข้อมูลประกอบจะเป็นบรรทัด info ไม่ใช่คำเตือน อ่านเป็นความรู้ ไม่ต้องแก้

## Settings — สิ่งที่สแกนเดินดู

### ขั้นที่ 1 — เปิด `/settings`

เปิด `http://127.0.0.1:8787/settings` หน้านี้ตั้งค่าว่า **การสแกนเดินดูอะไร** มีสองส่วนในภาพ ส่วนแรก What to scan คือโฟลเดอร์ที่ให้เดิน (ในเดโมคือ `~`) พาธที่ให้ข้าม และความลึก (depth) ที่ตั้งไว้ 8 ส่วนที่สอง S3 copy คือ destination, endpoint และ region

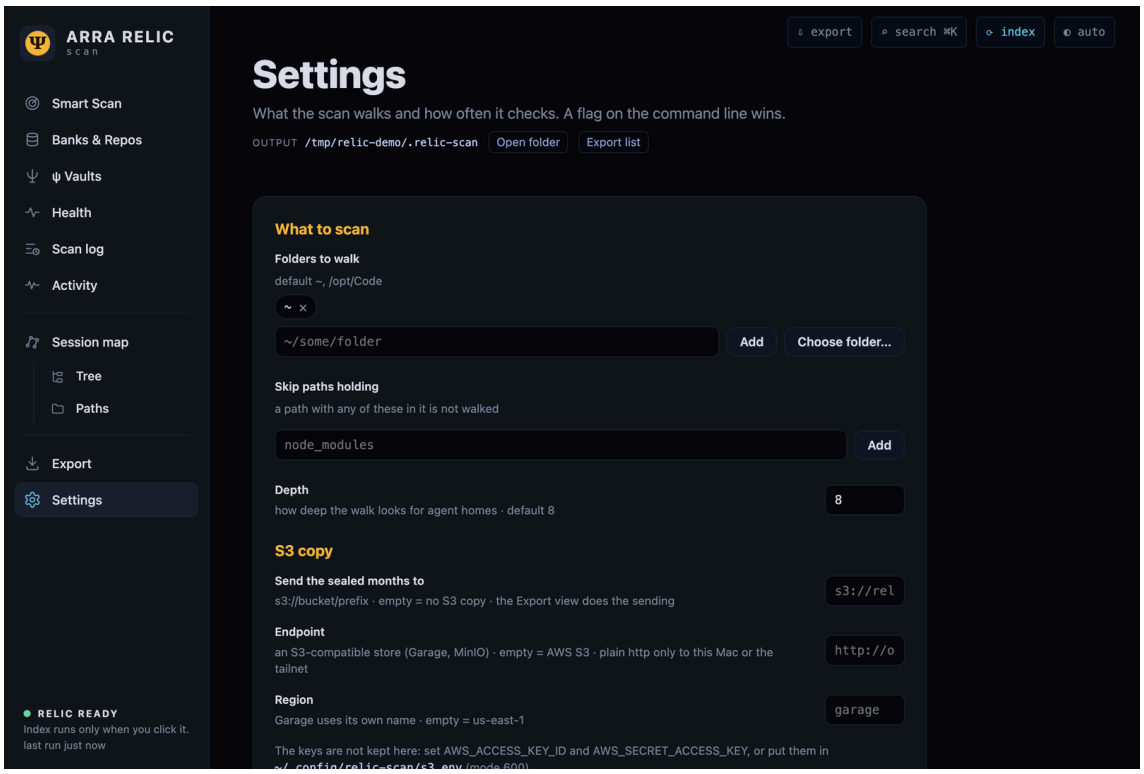
### ขั้นที่ 2 — แก้โฟลเดอร์และพาธเป็นชิป

โฟลเดอร์กับพาธที่ข้ามแสดงเป็นชิป (chip) กด × ลบทีละอัน หรือพิมพ์ในกล่องแล้วกด Enter เพิ่ม มีคำแนะนำให้เลือก และมีปุ่ม ปุ่ม Choose folder… ที่เปิดตัวเลือกโฟลเดอร์ของเครื่องผ่านเซิร์ฟเวอร์ เซิร์ฟเวอร์ตรวจให้ว่าโฟลเดอร์มีจริง และ depth อยู่ในช่วง 1 ถึง 32 ถ้าผิดมันบอกว่าผิดตรงไหน

ค่าทั้งหมดถูกเก็บใน `_config.json` ในโฟลเดอร์ดัชนี `relic-scan run` กับปุ่ม Index now ใช้ไฟล์เดียวกัน แต่ถ้าสั่งด้วยแฟลกในบรรทัดคำสั่ง **แฟลกชนะเสมอ**

### ขั้นที่ 3 — S3 copy และกุญแจ

ช่อง S3 copy ใส่ได้แค่ปลายทาง endpoint กับ region ส่วนกุญแจ **ไม่เก็บในหน้านี้เด็ดขาด** กุญแจมาจาก environment หรือไฟล์ `~/.config/relic-scan/s3.env` ที่มีสิทธิ์ 600 เท่านั้น เรื่องการส่งจริงอยู่ในบทที่ 7




รูปที่ 30: หน้า Settings: What to scan คือโฟลเดอร์ ~ พาธที่ข้าม depth 8 และส่วน S3 copy

ยังมีตัวเลือก Check changes อัตโนมัติทุก 15, 30 หรือ 60 นาทีตอนเซิร์ฟเวอร์ทำงานอยู่ มันแค่ตรวจ ไม่เคย index เอง และมีปุ่มเลือกธีม (auto, dark, light) ซึ่ง auto ตามระบบ

หกหน้าเว็บที่ผ่านมาทั้งสองบทครอบคลุมเกือบทั้งแอปแล้ว แต่บางคนไม่อยากเปิดเบราว์เซอร์ บทหน้าจึงพาไปดูดัชนีเดียวกันในเทอร์มินัลด้วย `relic-scan tui`

# บทที่ 5: ในเทอร์มินัล

สองบทก่อนหน้านี้เราเปิดดัชนีผ่านหน้าเว็บ แต่บางครั้งเราก็ไม่อยากออกจากเทอร์มินัล เช่น ต่อ ssh เข้าเครื่องอื่น หรือแค่อยากเช็กเร็ว ๆ ว่า session ล่าสุดของ repo นี้คืออะไร relic-scan มีหน้าจอเทอร์มินัลให้ใช้ในตัว ชื่อ `relic-scan tui` อ่านดัชนีชุดเดียวกับหน้าเว็บ ไม่ได้สร้างข้อมูลใหม่ และไม่แตะ transcript เลยแม้แต่ไฟล์เดียว

บทนี้เราจะเดินตามคีย์เป็นลำดับเดียวกับที่ใช้ถ่ายภาพ คือ เปิดโปรแกรม, กด `2`, กด Enter, กด Backspace, กด `3`, พิมพ์ `/` ตามด้วย `starfish`, กด Esc, กด Enter, กด Esc อีกครั้งเพื่อปิดกล่อง, กด `4` และกด `6` ทุกหน้าจอในบทนี้เป็นข้อความจริงของหน้าจอขนาด 100x30 บนบ้านเดโมเดิม ไม่มีอะไรตกแต่งเพิ่ม

## 5.1 เปิด relic-scan tui และอ่านแท็บแรก

### ขั้นที่ 1 — เปิดโปรแกรม

พิมพ์คำสั่งเดียวในโฟลเดอร์ไหนก็ได้ เพราะโปรแกรมหาดัชนีที่ `~/.relic-scan` เอง ถ้าเคยรันด้วย `--kind` หรือ `--month` ไว้ (บทที่ 2) ก็ใส่ตัวเลือกเดียวกันตอนเปิดเพื่อชี้ไปโฟลเดอร์แยกนั้น เช่น `relic-scan tui --kind claude` ตัวเลือกทั้งหมดดูได้ในขั้นที่ 4

```
relic-scan tui
```




รูปที่ 31: หน้าจอ overview ของ relic-scan tui: totals, rows by type และแผนภูมิ sessions per month

พอเปิดมา **หน้าแรกคือ overview** เสมอ ที่หัวจอบอกว่าดัชนีนี้มีอะไรอยู่ แล้วบรรทัดถัดไปเป็นแถบแท็บทั้งหกให้เห็นตั้งแต่แรก

```
RELIC·SCAN   144 sessions · 145 .jsonl · ~/.relic-scan
 1 overview   2 repos   3 sessions   4 paths   5 s3   6 ops
```

## ขั้นที่ 2 — อ่านสามกล่องของ overview

กล่องซ้ายบน `totals` สรุปตัวเลขรวม กล่องขวาบน `rows by type` แยกแถวตามชนิด ส่วนกล่องล่างเป็นแผนภูมิแท่ง **sessions per month** ตัวเลขทั้งหมดตรงกับที่หน้าเว็บ Smart Scan เคยโชว์ คือ 3,173 แถว และ 144 session

```
rows        3,173
indexed     144 files · 1.4 MB
span (BKK)  2026-07-02 → 2026-10-26
repos       8
```

ในกล่อง `totals` มีสามบรรทัดย่อยของ sessions คือ `main 144`, `subagent 0` และ `workflow 0` บ้านเดโมมีแต่ session หลัก ไม่มี subagent เลย ส่วน `rows by type` มี user text 628, assistant text 628, thinking 507, tool calls 705 และ tool results

705 เรียงเป็นแท่งพร้อมเปอร์เซ็นต์ บรรทัดสุดท้าย tool errors 161 คิดเป็น 22.8% ของ results ไม่ใช่ของแถวทั้งหมด ตัวหารต่างกัน จึงเขียนกำกับไว้ในภาพว่า `of results`

แผนภูมิล่างแสดงสี่เดือน 2026-07 ถึง 2026-10 ใต้แท่งเขียนจำนวน session ไว้สามสิบกว่าต่อเดือน (37, 37, 37 และ 33) เดือนสุดท้ายแท่งเตี้ยกว่านิดหน่อย ถ้ารวมสี่เดือนจะได้ 144 พอดีกับกล่อง totals

### ขั้นที่ 3 — แถบคีย์ที่ท้ายจอ

บรรทัดล่างสุดเป็นความจำช่วยตลอดเวลา **ไม่ต้องจำคีย์จากบทนี้ก็ได้** เพราะมันอยู่บนจอทุกแท็บ

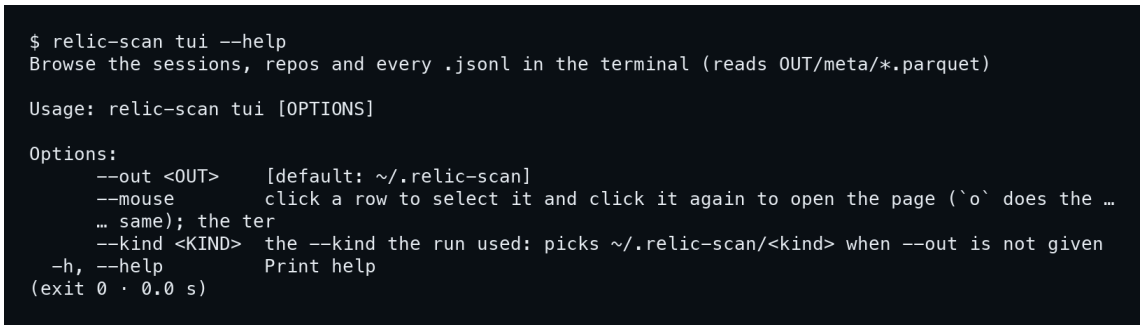
```
 q quit  1–6/tab view  / filter  ↑↓ jk g G move  enter open  o page  ⌫
back
```

คีย์ `1` ถึง `6` กระโดดไปแท็บตรง ๆ ส่วน Tab กับลูกศรขวาไล่ไปแท็บถัดไป Shift+Tab กับลูกศรซ้ายถอยกลับ การเลื่อนในรายการใช้ลูกศรขึ้นลง หรือ `j` `k` แบบ vim ก็ได้ `g` ไปบนสุด `G` ไปล่างสุด PgUp กับ PgDn ข้ามทีละ 20 แถว และ `q` ออกจากโปรแกรม

### ขั้นที่ 4 — ดูตัวเลือกทั้งหมดด้วย --help

ถ้าลืมว่าเปิดโปรแกรมได้แบบไหนบ้าง ให้ถาม `--help` คำสั่งนี้พิมพ์แล้วจบทันที ไม่เปิดหน้าจอเต็ม

```
relic-scan tui --help
```




รูปที่ 32: ผลของ relic-scan tui --help: บรรทัดอธิบายโปรแกรม Usage และตัวเลือก \--out, --mouse, --kind, --help

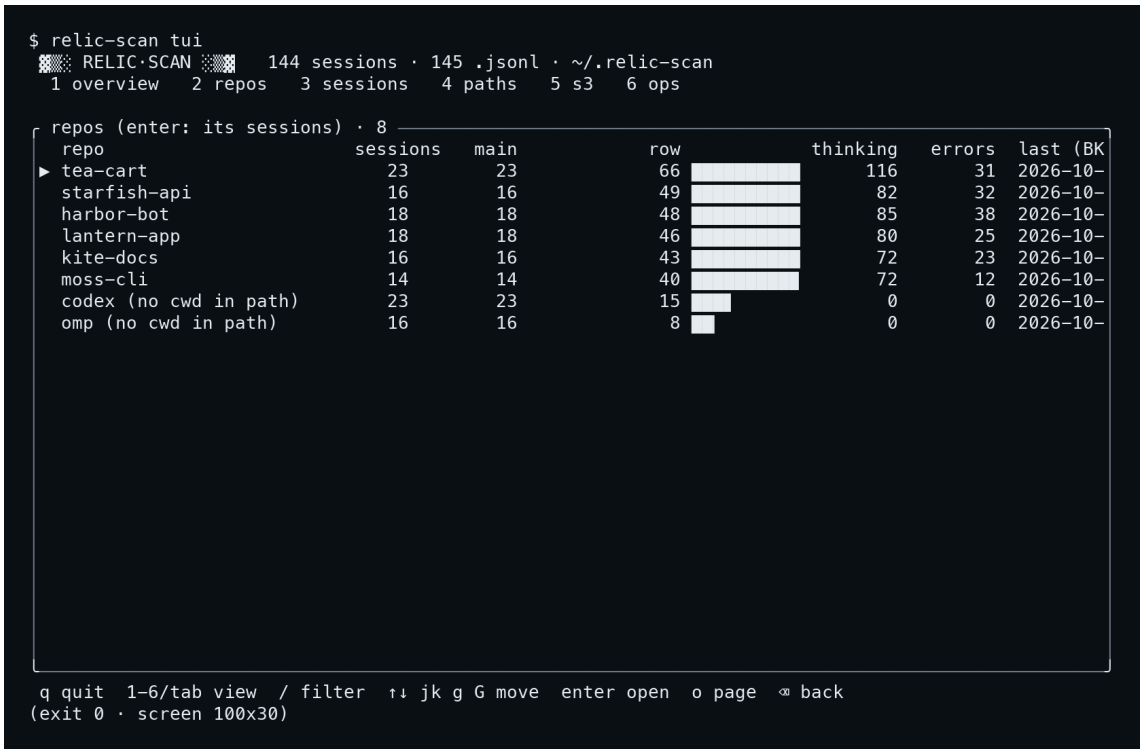
อ่านแล้วเห็นว่า **ตัวเลือกมีแค่สามตัว** `--out` ชี้โฟลเดอร์ดัชนี (ค่าเริ่มต้น `~/.relic-scan`), `--mouse` ให้คลิกแถวเพื่อเลือกแล้วคลิกซ้ำเพื่อเปิดหน้าเว็บ และ `--kind` ใช้ชื่อ kind เดียวกับที่ตอน `run` ใช้ เพื่อเลือกโฟลเดอร์ `~/.relic-scan/<kind>` เมื่อไม่ได้ใส่ `--out`

## 5.2 จาก repo ลงไปถึง session

### ขั้นที่ 1 — กด 2 ดู repo ทั้งแปด

กด `2` เพื่อเข้าแท็บ repos แต่ละแถวคือหนึ่ง repo มีจำนวน session แยกตาม main, แถว, thinking, errors และเวลา session ล่าสุด

```
┌ repos (enter: its sessions) · 8 ──────────────
│▶ tea-cart                       23        23
│  starfish-api                   16        16
│  codex (no cwd in path)         23        23
│  omp (no cwd in path)           16        16
```




รูปที่ 33: แท็บ repos: แปด repo เรียงตามจำนวนแถว มี tea-cart อยู่บนสุด

**รายการเรียงตามจำนวนแถว ไม่ใช่ตามจำนวน session** tea-cart จึงอยู่บนสุดด้วย 23 session ส่วน codex กับ omp อยู่ท้ายสุดแม้ codex จะมี 23 session เท่า tea-cart เพราะ session ของมันสั้น (ดูตัวเลข 152 และ 88 แถวในบทที่ 6) ลูกศร `▶` หน้าแถวคือแถวที่เลือกอยู่

ข้อสังเกตสองอย่าง อย่างแรก เครื่องหมาย (no cwd in path) บอกว่า Codex และ omp ไม่ได้ฝังชื่อโฟลเดอร์งานไว้ใน path ของไฟล์ relic-scan จึงจัดเป็นกลุ่มตามชื่อ agent อย่างที่สอง จอกว้างแค่ 100 ตัวอักษร คอลัมน์ขวาสุดจึงถูกตัด เช่น หัวคอลัมน์ `rows` กลายเป็น

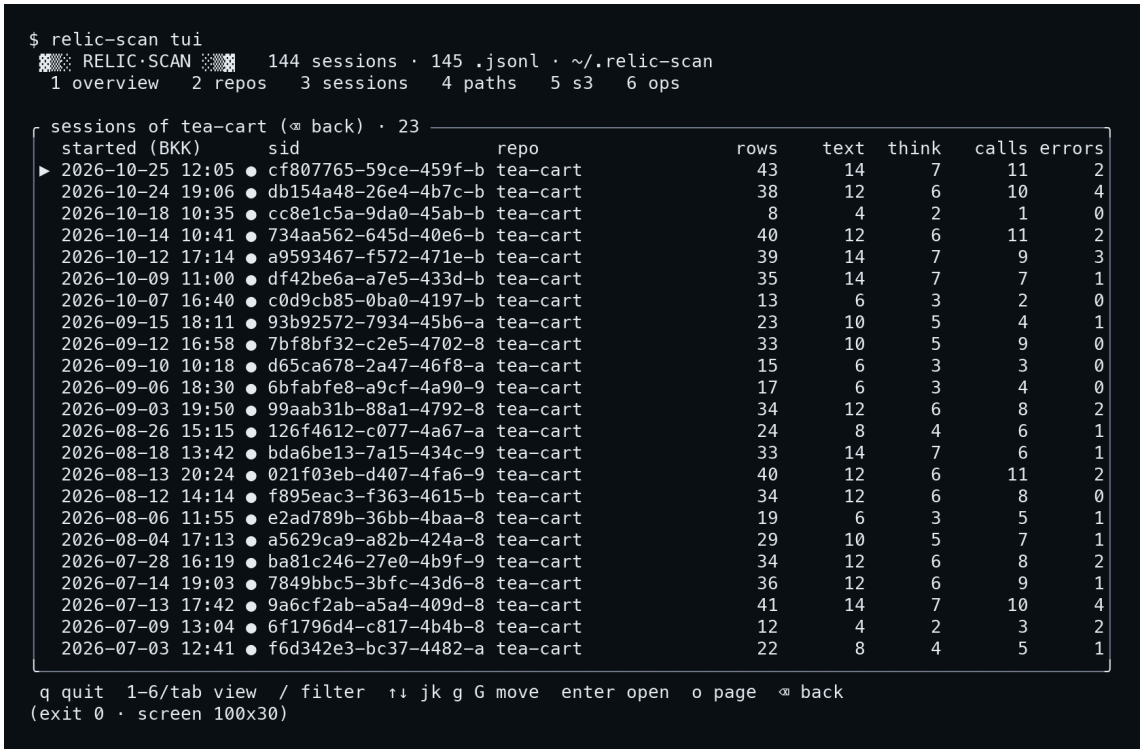
`row` และตัวเลขแถวของ tea-cart ที่ควรเป็น 662 ถูกตัดเหลือ `66` ถ้าอยากเห็นเต็ม ให้ขยายหน้าต่างเทอร์มินัลให้กว้างกว่านี้

### ขั้นที่ 2 — กด Enter ลง repo ที่เลือก

เลื่อน `▶` ไปที่ repo ที่ต้องการแล้วกด **Enter** ในภาพนี้เราไม่ได้เลื่อน จึงเข้า tea-cart ซึ่งเป็นแถวแรก

โปรแกรมสลับไปแท็บ sessions แต่กรองให้เหลือ repo เดียว ชื่อกล่องบอกไว้ชัดว่า `sessions of tea-cart (⌫ back) · 23`

```
|▶ 2026-10-25 12:05 ● cf807765-59ce-459f-b tea-cart    43   14   7  11
2|
|  2026-10-24 19:06 ● db154a48-26e4-4b7c-b tea-cart    38   12   6  10
4|
```



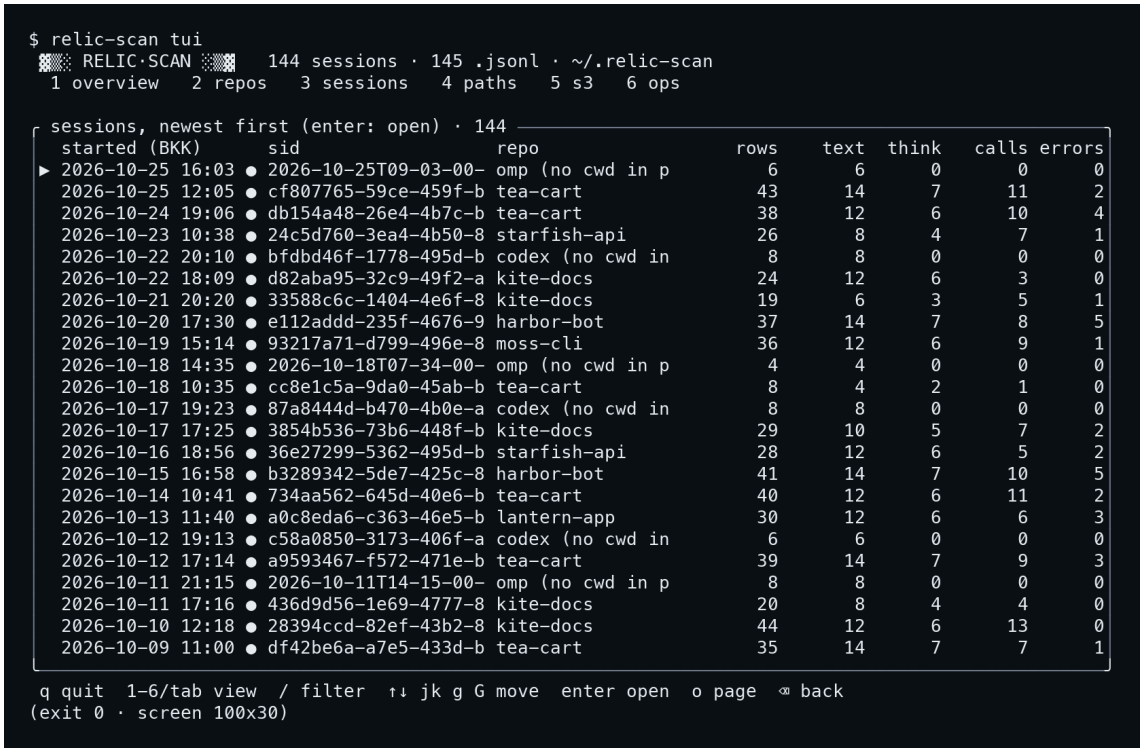

รูปที่ 34: รายการ 23 session ของ tea-cart เรียงจากใหม่ไปเก่า

คอลัมน์ `started (BKK)` เป็นเวลาไทย `sid` ถูกตัดเหลือ 20 ตัวแรกของ id ส่วน `rows` คือแถวทั้งหมดของ session นั้น แล้ว `text`, `think`, `calls`, `errors` แจกแจงว่าแถวเหล่านั้นเป็นอะไร session บนสุดมี 43 แถว เป็นข้อความ 14, thinking 7, tool calls 11 และ error 2 จุดกลม `●` หลังเวลาบอกว่าเป็น session หลัก

### ขั้นที่ 3 — Backspace กลับ แล้วกด 3 ดูทุก session

กด Backspace (ในแถบคีย์เขียนว่า `⌫ back`) เพื่อถอยกลับไปแท็บ repos จากนั้นกด `3` เพื่อไปแท็บ sessions แบบไม่กรอง กดเลขสามจะล้างการเลือก repo ทิ้งให้ด้วย จึงเห็นทั้ง 144 session

```
┌ sessions, newest first (enter: open) · 144 ──
```




รูปที่ 35: แท็บ sessions ทั้งหมด 144 รายการ เรียงใหม่สุดก่อน

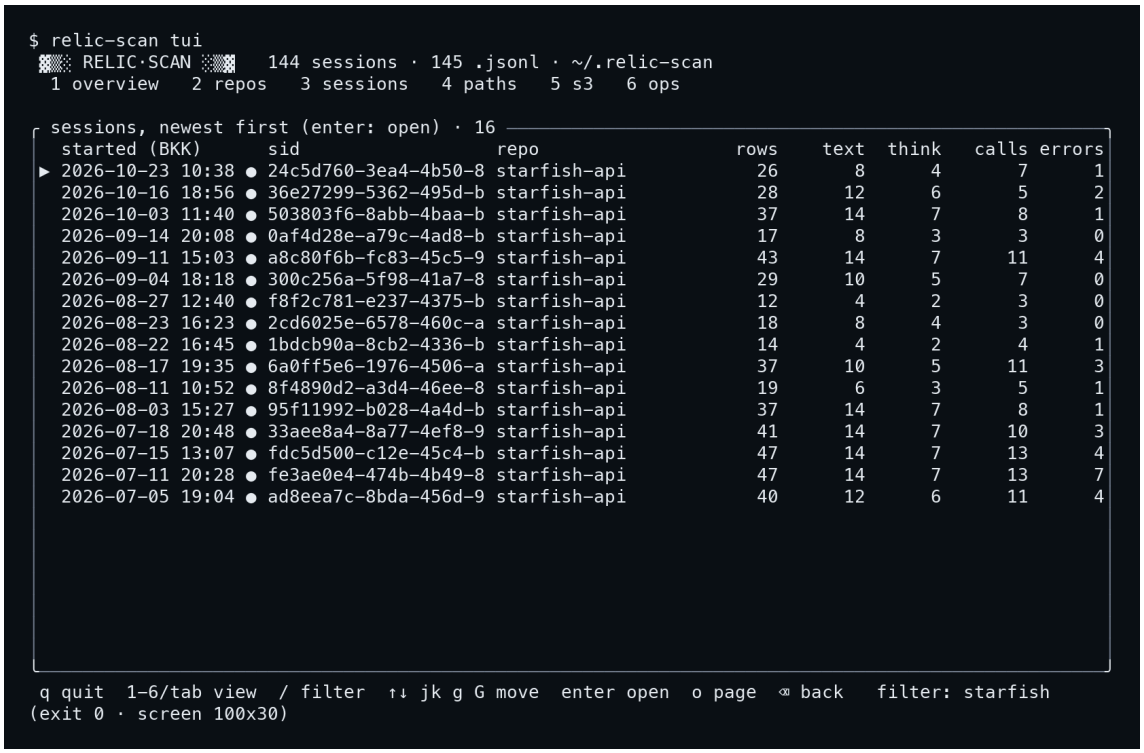
บรรทัดแรกคือ session ของ omp ตัวเลข `144` ท้ายชื่อกล่องคือจำนวนรายการที่เห็น **เลขท้ายชื่อกล่องเปลี่ยนตามตัวกรองเสมอ** จึงเป็นวิธีเช็กเร็วว่ากรองอยู่หรือเปล่า สังเกตด้วยว่า sid ของ omp ขึ้นต้นด้วยวันเวลา (`2026-10-25T09-03-00-`) ต่างจาก Claude ที่เป็น uuid เพราะ omp ตั้งชื่อไฟล์ด้วยเวลาเริ่ม

## 5.3 กรองรายการ เปิดรายละเอียด และข้ามไปหน้าเว็บ

### ขั้นที่ 1 — กด / แล้วพิมพ์คำที่ต้องการ

ตอนอยู่ในแท็บ sessions กด `/` แล้วพิมพ์ `starfish` จากนั้นกด Enter ระหว่างพิมพ์ รายการจะหดตามทุกตัวอักษร ส่วน Backspace ในโหมดพิมพ์ลบทีละตัวอักษร ไม่ใช่ถอยกลับ

```
┌ sessions, newest first (enter: open) · 16 ——
 …⌫ back   filter: starfish
```



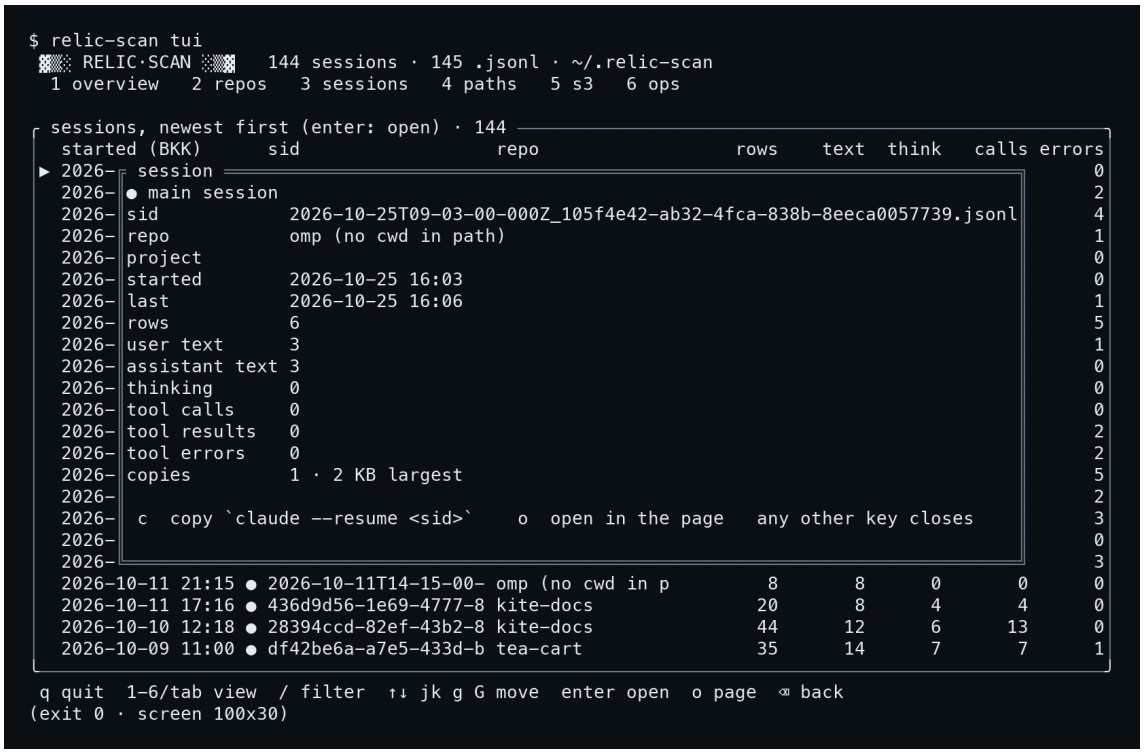

รูปที่ 36: แท็บ sessions หลังกรองคำว่า starfish เหลือ 16 รายการ ท้ายจอมี filter: starfish

เหลือ 16 session ตรงกับตัวเลข 16 ของ starfish-api ที่แท็บ repos และที่ DuckDB จะนับให้ในบทที่ 6 **ตัวกรองนับเป็นคำและใช้เงื่อนไข AND** ตาม README ถ้าพิมพ์สองคำแถวต้องมีทั้งสองคำ ที่ท้ายจอมีบรรทัด `filter: starfish` เตือนว่าตัวกรองยังทำงานอยู่แม้เราจะกด Enter ออกจากโหมดพิมพ์แล้ว

## ขั้นที่ 2 — Esc ล้างตัวกรอง แล้ว Enter เปิดรายละเอียด

กด Esc เพื่อล้างตัวกรอง รายการกลับมาเป็น 144 แล้วกด Enter ที่แถวบนสุด ซึ่งก็คือ session ของ omp เมื่อครู่ จะมีกล่องรายละเอียดซ้อนขึ้นมาเหนือรายการ

```
╟● main session
╟sid     2026-10-25T09-03-00-000Z_105f4e42-
ab32-4fca-838b-8eeca0057739.jsonl
╟repo    omp (no cwd in path)
╟rows    6
╟copies  1 · 2 KB largest
```




รูปที่ 37: กล่อง session ซ้อนบนรายการ แสดง sid, repo, เวลา, จำนวนแถวแยกชนิด และ copies

กล่องนี้ให้ **sid เต็ม** ซึ่งรายการตัดทิ้ง พร้อมเวลาเริ่มกับเวลาสุดท้าย (16:03 ถึง 16:06) จำนวนแถวแยกชนิด และบรรทัด `copies` ที่บอกว่าไฟล์นี้มีกี่สำเนา (1 สำเนา ใหญ่สุด 2 KB) ถ้ามี mirror หรือ backup สำเนาจะนับรวมตรงนี้ เรื่องการพับสำเนาเป็นแถวเดียวจะอธิบายในบทที่ 9

ท้ายกล่องบอกคีย์ที่ใช้ได้ในกล่องนี้ไว้ครบ

```
c  copy `claude --resume <sid>`    o  open in the page   any other key …
```

### ขั้นที่ 3 — c, o และ --mouse

กด `c` เพื่อคัดลอกคำสั่ง `claude --resume <sid>` ไว้ในคลิปบอร์ด พร้อมวางเพื่อกลับไปคุยต่อกับ session นั้นได้เลย แต่ **เฉพาะ session ของ Claude Code เท่านั้น** ในโค้ด keys.rs session ของ agent อื่นจะคัดลอกแค่ sid เปล่า ๆ เพราะคำสั่ง resume ของ Claude ใช้กับ Codex หรือ omp ไม่ได้ session ในภาพนี้เป็นของ omp จึงได้เฉพาะ sid

กด `o` เพื่อเปิดหน้าเว็บไปที่ session เดียวกัน คีย์ `o` ใช้ได้นอกกล่องด้วย ตอนเลือกแถว repo หรือโฟลเดอร์อยู่ ก็จะเปิดหน้าเว็บตรงจุดนั้น หน้าเว็บคือหน้าเดียวกับที่เปิดด้วย `relic-scan up` ในบทที่ 3 ส่วนคีย์อื่นนอกจาก `c`, `o` และ `q` จะปิดกล่องทั้งหมด

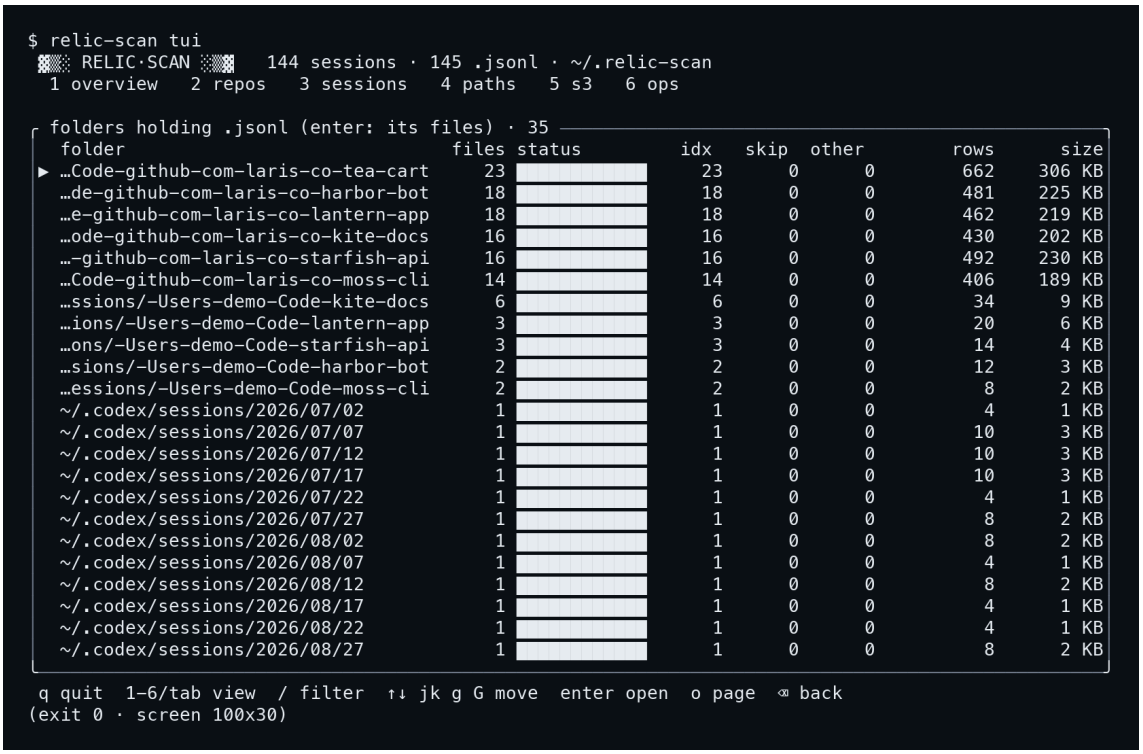
ถ้าชอบใช้เมาส์ ให้เปิดด้วย `--mouse` คลิกหนึ่งแถวเพื่อเลือก คลิกซ้ำที่แถวเดิมเพื่อเปิด ข้อมูลนี้มาจาก README เราไม่ได้ถ่ายภาพไว้ เพราะการคลิกจับเป็นภาพนิ่งไม่ได้

## 5.4 แท็บ paths และ ops

### ขั้นที่ 1 — ปิดกล่องด้วย Esc แล้วกด 4

กล่อง session ยังเปิดค้างอยู่ และ **ทุกคีย์ที่ไม่ใช่ `c`, `o`, `q` จะแค่ปิดกล่อง** ไม่ได้ส่งต่อไป ให้แท็บ ถ้ากด `4` ทันที กล่องจะปิดแล้วยังอยู่แท็บ sessions เดิม ภาพชุดนี้จึงกด Esc ปิดกล่องก่อน แล้วค่อยกด `4`

```
┌ folders holding .jsonl (enter: its files) · 35 ──
│▶ …laris-co-tea-cart     23 …     23      0      0   662    306 KB│
```



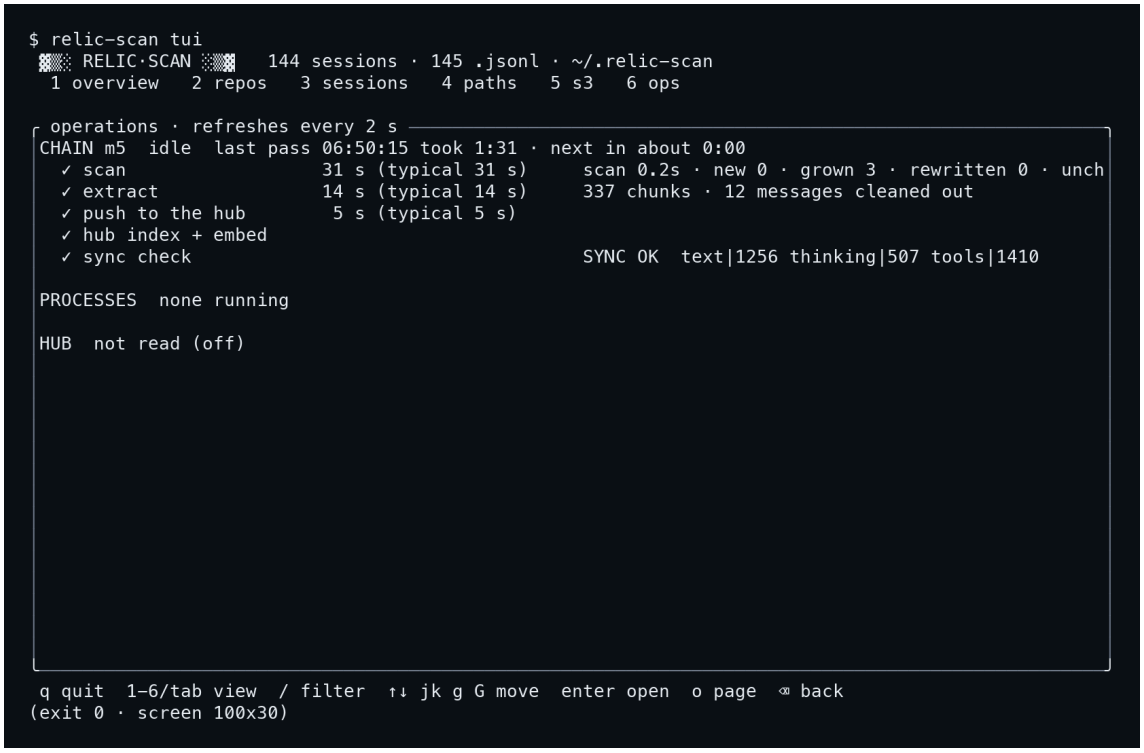

รูปที่ 38: แท็บ paths: 35 โฟลเดอร์ที่มี .jsonl แถวแรกคือ tea-cart

แท็บ paths แสดงโฟลเดอร์ที่มี `.jsonl` อยู่ 35 โฟลเดอร์ พร้อมจำนวนไฟล์ และตัวเลข indexed, skipped และ other โฟลเดอร์แรกคือ tea-cart 23 ไฟล์ 662 แถว 306 KB กด Enter ที่โฟลเดอร์เพื่อดูไฟล์ข้างใน และกด Enter ที่ไฟล์เพื่อคัดลอก path ของมัน Backspace ถอยจากรายการไฟล์กลับมารายการโฟลเดอร์ เนื้อหาเดียวกันดูเป็นภาพได้ที่หน้า Paths ของเว็บ (บทที่ 3)

### ขั้นที่ 2 — กด 6 ดู ops

คีย์ `6` เข้าแท็บ ops ซึ่งเป็นภาพเดียวกับคำสั่ง `relic-scan ops` แต่วาดใหม่ทุก 2 วินาที ชื่อกล่องก็บอกไว้ว่า `operations · refreshes every 2 s`

```
│CHAIN m5  idle  last pass 06:50:15 took 1:31 · next in about 0:00
│  ✓ scan                   31 s (typical 31 s)
│  ✓ extract                14 s (typical 14 s)
│  ✓ push to the hub         5 s (typical 5 s)
│  ✓ sync check             SYNC OK  text|1256 thinking|507 tools|1410
```




รูปที่ 39: แท็บ ops: chain ห้าขั้นพร้อมเวลา, PROCESSES none running และ HUB

อ่านจากบนลงล่าง **chain คือสายงานห้าขั้น** ได้แก่ `scan`, `extract`, `push to the hub`, `hub index + embed` และ `sync check` แต่ละขั้นมีเครื่องหมายถูก เวลาที่ใช้ และเวลาปกติ (typical) ให้เทียบ ถ้าขั้นไหนช้ากว่าปกติ ตรงนี้จะเห็นก่อนที่ใครจะบ่น บรรทัดสุดท้ายคือ `SYNC OK` พร้อมจำนวนแถวที่ฝั่งเราและฝั่ง hub นับตรงกัน

ตัวเลขสามตัวใน `SYNC OK` บวกกันได้ 1256 + 507 + 1410 = 3,173 เท่ากับจำนวนแถวบนหน้า Smart Scan ส่วน `PROCESSES` ขึ้นว่า `none running` แปลว่าไม่มีโปรเซสของ chain รันอยู่ตอนนั้น และ `HUB  not read (off)` แปลว่าไม่ได้ถาม hub เลย ซึ่งเป็นค่าเริ่มต้นในภาพนี้ เรื่อง chain ทั้งเส้นจะอธิบายเต็มในบทที่ 8

### ขั้นที่ 3 — แท็บ s3 ที่เหลือ

แท็บที่ห้า `s3` ไม่อยู่ในชุดภาพ เพราะบ้านเดโมไม่ได้ตั้งปลายทาง S3 ไว้ ตาม README ในแท็บนี้ `c` ตรวจเทียบกับ store โดยไม่ส่งอะไร และ `s` ส่งเฉพาะเดือนที่ยังขาด โดยไม่ทับเดือนที่ต่างกัน เป็นงานเดียวกับการ์ด S3 copy ในหน้า Export ซึ่งบทที่ 7 จะพาไปดู

ตอนนี้เรามีสามทางเข้าดัชนีแล้ว คือหน้าเว็บ เทอร์มินัล และ CLI ทางที่สี่ยืดหยุ่นที่สุดคือถามด้วย SQL ตรง ๆ บทถัดไปจะเปิด DuckDB ขึ้นมาถามไฟล์ Parquet เหล่านี้ทีละคำถาม

# บทที่ 6: ถามดัชนีด้วย DuckDB

หน้าเว็บกับ TUI ตอบคำถามที่คนทำ relic-scan คิดไว้ล่วงหน้า แต่คำถามของเราไม่ได้ตรงกับเขาเสมอไป เช่น โมเดลไหนตอบกี่ครั้งในเดือนไหน หรือ session ไหนกินโทเคนมากสุด พอดัชนีเป็นไฟล์ Parquet ธรรมดา เราก็เปิดด้วยเครื่องมืออะไรก็ได้ที่อ่าน Parquet เป็น บทนี้ใช้ DuckDB เพราะติดตั้งง่ายและรัน SQL บนไฟล์ได้เลย ไม่ต้องโหลดเข้าฐานข้อมูลก่อน

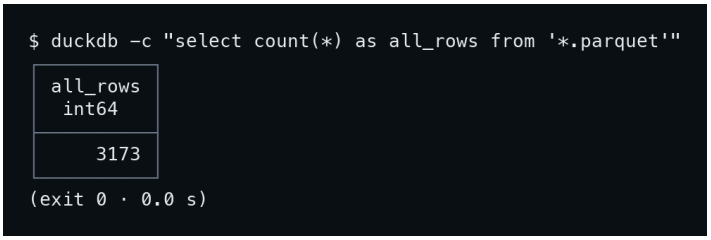
ทุกคำสั่งในบทนี้รันที่โฟลเดอร์ `~/.relic-scan` ของบ้านเดโม ( `cd ~/.relic-scan` ก่อน) เพราะ path ใน SQL เป็น `'*.parquet'` และ `'meta/sessions.parquet'` แบบสัมพัทธ์ และก่อนเริ่มมีกฎเรื่องความเป็นส่วนตัวหนึ่งข้อจาก AGENTS.md ทรานสคริปต์เก็บงานส่วนตัว อ่านเฉพาะที่ถามหา และอย่าคัดลอกแถวออกจากเครื่อง

## 6.1 นับทุกอย่างให้ได้เลขเดียวกับแอป

### ขั้นที่ 1 — นับแถวทั้งหมด

เริ่มจากคำถามที่ง่ายที่สุด ไฟล์ `YYYY-MM.parquet` หนึ่งไฟล์ต่อเดือน ใช้ glob `'*.parquet'` รวมทุกเดือนในโฟลเดอร์ แต่ไม่รวม `meta/` เพราะอยู่ในโฟลเดอร์ย่อย

```
duckdb -c \
  "select count(*) as all_rows from '*.parquet'"
```




รูปที่ 40: DuckDB นับแถวทั้งหมดได้ 3173

```
|     3173 |
```

ก่อนไปต่อ ลองดูว่าไฟล์เดือนหนึ่งมีคอลัมน์อะไรบ้างด้วย `describe`

```
duckdb -c "describe select * from '2026-10.parquet'"
```




รูปที่ 41: ผล describe ของไฟล์ 2026-10.parquet: 24 คอลัมน์ ชื่อและชนิด เช่น sid, ts_th, role, kind, name, text, body, err, model, out_tokens

ผลมี 24 คอลัมน์ ที่ใช้ในบทนี้คือ `role`, `kind`, `name`, `engine`, `sid`, `ts_th`, `text`, `body`, `err`, `model`, `msg_id` และคอลัมน์โทเคน `out_tokens`, `in_tokens`, `cache_read`, `cache_write` ส่วน `machines` กับ `sources` เป็นอาร์เรย์ (`VARCHAR[]`)

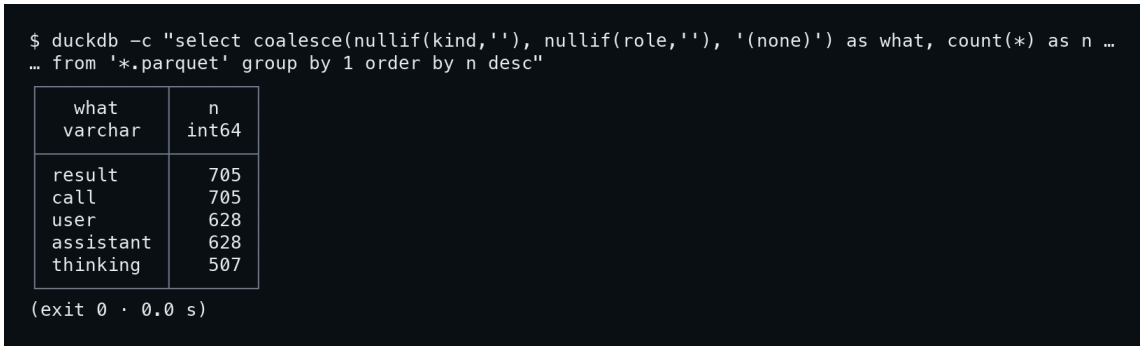
ผลนับแถวคือ 3173 แถว **เลขนี้ต้องตรงกับยอดบนหน้า Smart Scan** (3,173) ถ้าไม่ตรง แปลว่าเรามองคนละโฟลเดอร์ หรือดัชนีไม่ได้ถูกอัปเดตแล้ว AGENTS.md แนะนำให้นับทั้งหมดก่อนทุกครั้ง แล้วค่อยรายงานชิ้นส่วนเทียบกับยอดรวม จะได้ไม่หลงว่า “ข้อความผู้ใช้ 628” คือทั้งหมดของข้อมูล

### ขั้นที่ 2 — แยกตามว่าแต่ละแถวคืออะไร

หนึ่งแถวอาจเป็นข้อความ ความคิด คำเรียกเครื่องมือ หรือผลลัพธ์ของเครื่องมือ ดัชนีบอกด้วย **สองคอลัมน์ คือ `role` และ `kind`** ไม่ใช่คอลัมน์เดียว ข้อความกับ thinking มี `role` (user, assistant, thinking) แต่ `kind` ว่าง ส่วนการเรียกเครื่องมือมี `kind = 'call'` (ชื่อเครื่องมืออยู่ใน `name`) และผลลัพธ์มี `kind = 'result'` โดยไม่มี `role` คำสั่งข้างล่างจึงใช้

`coalesce(nullif(...))` เลือกค่าที่ไม่ว่างของสองคอลัมน์มารวมเป็นคอลัมน์เดียวชื่อ `what`

```
duckdb -c \
  "select coalesce(nullif(kind,''), nullif(role,''), '(none)') as what,
count(*) as n from '*.parquet' group by 1 order by n desc"
```




รูปที่ 42: ตารางจำนวนแถวแยกตามชนิด: result, call, assistant, user, thinking

```
| result    |   705 |
| call      |   705 |
| assistant |   628 |
| user      |   628 |
| thinking  |   507 |
```

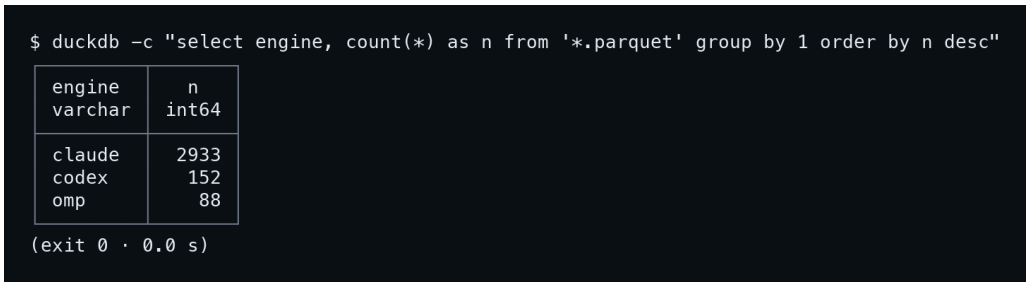
บวกห้าแถวได้ 705 + 705 + 628 + 628 + 507 = 3173 พอดี จึงไม่มีแถวตกหล่น และไม่มีแถว `(none)` ในบ้านเดโม ซึ่งข้อมูลจริงอาจมีประปรายได้ (AGENTS.md บอกว่าบางแถวไม่มีทั้ง role และ kind) ตัวเลขชุดนี้ตรงกับ `rows by type` ที่เห็นใน TUI บทที่ 5 ทุกตัว การจับคู่ call กับ result ที่เท่ากัน (705 ต่อ 705) ก็เป็นสัญญาณที่ดี คือทุกการเรียกเครื่องมือมีผลลัพธ์กลับมา

หนึ่งข้อควรรู้เรื่อง thinking ตั้งแต่ราวเดือนมีนาคม 2026 Claude Code เขียนบล็อก thinking ส่วนใหญ่เป็นลายเซ็นอย่างเดียว ไม่มีข้อความ แถวพวกนี้ยังนับเป็น thinking แต่ `text` ว่าง จึงไม่มีอะไรให้อ่านหรือทำ embedding

### ขั้นที่ 3 — แยกตาม engine

คอลัมน์ `engine` บอกว่าแถวมาจาก agent ตัวไหน นับด้วย `group by` ธรรมดา

```
duckdb -c \
  "select engine, count(*) as n from '*.parquet' group by 1 order by n
desc"
```




รูปที่ 43: จำนวนแถวต่อ engine: claude, codex, omp

```
| claude  |  2933 |
| codex   |   152 |
| omp     |    88 |
```

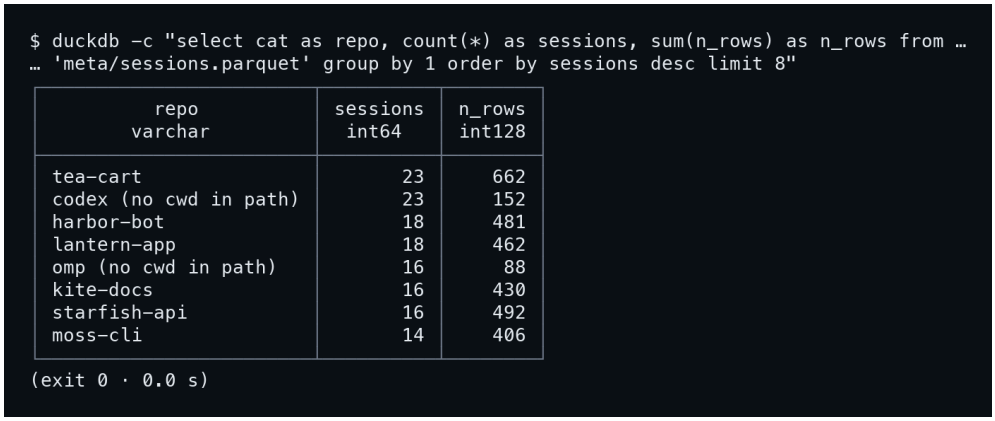
**2933 + 152 + 88 ได้ 3173 อีกครั้ง** Claude Code ครองข้อมูลแทบทั้งหมด ส่วน Codex กับ omp มีนิดเดียว ตัวเลข 152 และ 88 ตรงกับคอลัมน์ rows ของ `codex (no cwd in path)` และ `omp (no cwd in path)` ที่เห็นในแท็บ repos ของ TUI

## 6.2 session ต่อ repo

### ขั้นที่ 1 — ถามจากตาราง meta

ตาราง `meta/sessions.parquet` มีหนึ่งแถวต่อหนึ่ง session จึงใช้ตอบคำถามระดับ session ได้เร็ว ไม่ต้องกวาดทุกแถวของเดือน คอลัมน์ `cat` คือชื่อ repo และ `n_rows` คือจำนวนแถวของ session นั้น

```
duckdb -c \
  "select cat as repo, count(*) as sessions, sum(n_rows) as n_rows from
'meta/sessions.parquet' group by 1 order by sessions desc limit 8"
```




รูปที่ 44: ตาราง session และแถวต่อ repo ทั้งแปด

```
| tea-cart                |       23 |    662 |
| codex (no cwd in path)  |       23 |    152 |
| lantern-app             |       18 |    462 |
| omp (no cwd in path)    |       16 |     88 |
```

เลข 662 ของ tea-cart คือตัวเลขเต็มที่ TUI ตัดทิ้งเหลือ `66` ในบทที่ 5 ส่วน session รวมกัน 23 + 23 + 18 + 18 + 16 + 16 + 16 + 14 ได้ 144 ตรงกับจำนวน session ทั้งหมด **ผลรวมเป็นเครื่องมือตรวจที่ถูกที่สุด** ถ้ารวมแล้วไม่เท่ายอดที่รู้ ก็แปลว่าเราตัดแถวทิ้งไปที่ `limit` หรือมี repo เกินแปด

## 6.3 โมเดลและโทเคนต่อเดือน

### ขั้นที่ 1 — นับ API call และโทเคนที่ตอบออกมา

คำถามนี้ซับซ้อนกว่าข้อก่อนหน้า เพราะหนึ่งการตอบจาก API ไม่ใช่หนึ่งแถว Claude หนึ่งคำตอบ (หนึ่ง `msg_id`) แตกเป็นหลายแถว หนึ่งแถวต่อบล็อกเนื้อหา เช่น ข้อความหนึ่งแถว thinking หนึ่งแถว และ tool call อีกหนึ่งแถว ถ้าเรา `sum(out_tokens)` ตรง ๆ แล้วทุกแถวถือโทเคนซ้ำกัน ก็จะนับซ้ำ

ดัชนีแก้ปัญหานี้ไว้แล้ว **`out_tokens` ใส่ไว้ที่แถวเดียวต่อ `msg_id`** ดังนั้น `sum(out_tokens)` นับแต่ละคำตอบหนึ่งครั้งพอดี และ `count(distinct msg_id)` ก็นับจำนวน API call ได้ตรง ๆ ส่วน `model` ติดอยู่กับแถวข้อความ thinking และ tool call ของ assistant

```
duckdb -c \
  "select left(ts_th,7) as month, model, count(distinct msg_id) as calls,
sum(out_tokens) as out_tokens from '*.parquet' where model is not null
group by all order by month, calls desc limit 12"
```

```
$ duckdb -c "select left(ts_th,7) as month, model, count(distinct msg_id) as calls, …
… sum(out_tokens) as out_tokens from '*.parquet' where model is not null group by all order by …
… month, calls desc limit 12"
```

| month varchar | model varchar | calls int64 | out_tokens int128 |
|---|---|---|---|
| 2026-07 | claude-sonnet-5-5 | 52 | 70967 |
| 2026-07 | claude-haiku-5-5 | 45 | 50486 |
| 2026-07 | claude-opus-5-5 | 32 | 43302 |
| 2026-07 | glm-5.3 | 10 | 4925 |
| 2026-07 | gpt-6-astra | 0 | NULL |
| 2026-08 | claude-sonnet-5-5 | 52 | 65519 |
| 2026-08 | claude-haiku-5-5 | 35 | 48510 |
| 2026-08 | claude-opus-5-5 | 31 | 30756 |
| 2026-08 | glm-5.3 | 9 | 4371 |
| 2026-08 | gpt-6-astra | 0 | NULL |
| 2026-09 | claude-opus-5-5 | 61 | 75068 |
| 2026-09 | claude-haiku-5-5 | 52 | 63565 |

```
  12 rows                                  4 columns
(exit 0 · 0.0 s)
```

รูปที่ 45: ตารางโมเดล, จำนวน call และ out_tokens ต่อเดือน 12 แถวแรก

```
| 2026-07 | claude-sonnet-5-5 |    52 |      70967 |
| 2026-07 | glm-5.3           |    10 |       4925 |
| 2026-07 | gpt-6-astra       |     0 |       NULL |
```

คำสั่งนี้ใช้ `left(ts_th, 7)` ตัดเวลาไทยเหลือ `ปี-เดือน` และ `group by all` จัดกลุ่มตามคอลัมน์ที่ไม่ได้เป็น aggregate ให้เอง ท้ายสุดมี `limit 12` ภาพจึงเห็นแค่สองเดือนกับครึ่ง ถ้าอยากเห็นทั้งสี่เดือนให้เอา `limit` ออก

### ขั้นที่ 2 — อ่านแถวที่เป็น 0 และ NULL

ตรงนี้คือจุดที่ควรหยุดอ่านช้า ๆ แถว `gpt-6-astra` ของ Codex แสดง **`calls` เป็น 0 และ `out_tokens` เป็น NULL** ทั้งสองเดือน ไม่ได้แปลว่า Codex ไม่เคยตอบ ซึ่งเรารู้ว่าตอบ เพราะบ้านเดโมมี 152 แถวของ Codex ในภาพ 14 แล้ว

สาเหตุอยู่ที่ AGENTS.md ข้อมูลของ Codex มี `model` (ดึงจาก `turn_context` ล่าสุด) แต่ **ไม่มี `msg_id` และยังไม่มี `out_tokens`** ดังนั้น `count(distinct msg_id)` ของแถวพวกนี้นับค่า NULL ได้ศูนย์ และ `sum` ของค่า NULL ล้วนได้ NULL ไม่ใช่ศูนย์ ส่วนเงื่อนไข `where model is not null` ผ่านเพราะแถวมีโมเดลจริง กลุ่ม `gpt-6-astra` เลยโผล่ขึ้นมาในตารางแบบมีชื่อแต่ไม่มีตัวเลข

บทเรียนคือ อย่าอ่านศูนย์ว่า “ไม่ได้ใช้” ถ้าต้องการเปรียบเทียบโมเดลข้าม agent ให้ใช้จำนวนแถว ( `count(*)` ) แทนสำหรับ Codex และจำไว้ว่าโทเคนของมันยังไม่มีในดัชนีรุ่นนี้ ฝั่ง omp ไม่เป็นแบบนี้ `glm-5.3` มีทั้ง call (10) และโทเคน (4925) เพราะ omp ส่ง `responseId` มาเป็น `msg_id` ให้

## 6.4 โทเคนต่อ session

### ขั้นที่ 1 — ดูคอลัมน์โทเคนใน meta/sessions

คำถามเดียวกันแต่ระดับ session ไม่ต้องคำนวณเอง เพราะ `meta/sessions.parquet` สรุปไว้ให้ในหลายคอลัมน์ **ที่ขึ้นต้นด้วยโมเดลและโทเคน** คือ `models` (รายชื่อโมเดลที่ตอบเรียงตัวที่ตอบมากสุดก่อน), `model_msgs` (จำนวนคำตอบของแต่ละโมเดล ตามลำดับเดียวกัน), `in_tokens`, `out_tokens`, `cache_read_tokens` และ `cache_write_tokens` ทั้งหมดรวมจากการตอบของ API โดยนับแต่ละครั้งหนึ่งหน ถ้าทรานสคริปต์ไม่บันทึกไว้ (อย่างกรณี Codex) ค่าเป็น 0

```
duckdb -c \
  "select sid, models, model_msgs, in_tokens, out_tokens,
cache_read_tokens from 'meta/sessions.parquet' where out_tokens > 0 order
by out_tokens desc limit 4"
```

```
$ duckdb -c "select sid, models, model_msgs, in_tokens, out_tokens, cache_read_tokens from …
… 'meta/sessions.parquet' where out_tokens > 0 order by out_tokens desc limit 4"
```

| sid<br>varchar | models<br>varchar[] | model_msgs<br>int64[] | in_tokens<br>int64 | out_tokens<br>int64 | cache_read_tokens<br>int64 |
|---|---|---|---|---|---|
| 95f11992 | [claude-haiku-5-5] | [7] | 121 | 11285 | 335324 |
| 28394ccd | [claude-sonnet-5-5] | [6] | 135 | 11250 | 246245 |
| 99aab31b | [claude-opus-5-5] | [6] | 129 | 10476 | 298801 |
| a9593467 | [claude-sonnet-5-5] | [7] | 171 | 10399 | 540510 |

```
(exit 0 · 0.0 s)
```

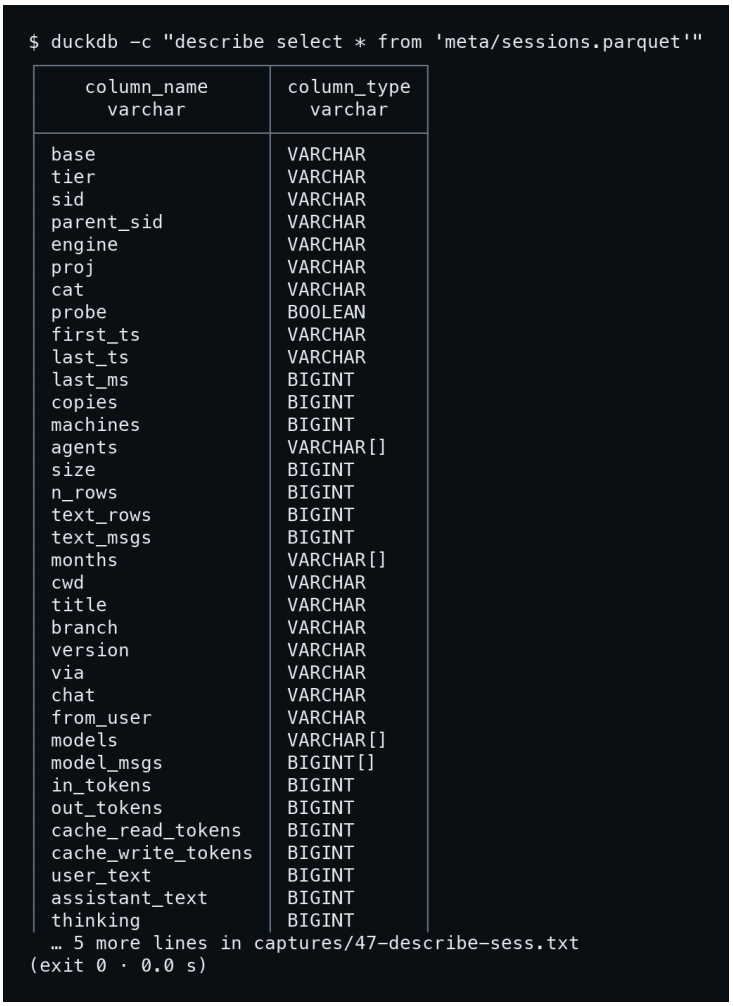
รูปที่ 46: สี่ session ที่ตอบยาวสุด: models, model_msgs, in_tokens, out_tokens, cache_read_tokens

```
| 95f11992 | [claude-haiku-5-5]  | [7]  |  121 |  11285 |
| 28394ccd | [claude-sonnet-5-5] | [6]  |  135 |  11250 |
```

`models` กับ `model_msgs` เป็นอาร์เรย์ จึงเห็นวงเล็บเหลี่ยม session บนสุดตอบด้วยโมเดลเดียวคือ haiku จำนวน 7 คำตอบ ถ้า session ใช้สองโมเดลสลับกัน อาร์เรย์ทั้งสองจะมีสองช่อง เรียงคู่กัน ใช้ `unnest` ในภายหลังได้ถ้าต้องการแตกเป็นแถว

ถ้าอยากเห็นคอลัมน์ทั้งหมดของตารางนี้ ให้ `describe` เหมือนที่ทำกับไฟล์เดือน

```
duckdb -c "describe select * from 'meta/sessions.parquet'"
```




รูปที่ 47: ผล describe ของ meta/sessions.parquet: 38 คอลัมน์ รวม models, model_msgs, in_tokens, out_tokens, cache_read_tokens, cache_write_tokens

ภาพมี 38 คอลัมน์ ชนิดของ `models` เป็น `VARCHAR[]` และ `model_msgs` เป็น `BIGINT[]` ตรงกับที่เห็นเป็นวงเล็บเหลี่ยม ส่วนคอลัมน์โทเคนทั้งสี่เป็น `BIGINT`

## ขั้นที่ 2 — เทียบ in, out และ cache

ตัวเลขในสามคอลัมน์นี้ต่างกันมาก **cache_read_tokens คือคอลัมน์ที่ใหญ่สุดเสมอ** แถวบนสุดมี in_tokens แค่ 121 และ out_tokens 11285 แต่ cache_read_tokens ถึง 335324 เหตุผลคือ Claude Code ส่งบริบทของบทสนทนาทั้งหมดซ้ำในทุกคำขอ แล้วส่วนที่เหมือนเดิมถูกอ่านจาก cache จึงไม่นับเป็น input ใหม่ การรวมสามคอลัมน์เข้าด้วยกันเป็น “โทเคนที่ใช้” จึงทำให้เลขบวมโดยไม่ได้บอกอะไร ควรรายงานแยกสามช่อง

ข้อควรระวังจาก AGENTS.md อีกข้อ เดือนที่ถูกเขียนก่อนจะมีคอลัมน์ input และ cache จะอ่านค่าเหล่านั้นเป็น NULL และคอลัมน์โทเคนใน `meta/sessions.parquet` นับเฉพาะเดือนที่มีคอลัมน์เหล่านั้น ถ้าเจอศูนย์หรือ NULL เต็มไปหมดในดัชนีเก่า ให้ย้ายโฟลเดอร์

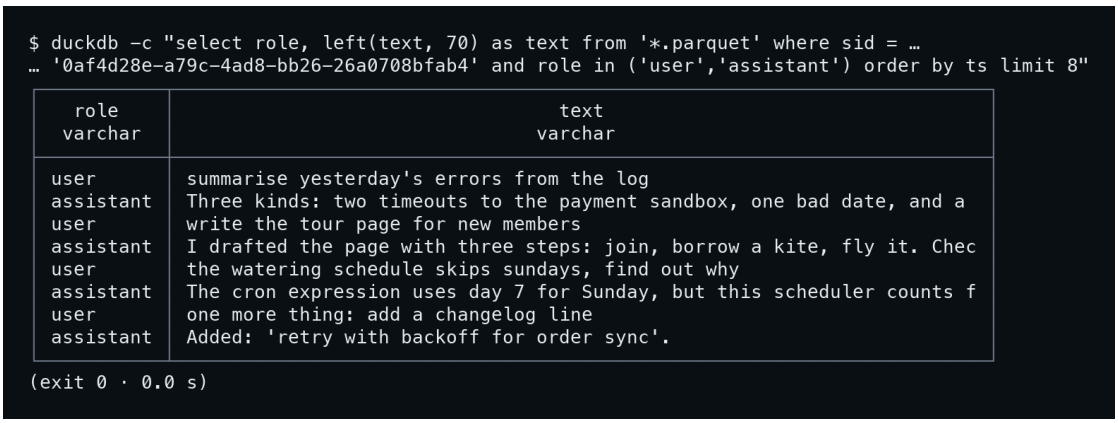
`~/.relic-scan` ไปเก็บ แล้วรัน `relic-scan run` สร้างใหม่ ข้อมูลในบ้านเดโมนี้สร้างด้วยรุ่นปัจจุบัน จึงไม่เจอปัญหานี้

## 6.5 อ่านบทสนทนาตามลำดับ

### ขั้นที่ 1 — เลือก session แล้วเรียงตามเวลา

ถ้าเจอ session ที่น่าสนใจ จะอ่านเนื้อหาก็กรองด้วย `sid` เรียงด้วย `ts` และเลือกเฉพาะ role ที่เป็นคนคุยกัน (`user`, `assistant`) ตัด thinking และ tool ออก ในตัวอย่างใช้ sid เต็มของ session หนึ่งใน starfish-api (ที่ TUI ในบทก่อนกรองเจอ) และตัดข้อความเหลือ 70 ตัวอักษรด้วย `left(text, 70)` เพื่อให้พอดีจอ

```
duckdb -c \
  "select role, left(text, 70) as text from '*.parquet' where sid =
'0af4d28e-a79c-4ad8-bb26-26a0708bfab4' and role in ('user','assistant')
order by ts limit 8"
```




รูปที่ 48: บทสนทนาแปดข้อความของ session หนึ่ง สลับ user กับ assistant

```
| user      | summarise yesterday's errors from the log
|
| assistant | Three kinds: two timeouts to the payment sandbox, one bad …
|
| user      | write the tour page for new members
|
```

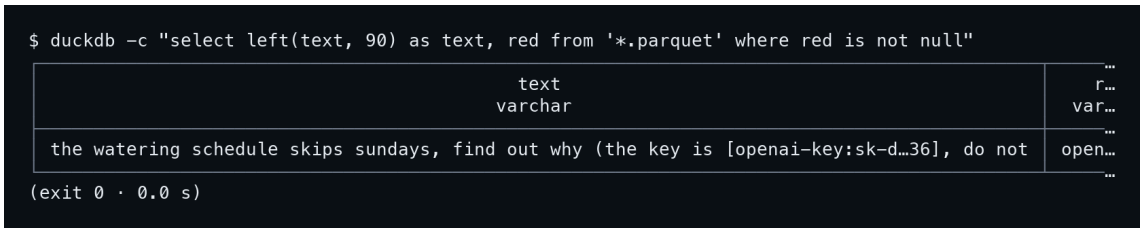
**ข้อความของผู้ใช้กับ assistant สลับกันครบสี่คู่** เดโมของเรามีสี่คู่ต่อ session แปดข้อความจึงครอบคลุมทั้งหมดพอดี ใน session จริงที่ยาวกว่านี้ `limit` จะตัดกลางคัน ให้เปลี่ยนเป็น `offset` เดินต่อ หรือกรองด้วยคำสำคัญ เช่น `text ilike '%parquet%'` แบบที่ AGENTS.md ยกตัวอย่างไว้

## 6.6 บรรทัดที่ถูก redact

### ขั้นที่ 1 — หาแถวที่มีคอลัมน์ red

ถ้าทรานสคริปต์มีคีย์ลับหลุดเข้ามาในข้อความ relic-scan จะไม่เก็บค่าจริงไว้ในดัชนี แต่แทนด้วยเครื่องหมายบอกชนิด และจดชนิดนั้นไว้ในคอลัมน์ `red` แถวทั่วไปคอลัมน์นี้เป็น NULL จึงกรองหาแถวที่ถูก redact ได้ด้วย `red is not null`

```
duckdb -c \
  "select left(text, 90) as text, red from '*.parquet' where red is not
null"
```




รูปที่ 49: หนึ่งแถวที่ข้อความถูก redact และคอลัมน์ red เป็น openai-key

```
…(the key is [openai-key:sk-d…36], do not | openai-key |
```

บ้านเดโมมีคีย์ปลอมที่ฝังไว้หนึ่งตัว (ขึ้นต้น `sk-demo-`) ในหนึ่ง prompt และผลคือมีแถวเดียวที่ `red` ไม่ว่าง ในข้อความ คีย์ถูกย่อเป็น `[openai-key:sk-d…36]` เหลือแค่ต้นกับท้ายสองสามตัวอักษร พอให้รู้ว่าเคยมีอะไรตรงนั้น แต่ไม่พอให้เอาไปใช้ต่อ **ชนิดคีย์อยู่ในวงเล็บและในคอลัมน์ red** ซึ่งช่วยให้นับได้ว่ามี secret ชนิดไหนหลุดเข้ามากี่ครั้ง เรื่องนี้บทที่ 9 จะกลับมาอธิบายเต็ม

## 6.7 months.json และ _scans.jsonl

### ขั้นที่ 1 — ดูตราผนึกของเดือนแรก

ไม่ใช่ทุกไฟล์ในดัชนีที่เป็น Parquet ไฟล์ `meta/months.json` เป็น JSON ธรรมดา เก็บ **หนึ่งรายการต่อเดือน** ใช้ `jq` ดึงรายการแรกมาดู (ต้องมี `jq` ติดเครื่อง)

```
jq -c '.months[0]' ~/.relic-scan/meta/months.json
```

```
$ jq -c '.months[0]' ~/.relic-scan/meta/months.json
{"month":"2026-07","rows":819,"first_ts":"2026-07-02T09:08:55.000Z","max_ts":"2026-07-28T09:31:07. …
… 000Z","keys":"deb9bcc29d4996bd545ea8297889d90d","bytes":34153,"mtime_ms":1792974179224,"late_row …
… s":0,"late_total":0,"sealed":true,"sealed_at":"2026-10-26T00:22:59Z","sealed_rows":819,"sealed_k …
… eys":"deb9bcc29d4996bd545ea8297889d90d","resealed":0,"updated_at":"2026-10-26T00:22:59Z"}
(exit 0 · 0.0 s)
```

รูปที่ 50: JSON ของเดือน 2026-07: rows 819, sealed true, late_rows 0

```
{"month":"2026-07","rows":819,"first_ts":"2026-07-02T09:08:55.000Z",…
"late_rows":0,"late_total":0,"sealed":true,…
"sealed_rows":819,… "resealed":0,…}
```

สามฟิลด์ที่ควรอ่านก่อน `rows` คือ 819 แถวของเดือน 2026-07 `sealed: true` บอกว่าเดือนนี้ปิดแล้ว (อยู่ก่อนเดือนปัจจุบันตามเวลากรุงเทพ) และ `late_rows: 0` บอกว่าไม่มีแถวใหม่แทรกย้อนหลังเข้ามาในเดือนที่ปิด ส่วน `keys` กับ `sealed_keys` เป็นแฮชของกุญแจแถวทั้งหมด ในภาพสองค่าเท่ากัน (`deb9bcc2…`) แปลว่าเนื้อเดือนนี้ไม่เปลี่ยนตั้งแต่ปิด ส่วน `resealed: 0` นับจำนวนครั้งที่แถวมาสายแล้วทำให้ปิดผนึกซ้ำ การตรวจจริงทำด้วย `relic-scan seals --verify` ในบทที่ 7

### ขั้นที่ 2 — ดูบันทึกการสแกนครั้งล่าสุด

ไฟล์ `_scans.jsonl` เป็นบันทึกต่อท้ายทุกครั้งที่ index หนึ่งรอบหนึ่งบรรทัด เหมือนที่หน้า Scan log และคำสั่ง `relic-scan log` อ่าน เราเอาบรรทัดสุดท้ายมาดู และตัดฟิลด์ `diff` ซึ่งยาวทิ้งด้วย `del(.diff)`

```
tail -1 ~/.relic-scan/_scans.jsonl | jq 'del(.diff)'
```

```
$ tail -1 ~/.relic-scan/_scans.jsonl | jq 'del(.diff)'
{"past_changed":0,"new":0,"grown":1,"rewritten":0,"unchanged":143,"gone":0,"before_range":0,"files …
… ":1,"bytes":783,"scan_secs":0.206738084,"secs":0.266117584,"at":1792974179,"unreadable":0,"stopp …
… ed":false,"resumed":0,"copies":0,"job":"index","range":null,"kind":[],"only":[],"rows":3173,"ses …
… sions":144,"diff_total":1}
(exit 0 · 0.0 s)
```

รูปที่ 51: JSON ของการสแกนล่าสุด: grown 1, unchanged 143, rows 3173, sessions 144

```
{"past_changed":0,"new":0,"grown":1,"rewritten":0,"unchanged":143,…
"files":1,"bytes":783,"scan_secs":0.206738084,…"job":"index",…
"rows":3173,"sessions":144,"diff_total":1}
```

บรรทัดนี้เล่าเรื่องรอบที่ transcript หนึ่งไฟล์โตขึ้นในบทที่ 2 `grown: 1` กับ `unchanged: 143` รวมกันได้ 144 ไฟล์ที่ index แล้ว อ่านจริงเพียงหนึ่งไฟล์ (`files: 1`)

ขนาด 783 ไบต์ ใช้เวลาสแกนราว 0.2 วินาที จบรอบด้วย `rows: 3173` และ `sessions: 144` ซึ่งเป็นยอดเดียวกับที่ DuckDB นับในขั้นแรกของบทนี้ **ตัวเลขทั้งสามทางตรงกัน ไม่ว่าจะถามผ่านเว็บ TUI หรือ SQL** นั่นแหละคือเหตุผลที่ดัชนีเดียวรองรับได้ทุกทาง

ตอนนี้เราอ่านดัชนีได้เองแล้ว แต่เดือนที่ปิดผนึกแล้วนั้นตรวจสอบกันอย่างไร และส่งสำเนาไปเก็บไว้ที่อื่นยังไง บทถัดไปว่าด้วยสองเรื่องนี้

# บทที่ 7: เดือนที่ปิดผนึก และสำเนาไป S3

index ของ relic-scan แบ่งเป็นไฟล์ Parquet เดือนละหนึ่งไฟล์ ซึ่งมีจังหวะชีวิตของมันเอง เดือนปัจจุบันยังเปิดอยู่ ข้อมูลใหม่ไหลเข้ามาได้ตลอด พอเดือนนั้นผ่านไป relic-scan ก็ปิดผนึกมัน แล้วไฟล์จะกลายเป็นของที่แก้ไม่ได้ บทนี้ดูว่าผนึกคืออะไร ตรวจอย่างไรว่ายังเหมือนเดิม และส่งเดือนที่ปิดแล้วไปเก็บสำรองที่ S3 อย่างไร

ทุกอย่างในสองหัวข้อแรกรันจริงบนบ้านเดโม ส่วนหัวข้อที่สามเรื่อง S3 บนบ้านเดโมไม่มี store ให้ส่ง คำสั่งในหัวข้อนั้นจึงมาจาก AGENTS.md และไม่มีภาพผลลัพธ์

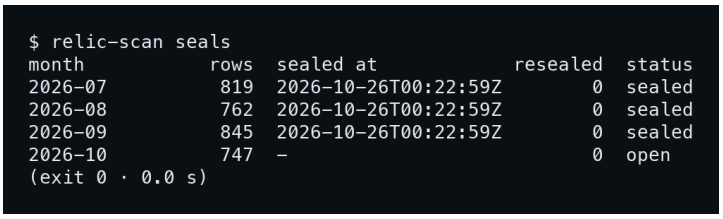
## 7.1 เดือนที่ปิดแล้วเป็นไฟล์อ่านอย่างเดียว

เดือนที่ “ปิด” คือเดือนที่อยู่ก่อนเดือนปัจจุบันตามเวลากรุงเทพ วันที่รันบ้านเดโมคือ 2026-10-26 เดือน 2026-07 ถึง 2026-09 จึงปิดแล้ว ส่วน 2026-10 ยังเปิด พอ relic-scan เห็นว่าเดือนไหนปิด มันจะจดผนึกไว้ แล้วตั้งไฟล์เดือนนั้นเป็นอ่านอย่างเดียว

### ขั้นที่ 1 — ดูสถานะผนึกของทุกเดือน

พิมพ์ `relic-scan seals` คำสั่งนี้อ่านอย่างเดียว ไม่เขียนอะไรเลย แต่ละบรรทัดคือหนึ่งเดือน บอกจำนวนแถว เวลาที่เห็นว่าปิด จำนวนครั้งที่ปิดซ้ำ และสถานะ

```
relic-scan seals
```




รูปที่ 52: ตาราง seals สี่เดือน สามเดือนแรก sealed และ 2026-10 เป็น open

อ่านตารางแล้วได้ว่า **สามเดือนแรกเป็น sealed และเดือน 2026-10 ยังเป็น open** แถวของสามเดือนที่ปิดคือ 819, 762 และ 845 แถว ส่วนเดือนที่เปิดมี 747 แถว รวมกันได้ 3,173 แถวพอดี ตรงกับตัวเลขบนหน้า Smart Scan คอลัมน์ `sealed at` ของเดือนที่เปิดเป็นขีด `–` เพราะยังไม่มีเวลาปิดให้จด

```
2026-07          819  2026-10-26T00:22:59Z        0  sealed
2026-10          747  -                           0  open
```

คอลัมน์ `resealed` เป็น 0 ทุกเดือน ค่านี้นับว่าเดือนที่ปิดไปแล้วต้องปิดใหม่กี่ครั้ง เพราะมีแถวข้ามาถึงทีหลัง เช่น เครื่องอื่นเอา transcript เก่ามาสำรองไว้ทีหลัง ในเดโมไม่มีเหตุแบบนั้น ค่าจึงเป็นศูนย์

ถ้าอยากรู้ว่าคำสั่งนี้รับตัวเลือกอะไร ถาม `--help`

```
relic-scan seals --help
```

```
$ relic-scan seals --help
Closed months and their seals (read-only files with the time, rows and key hash they were sealed …
… at); --verify checks each again

Usage: relic-scan seals [OPTIONS]

Options:
      --out <OUT>    [default: ~/.relic-scan]
      --kind <KIND>  the --kind the run used: picks ~/.relic-scan/<kind> when --out is not given
      --verify       read every sealed month again and compare it with its seal
  -h, --help         Print help
(exit 0 · 0.0 s)
```

รูปที่ 53: ผลของ relic-scan seals --help: คำอธิบาย Usage และตัวเลือก --out, \--kind, --verify

มีตัวเลือกให้ใช้ `--out`, `--kind` และ `--verify` ที่อ่านทุกเดือนที่ปิดซ้ำแล้วเทียบกับผนึก ซึ่งเป็นเรื่องของขั้นต่อไป

## ขั้นที่ 2 — ตรวจว่าเดือนที่ปิดยังเหมือนตอนปิด

ผนึกมีไว้ให้ตรวจ ใส่ `--verify` แล้ว relic-scan จะอ่านไฟล์ของแต่ละเดือนที่ปิดซ้ำอีกรอบ แล้วเทียบกับจำนวนแถวและ hash ของคีย์แถวที่จดไว้ตอนปิด

```
relic-scan seals --verify
```

```
$ relic-scan seals --verify
month           rows  sealed at            resealed  status
2026-07          819  2026-10-26T00:22:59Z        0  sealed, matches
2026-08          762  2026-10-26T00:22:59Z        0  sealed, matches
2026-09          845  2026-10-26T00:22:59Z        0  sealed, matches
2026-10          747  -                           0  open
(exit 0 · 0.0 s)
```

รูปที่ 54: ตาราง seals --verify สามเดือนที่ปิดขึ้น sealed, matches

สถานะของสามเดือนที่ปิดเปลี่ยนเป็น `sealed, matches` แปลว่า **ไฟล์วันนี้ตรงกับไฟล์ตอนปิดทุกแถว** เดือน 2026-10 ยังขึ้นแค่ `open` เพราะไม่มีผนึกให้เทียบ ถ้าไฟล์เดือนไหนถูกแก้หรือเสียหาย สถานะตรงนี้จะบอก ซึ่งเป็นเหตุผลที่ควรรัน `--verify` ก่อนส่งสำเนาไปไหน

### ขั้นที่ 3 — ดูผนึกดิบใน months.json

ตารางข้างบนอ่านมาจากไฟล์ `meta/months.json` ใช้ `jq` ดึงเดือนแรกมาดูทั้งก้อนได้ จะเห็นว่าผนึกคือข้อมูลธรรมดาไม่กี่ field

```
jq -c '.months[0]' ~/.relic-scan/meta/months.json
```

ผลลัพธ์ของคำสั่งนี้เป็นบรรทัด JSON เดียว ภาพเต็มอยู่ในบทที่ 6 หัวข้อ 6.7 ที่นี่อ่านเฉพาะ field ของผนึก ซึ่งเดือน 2026-07 ได้ `sealed_at` เป็น `2026-10-26T00:22:59Z` เท่ากับในตาราง seals ข้างบน

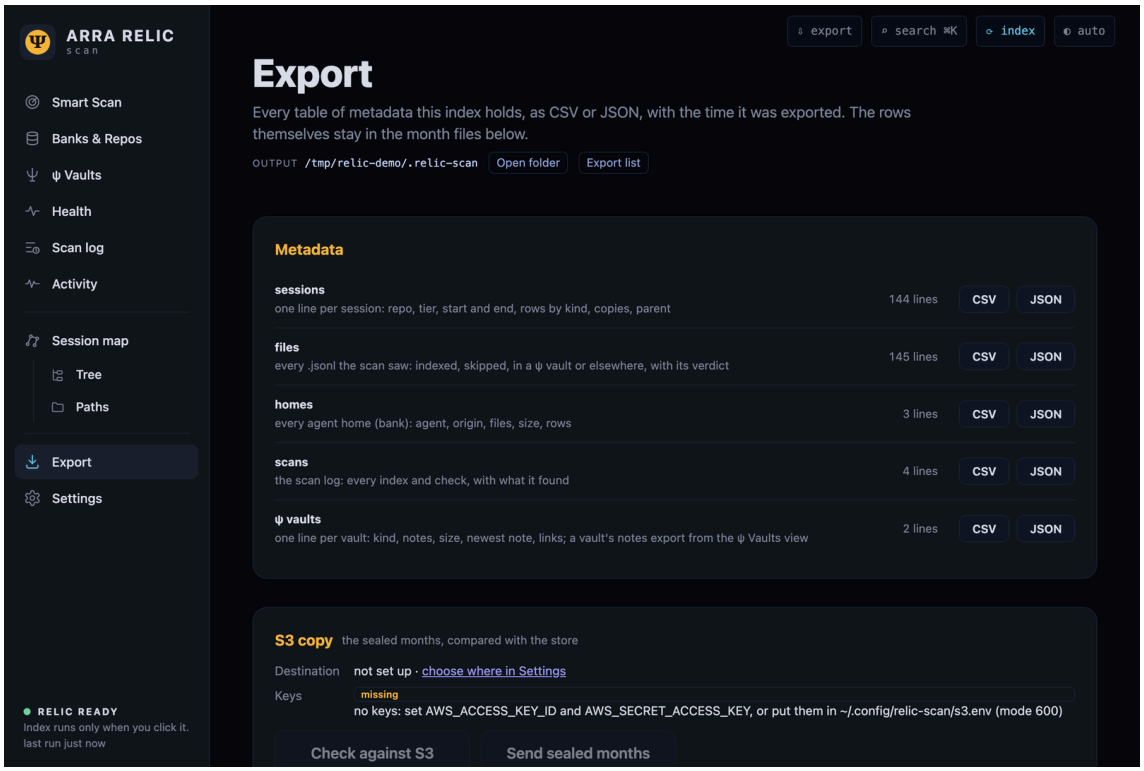
สามคู่ที่ต้องมองคือ `sealed_at` เวลาที่เห็นว่าปิด `sealed_rows` จำนวนแถวตอนนั้น และ `sealed_keys` hash ของคีย์แถวตอนนั้น **ถ้า `keys` กับ `sealed_keys` เท่ากัน แถวก็เท่ากัน** ใน JSON นั้น `keys` ของเดือน 2026-07 คือ `deb9bcc29d4996bd545ea8297889d90d` และ `sealed_keys` เป็นค่าเดียวกัน field `late_rows` คือแถวที่รอบนี้เพิ่มเข้ามาโดยมีเวลาเก่ากว่าแถวใหม่สุดเดิมของเดือน ปกติเป็น 0 ถ้าไม่เป็น 0 ก็คือเดือนที่ปิดแล้วมีของแทรกมา และ `resealed` จะขยับตาม

## 7.2 หน้า Export: metadata และสำเนา S3

พอรู้แล้วว่าเดือนไหนปิด ก็ถึงเวลาเอาออกไปข้างนอกได้ หน้า Export ของเว็บเป็นที่รวมของทั้งสองอย่าง คือ metadata ที่ดึงเป็นไฟล์ได้ทันที กับแผงสำเนา S3 เปิดหน้านี้ด้วย `relic-scan up` หรือ `relic-scan serve` แล้วเลือก Export ในเมนูซ้าย

### ขั้นที่ 1 — ดึง metadata เป็น CSV หรือ JSON

ตารางบนสุดของหน้าคือ Metadata แต่ละบรรทัดมีปุ่ม CSV กับ JSON ให้ดึงไปใช้ต่อ เช่น เอาไปเปิดใน spreadsheet




รูปที่ 55: หน้า Export ตาราง Metadata และแผง S3 copy ที่ยังไม่ตั้งปลายทาง

ตารางนี้ให้ **sessions 144 บรรทัด, files 145, homes 3, scans 4 และ ψ vaults 2** ตัวเลขชุดเดียวกับที่เห็นในหน้า Tree, Paths และ Vaults ของบทก่อน ๆ ข้อมูลที่ออกมาเป็นแค่ metadata ของ index ไม่ใช่เนื้อข้อความใน transcript

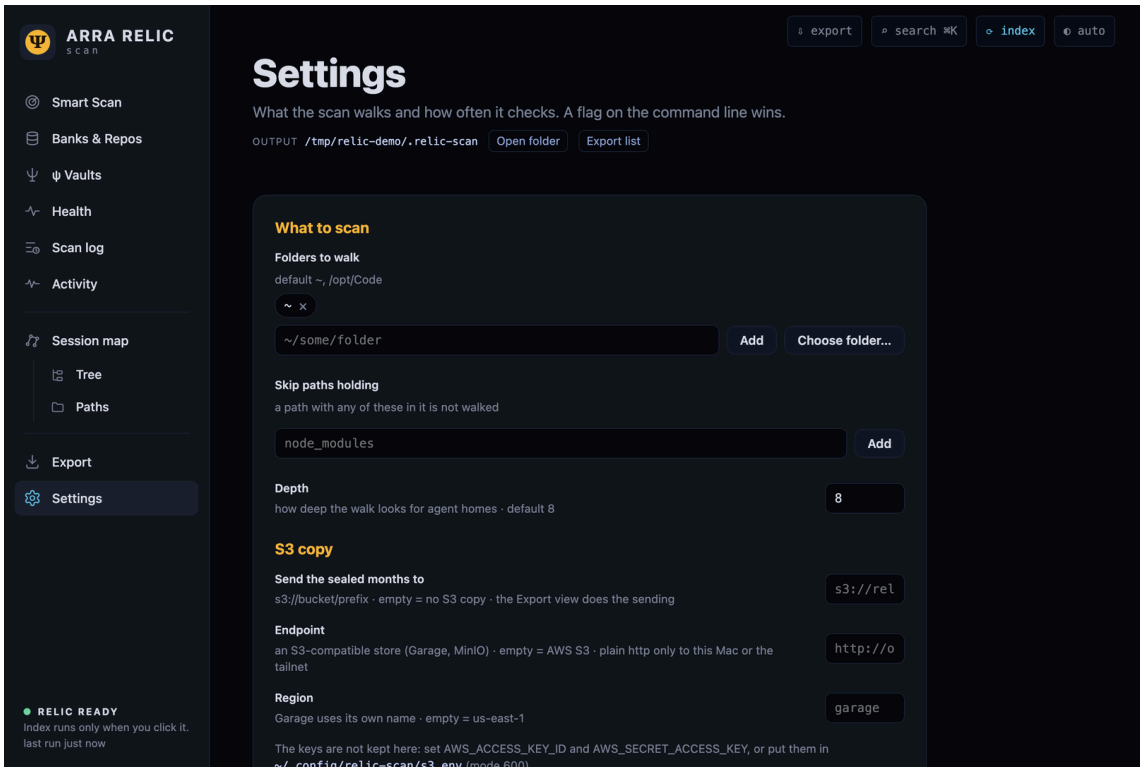
## ขั้นที่ 2 — อ่านแผง S3 copy

ใต้ตาราง คือแผง S3 copy ในภาพมันบอกว่ายังไม่ได้ตั้งปลายทาง และยังไม่มีคีย์ ปุ่ม Check against S3 กับ Send sealed months จึงใช้ไม่ได้ (ปุ่มส่งจางอยู่) นี่ตรงกับที่ AGENTS.md เขียนไว้ว่า **คีย์ไม่ถูกเก็บในหน้านี้เลย** หน้าเว็บตั้งได้แค่ปลายทาง

เมื่อตั้งครบแล้ว Check against S3 จะเทียบ store กับไฟล์ในเครื่อง และแสดงทีละเดือนว่า identical, older หรือ missing ส่วน Send sealed months จะส่งเฉพาะเดือนที่ปิดแล้ว คนที่ส่งคำขอไป S3 คือ server ไม่ใช่หน้าเว็บ หน้ายังคงไม่เรียกเครือข่ายเอง และการเขียนฝั่ง server ต้องมี header `X-Relic` ตามกฎของโค้ด

## ขั้นที่ 3 — ตั้งปลายทางในหน้า Settings

ปลายทางตั้งที่หน้า Settings ในส่วน S3 copy มีสามช่อง คือ destination, endpoint และ region ใน AGENTS.md ค่าเหล่านี้ถูกเก็บในไฟล์ `_config.json` ในชื่อ `s3.to`, `s3.endpoint` และ `s3.region`




รูปที่ 56: หน้า Settings ส่วน What to scan และส่วน S3 copy

ภาพนี้ยังเห็นส่วน What to scan ด้วย (โฟลเดอร์ที่เดิน skip paths และ depth 8) ซึ่งเป็นเรื่องของบทที่ 4 ส่วนที่เกี่ยวกับบทนี้คือส่วนล่าง ช่อง endpoint ใช้เมื่อ store ไม่ใช่ AWS S3 จริง เช่น Garage หรือ MinIO ที่ต้องมี endpoint แบบ path-style ถ้าไม่ตั้ง relic-scan ถือว่าเป็น AWS S3

ส่วนคีย์ AGENTS.md ให้วางไว้สองที่เท่านั้น คือใน environment ( `AWS_ACCESS_KEY_ID` กับ `AWS_SECRET_ACCESS_KEY` ) หรือในไฟล์ `~/.config/relic-scan/s3.env` เป็นบรรทัด `KEY=VALUE` ที่ตั้งสิทธิ์ 600 ไฟล์ที่คนอื่นอ่านได้ relic-scan จะปฏิเสธไม่ใช้ ไฟล์นี้คือที่ที่แอปซึ่งเปิดจาก Finder ใช้อ่านคีย์

## 7.3 ส่งไป S3 จากบรรทัดคำสั่ง

ในบ้านเดโมไม่มี store ให้ส่ง บล็อกในหัวข้อนี้จึงคัดมาจาก AGENTS.md ตามตัวอักษร ไม่ได้รันในการสาธิต และไม่มีภาพผลลัพธ์ให้ดู ผลลัพธ์จริงบนเครื่องของคุณอาจต่างไปจากที่คิดไว้ ให้อ่านผลบนหน้าจอของตัวเองเป็นหลัก

### ขั้นที่ 1 — ตั้งปลายทางและคีย์ในสภาพแวดล้อม

ก่อนส่ง ตั้งตัวแปรสองสามตัวใน shell **คีย์ห้ามพิมพ์ในบรรทัดคำสั่ง** เพราะจะค้างอยู่ใน history และ process list

```
export RELIC_S3_ENDPOINT=http://odin.follow-rankine.ts.net:3920
export AWS_ACCESS_KEY_ID=… AWS_SECRET_ACCESS_KEY=…
```

`RELIC_S3_ENDPOINT` ไม่ต้องตั้งถ้าใช้ AWS S3 จริง ส่วน `RELIC_S3_REGION` มีค่าเริ่มต้น us-east-1 ตัวอย่างข้างบนคือ Garage ที่ตั้งอยู่บน tailnet ตามที่ AGENTS.md ใช้อธิบาย และ **http ธรรมดาอนุญาตเฉพาะเครื่องนี้หรือ tailnet (`*.ts.net`, `100.64/10`)** นอกนั้นต้องใส่ `--insecure-http`

ตัวเลือกทั้งหมดของ `export` ดูได้จาก `--help` (ผลนี้รันจริง ต่างจากบล็อกด้านล่าง)

```
relic-scan export --help
```

```
$ relic-scan export --help
Copy the sealed months and their seals to an S3-compatible store (AWS S3, Garage, MinIO); …
… re-runnable, nothing

Usage: relic-scan export [OPTIONS] --to <TO>

Options:
      --out <OUT>            [default: ~/.relic-scan]
      --kind <KIND>          the --kind the run used: picks ~/.relic-scan/<kind> when --out is …
      … not given
      --to <TO>              where to: s3://bucket/prefix. Endpoint and keys come from …
      … RELIC_S3_ENDPOINT, AWS_
      --endpoint <ENDPOINT>  the S3 endpoint (default: RELIC_S3_ENDPOINT, else AWS S3)
      --region <REGION>
      --dry-run              only look (HEAD): say what would be sent, send nothing
      --overwrite            replace a sealed month that differs in the store (it was sealed …
      … again after a lat
      --verify               compare the store with the local files; send nothing; exit 1 when …
      … they differ
      --insecure-http        allow plain http to a host that is neither this machine nor on the …
      … tailnet
      --list                 say what the store holds, one line per month (reads meta/months.json …
      … and export-m
  -h, --help                 Print help
(exit 0 · 0.0 s)
```

รูปที่ 57: ผลของ relic-scan export --help: ตัวเลือก --out, --kind, --to, \--endpoint, --region, --dry-run, --overwrite, --verify, \--insecure-http, --list

ในภาพ `--to` เป็นตัวเลือกบังคับ (ในบรรทัด Usage) ส่วนที่เหลือที่ขั้นต่อ ๆ ไปใช้ คือ `--dry-run`, `--verify`, `--list`, `--overwrite` และ `--insecure-http` ส่วน `--endpoint` กับ `--region` ใช้แทนตัวแปร `RELIC_S3_ENDPOINT` และ `RELIC_S3_REGION`

### ขั้นที่ 2 — ลองก่อนด้วย --dry-run

คำสั่งแรกที่ควรรันคือ `--dry-run` มัน**ส่งแค่คำขอ HEAD ไปถาม store** แล้วบอกว่าจะส่งอะไร ไม่เขียนอะไรลง store

```
relic-scan export --to s3://bucket/relic --dry-run
```

ตัวอย่างนี้ไม่ได้รันในเดโม เพราะไม่มี store ผลที่ได้คือรายการเดือนที่จะถูกส่ง ถ้า store มีเดือนนั้นอยู่แล้วและ sha256 ตรง มันก็ไม่ถูกส่งซ้ำ

### ขั้นที่ 3 — ส่งจริง

พอพอใจกับรายการ ก็ตัดคำว่า `--dry-run` ออก

```
relic-scan export --to s3://bucket/relic
```

คำสั่งนี้ส่งเดือนที่ปิดแล้ว จากนั้นส่ง `meta/months.json` (ผนึก) และ `export-manifest.json` แต่ละ object เก็บ sha256 ไว้ใน `x-amz-meta-sha256` จึง **รันซ้ำแล้วไม่ส่งของที่อยู่ใน store แล้ว** เดือนที่ยังเปิดอยู่ไม่ถูกส่ง และ relic-scan ไม่ list หรือลบอะไรใน store เลย

### ขั้นที่ 4 — ตรวจและดูรายการ

หลังส่ง ใช้ `--verify` **เทียบ store กับไฟล์ในเครื่อง** ถ้าต่างกันคำสั่งจะออกด้วยรหัส 1 หรือใช้ `--list` ดูว่า store มีอะไรบ้าง

```
relic-scan export --to s3://bucket/relic --verify
relic-scan export --to s3://bucket/relic --list
```

`--list` พิมพ์หนึ่งบรรทัดต่อเดือน มีจำนวนแถว วันที่ ขนาด sha256 และผนึก โดยอ่านแค่สองไฟล์เล็ก ไม่ list ทั้ง store

### ขั้นที่ 5 — เมื่อเดือนที่ปิดแล้วต้องปิดใหม่

บางครั้งมีแถวช้ามาถึงเดือนที่ปิดไปแล้ว ผนึกจึงเปลี่ยน (`resealed` เพิ่ม) เดือนนั้นใน store ก็ต่างจากไฟล์ในเครื่อง ตอนนี้ **export จะปฏิเสธการเขียนทับ** เว้นแต่ใส่ `--overwrite` ตามที่ AGENTS.md ระบุ

```
relic-scan export --to s3://bucket/relic --overwrite
```

scripts/README.md บอกเรื่องเดียวกันถึงงานสำรองตอนกลางคืน `relic-export-nightly` ที่ timer รัน มันไม่ทับเดือนที่ปิดแล้ว ถ้าอยากทับต้องรัน

`--overwrite` ด้วยมือ นี่เป็นการออกแบบที่ตั้งใจ เพื่อไม่ให้งานอัตโนมัติเขียนทับสำเนาเก่าที่ยังดีอยู่โดยไม่มีใครเห็น

เรื่องสำเนาไปข้างนอกจบแค่นี้ บทต่อไปดูทางเดินทั้งเส้นของข้อมูล ตั้งแต่ relic-scan ไปถึง hub และดูว่า `SYNC OK` บอกอะไร

# บทที่ 8: ทางเดินของข้อมูลข้ามเครื่อง

relic-scan ไม่ได้ทำงานโดดเดี่ยว บนเครื่องของทีมเขา มันเป็นขั้นแรกของ chain สามขั้นที่ส่งข้อมูลต่อกันไปจนถึง hub ที่รวมประวัติจากหลายเครื่อง บทนี้เปิดดูว่า chain ทำอะไรอยู่ตอนนี้ด้วย `relic-scan ops` หน้า Activity และแท็บ ops ในเทอร์มินัล แล้วอธิบายว่าแต่ละขั้นคืออะไร และเส้นสุดท้าย `SYNC OK` ยืนยันอะไร

ตัวเลขทุกตัวในบทนี้มาจากการรันบนบ้านเดโมในเครื่อง m5 ของทีม ซึ่งมี chain รันอยู่จริง เนื้อหาเรื่อง relic-extract และ hub มาจาก docs/TUTORIAL.md กับ docs/TALK.md ไม่มีตัวเลขเพิ่มเติมนอกจากที่เห็นในภาพ

## 8.1 relic-scan ops: ดู chain ทั้งเส้น

`relic-scan ops` แสดงว่า chain กำลังทำอะไร ขั้นไหนใช้เวลาเท่าไร มี process อะไรรันอยู่ และ hub อยู่ในสถานะไหน มันเป็นคำสั่งอ่านอย่างเดียวเหมือน `seals`

### ขั้นที่ 1 — รัน ops โดยไม่แตะ hub

ปกติ ops อ่านสถานะของ hub ด้วยคำขอ GET หนึ่งครั้ง ซึ่งเป็นการเรียกเครือข่ายอย่างเดียวของคำสั่งนี้ ถ้าไม่อยากให้ออกไปเลย ใส่ `--no-hub` ตัวแปร `RELIC_OPS_HUB=off` ให้ผลเหมือนกัน

```
relic-scan ops --no-hub
```

```
$ relic-scan ops --no-hub
relic ops · 07:23:00

CHAIN m5  idle  last pass 06:50:15 took 1:31 · next in about 0:00
  ✓ scan                31 s (typical 31 s)     scan 0.2s · new 0 · grown 3 · rewritten 0 · …
  … unchanged 145 · gone 0 · 0.00 GB to read
  ✓ extract             14 s (typical 14 s)     337 chunks · 12 messages cleaned out
  ✓ push to the hub      5 s (typical 5 s)
  ✓ hub index + embed
  ✓ sync check                                   SYNC OK  text|1256 thinking|507 tools|1410

PROCESSES  none running

HUB  not read (off)

page  http://127.0.0.1:8787/ops
(exit 0 · 0.0 s)
```

รูปที่ 58: ผล relic-scan ops ส่วน CHAIN ห้าขั้นพร้อมเวลา, PROCESSES none running และ HUB not read

บรรทัดแรกของ chain บอกเครื่อง สถานะ และรอบล่าสุด

```
CHAIN m5  idle  last pass 06:50:15 took 1:31 · next in about 0:00
```

อ่านได้ว่า **chain บนเครื่อง m5 ว่างอยู่ รอบล่าสุดเริ่ม 06:50:15 ใช้เวลา 1:31** `idle` แปลว่าไม่มีขั้นไหนกำลังทำงานตอนนี้ ส่วน `HUB  not read (off)` ตรงบรรทัดล่างเป็นผลของ `--no-hub` ไม่ใช่ hub ล่ม

### ขั้นที่ 2 — อ่านห้าขั้นทีละบรรทัด

ใต้บรรทัดหัวมีห้าขั้น แต่ละขั้นมีเครื่องหมายถูก เวลาที่ใช้ครั้งนี้ ตามด้วยเวลาปกติ (`typical`) ในวงเล็บ แล้วเป็นรายละเอียดด้านขวา

```
✓ scan                  31 s (typical 31 s)   scan 0.2s · new 0 · grown 3
…
✓ extract               14 s (typical 14 s)     337 chunks · 12 messages
…
✓ push to the hub        5 s (typical 5 s)
✓ hub index + embed
✓ sync check                                     SYNC OK  text|1256 …
```

เวลา **ครั้งนี้เท่ากับเวลาปกติทุกขั้น** แปลว่ารอบนี้ไม่ช้ากว่าเคย ถ้าขั้นไหนช้าผิดปกติ ตัวเลขสองตัวนี้ต่างกันทันที ให้ดูขั้น scan ที่รายละเอียดเป็นสรุปของ `relic-scan run` ที่เราคุ้นจากบทที่ 2 คือ new 0, grown 3, rewritten 0, unchanged 145, gone 0 เปิดไฟล์ที่ต้องอ่านเพิ่ม 0.00 GB นั่นคือรอบนี้ transcript โตแค่ 3 ไฟล์

### ขั้นที่ 3 — ขั้นที่เหลือและ process

ขั้น extract รายงานว่า**ตัดได้ 337 chunks** และตัดข้อความออกมาได้ 337 chunks และกรองทิ้ง 12 messages ขั้น push to the hub ส่งผลไปที่ hub ขั้น hub index + embed ไม่แสดงรายละเอียดฝั่งนี้เพราะงานเกิดบน hub เอง ส่วนบรรทัดที่สี่ `PROCESSES  none running` แปลว่าขณะที่รันคำสั่ง ไม่มี process ของ chain ค้างอยู่

ท้ายเอาต์พุตมีบรรทัด `page  http://127.0.0.1:8787/ops` ที่ชี้ไปหน้า Activity ถ้าเปิดบน terminal ที่รองรับ ลิงก์ทั้งสองคลิกได้ ตัวแปร `RELIC_LINKS=off` หรือ `NO_COLOR` ปิดลิงก์ และ `RELIC_PAGE_URL` ย้ายที่อยู่ของหน้า ใส่ `--watch` เมื่ออยากให้จอรีเฟรชต่อเนื่อง และ `--json` เมื่อต้องการให้เครื่องมืออื่นอ่าน

### ขั้นที่ 4 — ดูตัวเลือกของ ops

อยากรู้ว่า ops รับอะไรได้บ้าง ให้ถาม `--help` ตัวเลือก **`--no-hub` คือทางเดียวที่ทำให้คำสั่งนี้ไม่ใช้เครือข่ายเลย**

```
relic-scan ops --help
```

```
$ relic-scan ops --help
What the relic chain is doing right now: the pass in flight with its stages and ETA, running …
… processes, the hu

Usage: relic-scan ops [OPTIONS]

Options:
      --out <OUT>    [default: ~/.relic-scan]
      --kind <KIND>  the --kind the run used: picks ~/.relic-scan/<kind> when --out is not given
      --json         the snapshot as JSON (what the page and the menu-bar app read), once
      --watch        redraw every 2 seconds until Ctrl-C
      --no-hub       do not ask the hub (no network)
  -h, --help         Print help
(exit 0 · 0.0 s)
```

รูปที่ 59: ผล relic-scan ops --help แสดง Usage และตัวเลือก --out, --kind, --json, \--watch, --no-hub

มีตัวเลือกหกตัวรวม `--help` ตัวที่ใช้บ่อยคือ `--watch` ที่วาดจอใหม่ทุก 2 วินาทีจนกด Ctrl-C และ `--json` ที่พิมพ์สแนปช็อตเป็น JSON หนึ่งครั้งให้หน้าเว็บกับแอปเมนูบาร์อ่าน ส่วน `--out` กับ `--kind` ชี้ไปโฟลเดอร์ index ที่ run ใช้

อยากเทียบผล ให้รันแบบไม่แตะ hub อีกครั้ง

```
relic-scan ops --no-hub
```

```
$ relic-scan ops --no-hub
relic ops · 07:23:00

CHAIN m5  idle  last pass 06:50:15 took 1:31 · next in about 0:00
  ✓ scan                 31 s (typical 31 s)     scan 0.2s · new 0 · grown 3 · rewritten 0 · …
  … unchanged 145 · gone 0 · 0.00 GB to read
  ✓ extract              14 s (typical 14 s)     337 chunks · 12 messages cleaned out
  ✓ push to the hub       5 s (typical 5 s)
  ✓ hub index + embed
  ✓ sync check                                   SYNC OK  text|1256 thinking|507 tools|1410

PROCESSES  none running

HUB  not read (off)

page  http://127.0.0.1:8787/ops
(exit 0 · 0.0 s)
```

รูปที่ 60: ผล relic-scan ops --no-hub ซ้ำจากขั้นที่ 1 ห้าขั้นของ chain และ SYNC OK

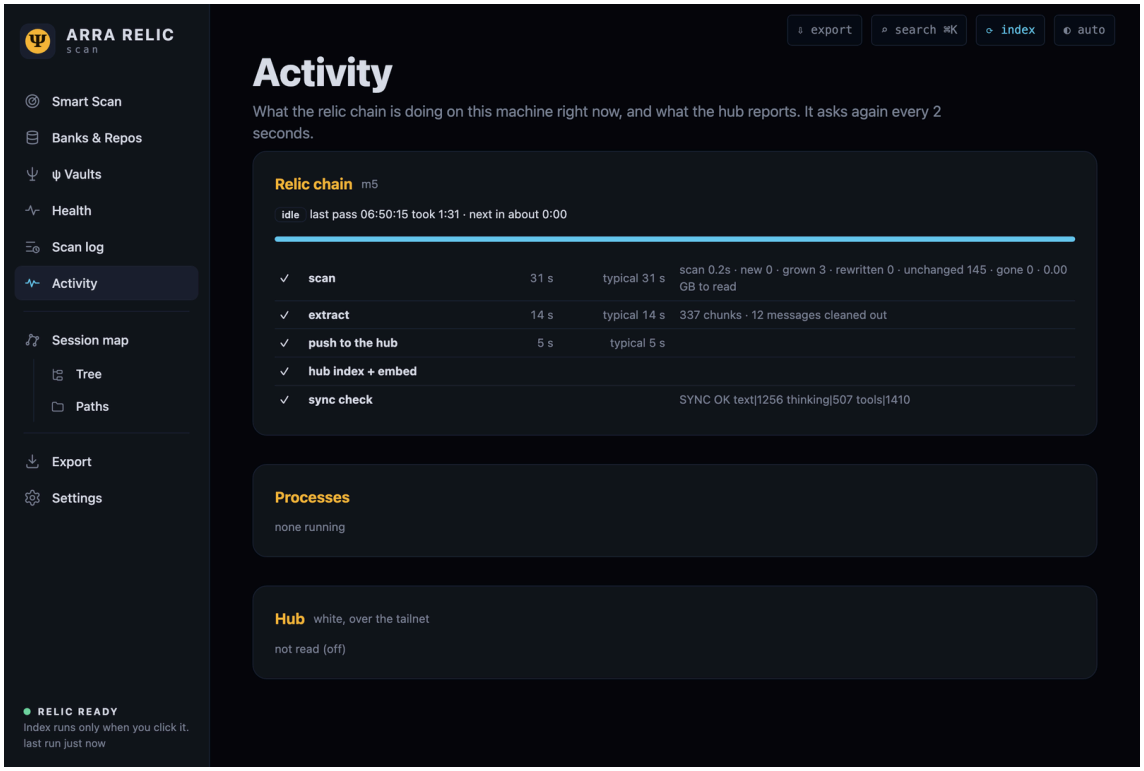
ภาพนี้ซ้ำจากขั้นที่ 1 เพื่อให้เทียบกับ `--no-hub` ตรงหน้า: ได้เอาต์พุตห้าขั้นเดิม ไม่มี HUB ที่อ่านจากเครือข่าย

## 8.2 หน้า Activity

หน้า Activity แสดงข้อมูลชุดเดียวกันบนเว็บ เปิดจาก `relic-scan up` แล้วเลือก Activity ในเมนูซ้าย ที่อยู่คือ `/ops` ตามบรรทัด page ข้างบน

### ขั้นที่ 1 — เปิดหน้า Activity

หน้านี้แบ่งเป็นสามการ์ด คือ Relic chain, Processes และ Hub หน้าเว็บเองไม่เรียกเครือข่าย ส่วนการ์ด Hub อ่านสถานะด้วย GET หนึ่งครั้งตามที่ AGENTS.md ระบุ




รูปที่ 61: หน้า Activity การ์ด Relic chain ห้าขั้นพร้อมเวลา, Processes ไม่มี, Hub ไม่ได้อ่าน

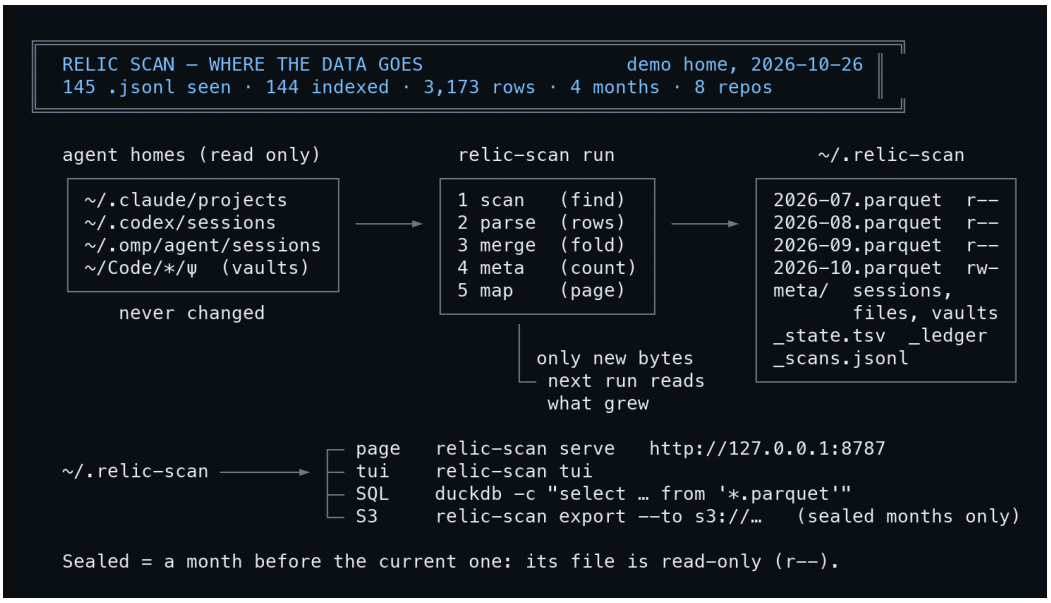
ภาพนี้บอกเรื่องเดียวกับ `ops` **การ์ด Relic chain มีห้าขั้นเรียงจาก scan จนถึง sync check** การ์ด Processes ว่างเปล่า และการ์ด Hub ขึ้นว่าไม่ได้อ่าน เพราะปิดไว้ ข้อดีของหน้านี้คือเปิดทิ้งไว้ได้โดยไม่ต้องพิมพ์ซ้ำ และเห็นรอบใหม่เข้ามาเอง

## 8.3 จาก relic-scan ไปถึง hub

chain สามขั้นตามที่ docs/TUTORIAL.md เขียนไว้ แต่ละขั้นอ่านผลของขั้นก่อนโดยไม่เคยแก้มัน

```
 relic-scan                relic-extract                relic4
embed
```

```
find + index        ──►  pick, clean, dedupe, cut  ──►  vectors, search
~/.relic-scan/            ~/.relic-extract/<stream>/
```




รูปที่ 62: แผนภาพ WHERE THE DATA GOES: จาก agent homes ผ่าน relic-scan run ห้าขั้น ไปยัง ~/.relic-scan แล้วออกทางหน้าเว็บ, tui, SQL และ S3

ภาพนี้เห็นสองอย่างที่ร้อยแก้วไม่ได้บอก คือ **โฟลเดอร์ต้นทางไม่ถูกแก้เลย** และไฟล์เดือนที่ปิดแล้วเป็น `r--` มีแต่เดือนปัจจุบัน `rw-` ตัวเลขบนหัวภาพคือ 145 ไฟล์ที่เห็น, 144 ที่ทำ index, 3,173 แถว, 4 เดือน และ 8 repo

ผังนี้คัดมาจากบทนำของ TUTORIAL.md คำสั่งต่อไปนี้ไม่ได้รันในการสาธิต เพราะ relic-extract กับ agent-relic-v4 เป็น repo ส่วนตัวในตอนนี้ ผู้อ่านที่ไม่มีสิทธิ์เข้าจึงรันขั้นสองและสามเองไม่ได้ ส่วน relic-scan เป็นขั้นเดียวที่เปิดสาธารณะ

### ขั้นที่ 1 — relic-scan หาและทำ index

ขั้นแรกคือสิ่งที่เล่มนี้เล่ามาตลอด คือหา transcript ของ Claude Code, Codex, OMP, OMX, Gemini, Pi และ thClaws บนเครื่อง พร้อม ψ vault ทุกแห่ง แล้วทำเป็น Parquet รายเดือนใน `~/.relic-scan` และเพราะ ledger ทำให้ **สำเนาที่มาจากเครื่องอื่นเสียแค่ค่า hash ไม่ต้องเสียค่า parse** รอบประจำวันของ chain จึงเร็ว

### ขั้นที่ 2 — relic-extract ตัดเป็น chunk

ขั้นที่สองอ่านไฟล์เดือนของ relic-scan แล้ว**ตัดเป็นหนึ่งแถวต่อหนึ่ง chunk** แล้วเขียนส่วนที่ต้องเอาไป embed ออกมาเป็นหนึ่งแถวต่อหนึ่ง chunk ไฟล์ TOML เดียวกำหนดว่าจะ

เอาอะไรเข้า และเครื่องมือจะทำความสะอาด ตัดข้อความซ้ำ และตัดชิ้น เอาต์พุตอยู่ใน `~/.relic-extract/<stream>/YYYY-MM.parquet` มีสาย text, thinking และ tools

บรรทัด `337 chunks · 12 messages cleaned out` ในภาพ 20-ops คือผลของขั้นนี้ในรอบล่าสุด และตัวเลข text, thinking, tools ที่ปลายบรรทัด SYNC OK ก็เป็นสายเดียวกันนี้

### ขั้นที่ 3 — push แล้ว hub ทำ index และ embed

ขั้น push **ส่งผลไปที่ hub** (repo `relic5` พอร์ต 8795 บนเครื่อง white ตาม TALK.md) ขั้น hub index + embed รันบน hub เอง ใช้ relic4 ที่ embed เฉพาะ key ที่ยังไม่มี vector ข้อความเดียวกันจึงได้ vector เดียวกันไม่ว่าอยู่เดือนไหน

TALK.md เล่าด้วยว่าสามส่วนนี้มี agent ประจำ repo ของตัวเอง relic4-scan ดูแล index relic5 ดูแล hub และ relic-extract ดูแลข้อความที่ตัดออกมา ทุกฝ่ายอยู่ใน repo ของตัวเองและขอการเปลี่ยนแปลงจากกัน ข้อควรจำจากเอกสารนั้นมีข้อเดียวที่เกี่ยวกับผู้ใช้ปกติ คือ **ถ้าตัวเลขบน hub ดูผิด ให้เช็ก index ของเครื่องเองก่อน** ด้วย DuckDB บน `~/.relic-scan/meta/sessions.parquet` ถ้าเครื่องถูก บั๊กอยู่ที่การรวมข้อมูลบน hub

## 8.4 ตรวจว่า sync ตรงกัน

บรรทัดสุดท้ายของ chain คือการยืนยันว่าสิ่งที่เครื่องส่ง กับสิ่งที่ hub ถืออยู่ ตรงกัน

### ขั้นที่ 1 — อ่านบรรทัด SYNC OK

ในภาพ 20-ops ขั้น sync check ขึ้นว่า

```
SYNC OK  text|1256 thinking|507 tools|1410
```

สามตัวเลขนี้เป็นสาย text, thinking และ tools ตามลำดับ และบวกกันได้ 1256 + 507 + 1410 = 3,173 เท่ากับจำนวนแถวบนหน้า Smart Scan ของบ้านเดโม TALK.md ระบุว่า **ทุกรอบของ chain จบด้วยบรรทัด SYNC OK แบบนี้** ที่ m5 และ mba ถูกเขียนไว้ใน `~/Library/Logs/relic-chain.log` ส่วนบน white อยู่ที่ `~/.local/state/relic-chain.log` และ `/api/status` ของ hub (`freshness.machines.*.sync`) ต้องแสดงเลขสามตัวเดียวกัน

### ขั้นที่ 2 — ดูแท็บ ops ในเทอร์มินัล

แท็บที่หกของ `relic-scan tui` คือ ops ใช้ข้อมูลเดียวกัน รีเฟรชทุก 2 วินาที และมี process ที่รันอยู่ให้ดูเพิ่ม

```
relic-scan tui
```

ภาพของแท็บนี้อยู่ในบทที่ 5 หัวข้อ 5.4 ห้าขั้นของ chain ที่เห็นตรงกับ `relic-scan ops` ทุกบรรทัด

ในกรอบ ops **ส่วน** `PROCESSES` **ขึ้นว่า** `none running` และ `HUB  not read (off)` เหมือนกับ `relic-scan ops --no-hub` ตัวเลข SYNC OK ในจอนี้ (`text|1256 thinking|507 tools|1410`) ตรงกับที่เห็นใน `relic-scan ops` ทุกตัว เพราะอ่านจากกรอบเดียวกัน

### ขั้นที่ 3 — เมื่อเลขสองฝั่งไม่ตรงกัน

TALK.md ให้เกณฑ์ไว้ชัด **เลขไม่ตรงที่ค้างเกินสองรอบ**คือเรื่องที่ควรแจ้งทีม relic5 ส่วนคำเตือน `relic-scan differs between machines` ที่ขึ้นหลังติดตั้งเวอร์ชันใหม่ จะหายเองเมื่อแต่ละเครื่อง push ด้วยไฟล์ใหม่แล้ว

อีกข้อที่ทีมเจอจริงคือการเปลี่ยนไฟล์ binary ที่กำลังรันอยู่ TALK.md แนะให้ `rm -f` แล้วค่อย `cp` และทำหลังจาก build ตัวใหม่เสร็จเท่านั้น เพราะบนเครื่อง white เคยมีครั้งที่ `cp` ล้มหลัง `rm` แล้ว relic-scan หายไปหกนาที ดูรายละเอียดเรื่องนี้อีกครั้งในหัวข้อ 9.4

ถึงตรงนี้ได้เห็นแล้วว่าข้อมูลเดินไปไหนบ้าง บทสุดท้ายของภาคนี้ตอบคำถามที่ควรถามก่อนรันคำสั่งแรก คือ relic-scan แตะอะไรบนเครื่องคุณ

# บทที่ 9: ความเป็นส่วนตัว และข้อควรระวัง

transcript ของเอเจนต์คือบันทึกงานจริงของคุณ มีคำสั่งที่พิมพ์ ไฟล์ที่เปิด และบางทีก็มีคีย์ที่เผลอวางลงไป เครื่องมือที่ไล่อ่านทุกไฟล์พวกนี้จึงต้องตอบให้ได้ว่า มันอ่านอะไร เขียนอะไร และส่งอะไรออกไปข้างนอกหรือเปล่า บทนี้ตอบคำถามเหล่านั้นด้วยกฎในโค้ดและของที่เห็นในเดโม แล้วปิดด้วยปัญหาที่ทีมเจอจริงจากเอกสารของ repo

หมายเหตุเรื่องข้อมูล บ้านเดโมของเล่มนี้เป็นของสมมติทั้งหมด สร้างโดย `make-demo-home.py` มี transcript 144 ไฟล์ และมีความลับปลอมอยู่เพียงอันเดียว คือคีย์ `sk-demo-…` ในหนึ่ง prompt เพื่อใช้แสดงหัวข้อ 9.2

## 9.1 อ่านอย่างเดียว ไม่แก้ ไม่เรียกเน็ต

กฎของโค้ดใน AGENTS.md มีสี่ข้อที่เกี่ยวกับความเป็นส่วนตัวโดยตรง หนึ่ง transcript และ vault เป็นอ่านอย่างเดียว สอง relic-scan เขียนเฉพาะในโฟลเดอร์ผลลัพธ์ของตัวเอง คือ `~/.relic-scan` สาม หน้าเว็บไม่เรียกเครือข่ายและไม่โหลด library จากข้างนอก สี่ การเขียนฝั่ง server ต้องมี header `X-Relic`

### ขั้นที่ 1 — ดูก่อนว่าจะอ่านอะไรด้วย --dry-run

ก่อนรันครั้งแรก AGENTS.md ให้ถามเจ้าของเครื่องก่อน เพราะคำสั่ง `run` อ่านทุก transcript ใต้โฟลเดอร์บ้าน ทางที่ปลอดภัยคือรัน `--dry-run` ก่อน มันสแกนแล้วบอกว่าจะอ่านอะไร โดยไม่เขียนอะไรเลย

```
relic-scan run --dry-run
```




รูปที่ 63: กล่อง RELIC SCAN ผล dry run: would read 144 files, new 144, nothing written, ปลายทาง ~/.relic-scan

ภาพนี้ซ้ำจากบทที่ 1 อ่านแล้วเห็นว่า new 144 ส่วน grown, rewritten, unchanged และ gone เป็น 0 หมด เพราะยังไม่เคยทำ index ตัวเลข 0.00 GB คือขนาดที่จะอ่าน ซึ่งเล็กมากในบ้านเดโม

กล่องในภาพมีสองบรรทัดที่เป็นหัวใจ

```
| dry run · an index would read 144 files, 0.00 GB       |
| scan 0.2s · nothing written                            |
```

`nothing written` **คือคำยืนยันว่าการดูก่อนไม่แตะดิสก์** ส่วน 144 ไฟล์คือจำนวนที่สแกนเห็นและจะอ่านถ้ารันจริง ปลายทางที่ลูกศรชี้คือ `~/.relic-scan` ที่เดียว

### ขั้นที่ 2 — เมื่อไหร่ที่ข้อมูลออกจากเครื่อง

ข้อมูลออกจากเครื่องได้**สองทางเท่านั้น**ตาม AGENTS.md ทางแรกคือ `relic-scan export --to s3://…` ซึ่งคุณสั่งเอง ทางที่สองคือ `relic-scan ops` กับมุมมอง Activity ที่อ่านสถานะของ hub ด้วย GET หนึ่งครั้ง ตัวแปร `RELIC_OPS_HUB=off` หรือ `--no-hub` ข้ามขั้นนี้ ในบทที่ 8 เราใช้ `--no-hub` ตลอด

นอกจากสองทางนี้ ไม่มีการเรียกเครือข่าย ถ้าอยากพิสูจน์เอง ก็ดูได้ว่าหน้าเว็บให้บริการที่ `127.0.0.1` เท่านั้น ไม่เปิดให้เครื่องอื่น และทาง S3 ก็ไม่เคยเก็บคีย์ไว้ในไฟล์ตั้งค่า (หัวข้อ 7.2)

### ขั้นที่ 3 — ตรวจว่าสำเนาที่ปิดผนึกยังไม่ถูกแตะ

ความเป็นส่วนตัวมีอีกด้านคือความถูกต้อง ข้อมูลที่เปลี่ยนไปโดยไม่มีใครรู้ก็อันตราย `seals --verify` อ่านเดือนที่ปิดซ้ำและเทียบกับผนึก

```
relic-scan seals --verify
```

```
$ relic-scan seals --verify
month           rows  sealed at            resealed  status
2026-07          819  2026-10-26T00:22:59Z        0  sealed, matches
2026-08          762  2026-10-26T00:22:59Z        0  sealed, matches
2026-09          845  2026-10-26T00:22:59Z        0  sealed, matches
2026-10          747  –                           0  open
(exit 0 · 0.0 s)
```

รูปที่ 64: ผล relic-scan seals --verify ตารางสี่เดือน สามเดือนแรก sealed, matches เดือน 2026-10 เป็น open

ภาพนี้ซ้ำจากบทที่ 7 ตารางมีสี่เดือน แถวละหนึ่งเดือน ได้แก่ 819, 762, 845 และ 747 แถว เดือนสุดท้ายเป็น `open` เพราะยังไม่ปิด

เดือน 2026-07, 2026-08 และ 2026-09 ขึ้น `sealed, matches` ครบ ภาพนี้เป็นภาพเดียวกับที่ดูในบทที่ 7 แต่ที่นี่ใช้ในมุมต่างออกไป คือเป็นหลักฐานว่าก่อนส่งสำเนาไป S3 ไฟล์ที่ส่งตรงกับที่ปิดไว้ทุกแถว

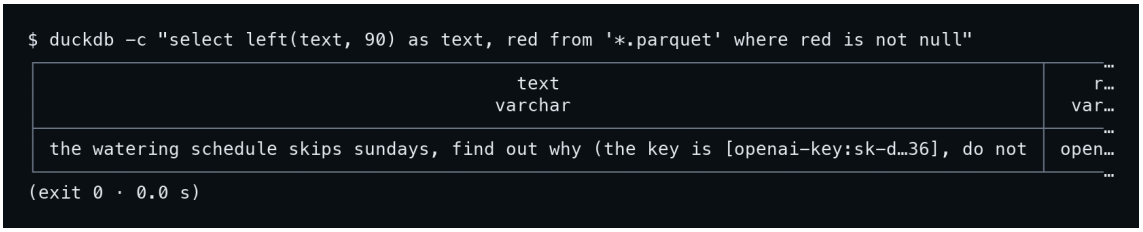
## 9.2 redact: ความลับถูกแทนด้วยป้าย

ในแถวของ index ที่ข้อความมีคีย์หลุดเข้ามา relic-scan ไม่ทิ้งคีย์ไว้ตามเดิม README บอกสั้น ๆ ว่าขั้นอ่านต่อบรรทัดทำ “redaction” ไว้พร้อม text, thinking และ tools ผลของมันเห็นได้ในคอลัมน์ `red` ของ Parquet

### ขั้นที่ 1 — ถามหาแถวที่ถูก redact

แถวที่ไม่ถูก redact มี `red` เป็น NULL ดังนั้นการกรองด้วย `red is not null` ได้เฉพาะแถวที่มีของลับถูกแทนที่

```
duckdb -c "select left(text, 90) as text, red from '*.parquet' \
  where red is not null"
```




รูปที่ 65: ผล duckdb แถวเดียวที่ red ไม่ว่าง ข้อความมี [openai-key:sk-d…36] และคอลัมน์ red เป็น openai-key

ภาพนี้ซ้ำจากบทที่ 6 ตารางมีแถวเดียว ข้อความถูกตัดที่ 90 ตัวอักษร และคีย์ปลอมในนั้นกลายเป็นป้ายแล้ว

ทั้งบ้านเดโมมีแถวเดียวที่ถูก redact ซึ่งเป็น prompt ที่ใส่คีย์ปลอมไว้ ในคอลัมน์ข้อความคีย์ไม่ปรากฏเต็ม

```
(the key is [openai-key:sk-d…36], do not …   openai-key
```

อ่านได้ว่า **คีย์ถูกแทนด้วยป้ายชนิด** `[openai-key:…]` ในป้ายเหลือแค่ส่วนต้น `sk-d…` กับเลขท้าย 36 เอกสารไม่ได้อธิบายรูปแบบป้ายละเอียดกว่านี้ จึงไม่ตีความเกินภาพ และคอลัมน์ `red` ระบุชนิดว่า `openai-key`

### ขั้นที่ 2 — อ่านผลอย่างระวัง

การ redact ช่วยให้ index ไม่ถือคีย์เต็มไว้ แต่มันเป็นการแทนที่ในแถวของ index เท่านั้น **ไฟล์ transcript ต้นฉบับไม่ถูกแก้** เพราะกฎ “อ่านอย่างเดียว” ดังนั้นคีย์เต็มยังอยู่ใน `.jsonl` ของเอเจนต์ตามเดิม ถ้าคีย์หลุดเข้าไปจริง ต้องหมุนคีย์ที่ต้นทาง ไม่ใช่หวังว่า index จะแก้ให้

AGENTS.md ยังเตือนอีกข้อสำหรับทุกคนที่ถาม index ว่า transcript เก็บงานส่วนตัว ให้อ่านเฉพาะสิ่งที่เจ้าของขอให้หา และไม่คัดลอกแถวออกจากเครื่อง

## 9.3 สำเนาซ้ำถูกพับเป็นแถวเดียว

เครื่องหนึ่งมักมี transcript ชุดเดียวกันหลายที่ เช่น mirror จากเครื่องอื่น ที่เก็บถาวร หรือสำรอง ถ้าแต่ละสำเนานับเป็นแถวแยก ตัวเลขทุกตัวจะเพี้ยนไปหมด และข้อความเดียวกันถูกอ่านซ้ำหลายรอบ

AGENTS.md อธิบายว่า **สำเนาของ transcript ในหลาย home ถูกพับเป็นแถวเดียว** แม้สำเนาจะต่างกันที่ metadata เช่น cwd ที่บันทึก คอลัมน์ `sources` ของแถวนั้นบอกทุกสำเนาที่พบ ในรูป `agent:path:line` จึงรู้ได้ว่าแถวหนึ่งมาจากไหนบ้าง

### ขั้นที่ 1 — เห็นผลของการพับในตัวเลข

**ตัวเลขไม่ถูกนับซ้ำ** ผลในเดโมคือบนหน้า Tree และ Paths ตัวเลขที่เห็นคือ 145 ไฟล์ 144 sessions กับ 3,173 แถว โดยไม่มีการนับซ้ำ เพราะเดโมไม่มี mirror จริง ผลของการพับเลยไม่เด่นในภาพ บทนี้ไม่แต่งตัวเลขการพับขึ้นมา

อีกชั้นหนึ่งคือ ledger ที่บทก่อนเล่าไว้ AGENTS.md ระบุว่าสำเนาที่ ledger จำได้จะไม่ถูกอ่านซ้ำ และถูกจดไว้ใน `_state.tsv` (คอลัมน์ `copy_of`) กับ `_ledger.tsv` แทนที่จะอยู่ใน `sources` ของแถว ถ้าอยากให้เห็นว่า relic-scan ข้ามอะไรไปบ้าง ใช้ `relic-scan log` ซึ่งนับ `copies`, `resumed` และ `past_changed` พร้อมรายการไฟล์ทีละไฟล์

### ขั้นที่ 2 — มองจากมุมความเป็นส่วนตัว

การพับช่วยสองเรื่อง หนึ่ง ข้อความส่วนตัวชิ้นเดียวไม่ถูกเพิ่มเป็นหลายแถวใน index เมื่อก๊อปอยู่หลายที่ สอง คุณรู้ว่าแถวนั้นมีรอยอยู่ที่ไหนบ้างจาก `sources` ถ้าต้องไปจัดการ เช่น ลบสำเนาเก่า ต้องรู้ว่ามีสำเนาที่ไหน

แต่ข้อควรระวังคือ การลบแถวออกจาก index ไม่ลบไฟล์ต้นฉบับ และการลบไฟล์ต้นฉบับก็ไม่ได้ลบแถวใน index ทันที ที่มาของเรื่องนี้อยู่ที่หัวข้อต่อไป

## 9.4 ปัญหาที่ทีมเจอจริง และทางแก้

หัวข้อนี้รวมเฉพาะข้อควรระวังที่เขียนไว้ในเอกสารของ repo เอง ไม่มีข้อไหนมาจากการเดาของผู้เขียนเล่ม ทุกข้อบอกที่มา เรียงตามโอกาสที่คุณจะเจอ

### ขั้นที่ 1 — เดือนที่ปิดแล้วต่างจากใน store

ที่มา AGENTS.md ส่วน “Copy the sealed months” และ scripts/README.md อาการคือ เดือนที่ปิดไปแล้วถูกปิดใหม่หลังมีแถวช้ามาถึง สำเนาใน S3 จึงต่างจากไฟล์ในเครื่อง

```
A sealed month that differs in the store (it was sealed again after
a late row) is refused unless --overwrite.
```

บล็อกข้างบนคัดจาก AGENTS.md ตามตัวอักษร และไม่ได้รันในเดโม **ทางแก้คือรัน export ด้วย `--overwrite` ด้วยมือ** หลังตรวจแล้วว่าต้องการจริง งานสำรองตอนกลางคืนไม่ทับให้เอง

### ขั้นที่ 2 — index เก่ามีรูปร่างไม่เหมือนเวอร์ชันปัจจุบัน

ที่มา AGENTS.md ส่วน “Query the index” อาการคือ คอลัมน์ `model`, `msg_id`, `out_tokens` เป็น NULL ใน index ที่เขียนด้วยเวอร์ชัน v26.10.23-alpha.1858 หรือเก่ากว่า และเดือนที่เขียนก่อนมีคอลัมน์ `in_tokens`, `cache_read`, `cache_write` ก็อ่านเป็น NULL เช่นกัน ส่วน `meta/sessions.parquet` ก็ไม่มี `cwd`, `title`, `branch` หรือ `version` จนกว่าจะอ่านไฟล์ใหม่

ทางแก้ที่เอกสารให้: **ทำ index ใหม่ตั้งแต่ต้น** คือย้าย `~/.relic-scan` ออกไปข้าง ๆ แล้วรัน `relic-scan run` ใหม่

```
move ~/.relic-scan aside, then relic-scan run
```

### ขั้นที่ 3 — index ใหม่ทิ้งแถวของไฟล์ที่หายไป

ที่มา docs/TALK.md ส่วน “Who fixes what” อาการคือ index ใหม่ตั้งแต่ต้น**จะทิ้งแถวที่ไฟล์ต้นทางหายไปแล้ว** ดังนั้นถ้าคุณเคยลบหรือย้าย transcript เก่า ตัวเลขหลังทำใหม่

อาจน้อยลง ทางแก้ที่เอกสารระบุคือ เทียบจำนวนแถวก่อนสลับไป index ใหม่ เก็บของเก่าไว้ข้าง ๆ จนแน่ใจ

### ขั้นที่ 4 — รันบางช่วงเวลาแล้วหยุด หรือรันชนกัน

ที่มา docs/TUTORIAL.md ท้ายเล่ม ตารางเมื่อมีอะไรผิด สองข้อ ข้อแรก ถ้า run หยุดเพราะมีตัวอื่นถือ lock ซึ่งหมายความว่า **หนึ่ง writer ต่อหนึ่งโฟลเดอร์** ให้รอตัวนั้นเสร็จ ข้อสอง ถ้า run แบบช่วงหยุดด้วยข้อความ “indexed without one” แปลว่า `--month` และธงวันที่อื่น ๆ เขียนลงโฟลเดอร์ของตัวเอง ไม่ต้องตั้ง `--out`

ถ้าไฟล์ที่เจอน้อยกว่าที่คิด ตารางนั้นแนะนำให้เช็ก root ที่สแกน (ธง `--root` หรือหน้า Settings) และรัน `relic-scan log --diff 0`

### ขั้นที่ 5 — เปลี่ยน binary ที่กำลังรันอยู่

ที่มา scripts/README.md และ docs/TALK.md อาการคือ บน macOS ถ้า `cp` ทับ binary ที่กำลังรันอยู่ ระบบจะฆ่าโปรเซสนั้น ทางแก้ที่เขียนไว้คือ **ลบก่อนด้วย `rm -f` แล้วค่อย `cp`**

```
rm -f ~/.local/bin/relic-scan && cp target/release/relic-scan ~/.local/
bin/
```

บรรทัดข้างบนเป็นการเรียบเรียงตามคำแนะนำ ไม่ได้รันในเดโม เส้นทางปลายทาง `~/.local/bin/relic-scan` มาจากที่ install.sh ติดตั้ง TALK.md เตือนเพิ่มอีกชั้น ให้ทำหลัง build ตัวใหม่เสร็จแล้วเท่านั้น เพราะเคยมีครั้งที่ `cp` ล้มหลัง `rm` แล้ว relic-scan หายไปหกนาทีบนเครื่อง white

ครบทั้งเก้าบทของการใช้งาน การอ่าน index และการเดินข้อมูล ส่วนภาคผนวกต่อจากนี้ลงไปในรูปแบบของ transcript ที่ relic-scan อ่านจริง บรรทัดต่อบรรทัด

# บทที่ 10: รูปแบบของ transcript ที่ relic-scan อ่าน

บทนี้เป็นบท protocol ของหนังสือ ตอบคำถามเดียวว่า บรรทัดใน `.jsonl` หน้าตาเป็นยังไง แล้ว relic-scan หยิบ field ไหนไปทำเป็นแถวใน Parquet. ตัวอย่างทุกบรรทัดมาจาก `book/make-demo-home.py` ซึ่งสร้าง transcript ของ demo home ด้วยรูปทรงเดียวกับของจริง (ค่าในนั้นแต่งขึ้นหมด). ส่วนที่อ่านจากซอร์สคือ `src/engine.rs`, `src/meta/rules.rs`, `src/meta/agg.rs`, `src/parse/merge.rs`.

ป้ายที่ใช้ในบท: [capture] คือมีผลจริงใน `captures/` ให้เทียบ ส่วน [source] คือยืนยันจากการอ่านโค้ด ไม่ได้รันซ้ำในบทนี้.

## 10.1 Claude Code: หนึ่งบรรทัด หนึ่ง event

ไฟล์อยู่ใต้ `~/.claude/projects/<project-dir>/<session-uuid>.jsonl`. บรรทัดหนึ่งเป็น JSON object หนึ่งตัว มี `type` บอกชนิด (`user`, `assistant`, `custom-title`, `summary`) กับ `sessionId`, `timestamp`, `cwd`, `gitBranch`, `version`.

### ขั้นที่ 1 — ดูบรรทัด assistant ที่มี usage

บรรทัดข้างล่างสร้างจาก `claude_session()` ใน make-demo-home.py (ตัดให้สั้น จัดบรรทัดใหม่เป็น pretty JSON). **assistant หนึ่ง response แตกเป็นหลายบรรทัด** บรรทัดละ content block เดียว และทุกบรรทัดแปะ `message.id` กับ `usage` ชุดเดียวกันซ้ำ.


```
{
  "type": "assistant",
  "timestamp": "2026-07-03T09:14:22Z",
  "sessionId": "0af4d28e-a79c-4ad8-bb26-26a0708bfab4",
  "cwd": "/opt/Code/github.com/laris-co/kite-docs",
  "gitBranch": "main",
  "version": "2.1.40",
  "message": {
    "id": "msg_0af4d28e_0",
    "role": "assistant",
    "model": "claude-opus-5-5",
    "usage": {
      "input_tokens": 21,
```

```
      "output_tokens": 1180,
      "cache_read_input_tokens": 4200,
      "cache_creation_input_tokens": 900
    },
    "content": [
      { "type": "tool_use", "id": "toolu_0af4d2_0_0",
        "name": "Bash", "input": { "command": "ls" } }
    ]
  }
}
```

ตัวเลขใน usage ตัวอย่างนี้แต่งเพื่อให้อ่านง่าย ไม่ใช่ค่าจาก capture. สิ่งที่ relic-scan อ่านจากบรรทัดแบบนี้ [source: engine.rs `extract`, `usage_of`, `extract_tools`]:

- `sessionId` เป็นรหัส session, `timestamp` เป็นเวลา (เก็บทั้ง UTC และ `ts_th` เวลากรุงเทพ)
- `message.role` เป็น `user` หรือ `assistant`; ถ้าเป็น `developer`, `system` หรือ `toolResult` จะไม่เป็นแถวข้อความ
- `message.content` เป็น string ธรรมดา (ข้อความของ user) หรือ array ของ block: `text`, `thinking`, `tool_use`, `tool_result`
- `message.model`, `message.id`, `usage.output_tokens`, `input_tokens`, `cache_read_input_tokens`, `cache_creation_input_tokens`
- `cwd`, `gitBranch`, `version` ไปลง `meta/sessions.parquet`; ชื่อ session มาจากบรรทัด `custom-title`

### ขั้นที่ 2 — ดูผลลัพธ์ของเครื่องมือกลับมา

ผลของ tool มาเป็นบรรทัด `type: user` ที่ content เป็น array ของ `tool_result`. คู่กับ call ด้วย `tool_use_id` ซึ่งตรงกับ `id` ของ `tool_use`. **`is_error: true` คือที่มาของคอลัมน์ `err`.**

```
{
  "type": "user",
  "sessionId": "0af4d28e-a79c-4ad8-bb26-26a0708bfab4",
  "message": {
    "role": "user",
    "content": [
      { "type": "tool_result",
        "tool_use_id": "toolu_0af4d2_0_0",
        "content": "README.md  src",
        "is_error": false }
```

```
    ]
  }
}
```

บรรทัดไหนที่ JSON พัง หรือบรรทัดสุดท้ายที่ยังเขียนไม่จบ relic-scan จะไม่เอา: ตัดที่ `\n` ตัวสุดท้าย เว้นแต่ท้ายไฟล์ parse เป็น JSON ได้ [source: parse/mod.rs `complete`]. รอบหน้าไฟล์โตขึ้นก็อ่านต่อจาก offset เดิม ผลเท่ากับอ่านใหม่ทั้งไฟล์ (มี test `incremental_equals_cold` ใน parse/tests.rs).

## 10.2 Codex rollout: ห่อด้วย payload

Codex เก็บเป็น `~/.codex/sessions/YYYY/MM/DD/rollout-<เวลา>-<uuid>.jsonl` (หรือ `.jsonl.zst` ที่เก็บแล้ว ซึ่งถอดระหว่างอ่าน). ทุกบรรทัดมี `type` ชั้นนอก กับ `payload` ชั้นใน ไม่มี `message`.

### ขั้นที่ 1 — สามชนิดบรรทัดที่ต้องรู้

`session_meta` ให้รหัส session (`payload.id`) กับ `cwd`. `turn_context` ให้ชื่อโมเดล (`payload.model`). `response_item` คือเนื้อหา. **Codex ไม่เขียนโมเดลบนข้อความเอง** relic-scan จึงจำโมเดลจาก `turn_context` ล่าสุดแล้วแปะให้แถวที่ตามมา.

```
{ "type": "session_meta", "timestamp": "2026-07-05T03:00:00Z",
  "payload": { "id": "5d1c…", "cwd": "/Users/demo/Code/moss-cli" } }
{ "type": "turn_context", "timestamp": "2026-07-05T03:00:00Z",
  "payload": { "model": "gpt-6-astra",
               "cwd": "/Users/demo/Code/moss-cli" } }
{ "type": "response_item", "timestamp": "2026-07-05T03:01:10Z",
  "payload": { "type": "message", "role": "assistant",
    "content": [ { "type": "output_text", "text": "Added." } ] } }
```

ส่วน tool ของ Codex อยู่ใน `response_item` เหมือนกัน แต่ `payload.type` เป็น `function_call` (ชื่อใน `name`, อาร์กิวเมนต์ใน `arguments`), `custom_tool_call`, และผลเป็น `function_call_output` จับคู่ด้วย `call_id`. ส่วน `reasoning` อ่านจาก `summary` เพราะเนื้อคิดจริงเข้ารหัส [source: engine.rs `extract_tools`, `thinking_of`, `turn_model`].

ผลที่เห็นบนจอ [capture 16]: แถวของ `gpt-6-astra` นับ `calls` ได้ 0 และ `out_tokens` เป็น `NULL` ทุกเดือน เพราะไม่มี `msg_id` และไม่มี usage ให้เก็บ.

## 10.3 omp: message ชั้นเดียว ชื่อ field ต่างออกไป

omp (และ pi) เก็บใต้ `~/.omp/agent/sessions/<project>/<เวลา>_<uuid>.jsonl`. บรรทัดเป็น `type: "message"` แล้วมี `message` ข้างใน หน้าตาคล้าย Claude แต่ id กับ usage ใช้ชื่อสั้นกว่า.

### ขั้นที่ 1 — ดูชื่อ field ที่ต่าง

```
{
  "type": "message",
  "id": "a0",
  "timestamp": "2026-07-04T10:02:31Z",
  "message": {
    "role": "assistant",
    "model": "glm-5.3",
    "responseId": "r_95f119_0",
    "usage": { "input": 412, "output": 230,
               "cacheRead": 1800, "cacheWrite": 0 },
    "content": [ { "type": "text", "text": "Done." } ]
  }
}
```

เทียบชื่อ field ของ Claude กับ omp (จาก `usage_of` ใน engine.rs **[source]**):

- response id: Claude `message.id`, omp `message.responseId`
- output: `usage.output_tokens` กับ `usage.output`
- input: `usage.input_tokens` กับ `usage.input`
- cache read: `usage.cache_read_input_tokens` กับ `usage.cacheRead`
- cache write: `usage.cache_creation_input_tokens` กับ `usage.cacheWrite`

โค้ดลองชื่อแบบ Claude ก่อน ไม่เจอก็ใช้ชื่อแบบ omp. ผลของ tool ใน omp มาเป็น `message.role == "toolResult"` พร้อม `toolCallId`, `toolName`, `isError` และ call เป็น block ชนิด `toolCall` (อาร์กิวเมนต์ใน `arguments`). แถวพวกนี้เป็น tool row อยู่แล้ว จึงไม่นับเป็นข้อความซ้ำอีก.

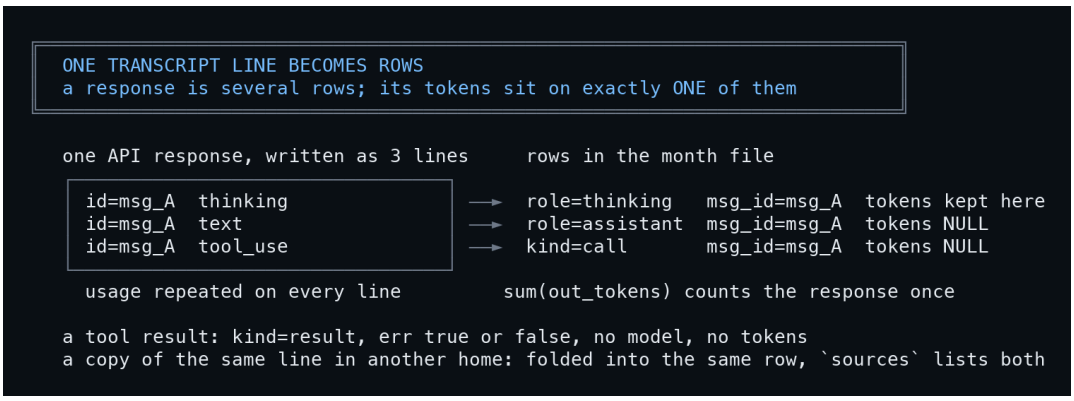
## 10.4 บรรทัดหนึ่งกลายเป็นแถวได้ยังไง

### ขั้นที่ 1 — อ่านกฎที่ตัดสินชนิดของแถว

บรรทัดหนึ่งแตกได้เป็นหลาย “stream”: `text` (ข้อความ user/assistant), `thinking`, และ `tools` (call กับ result). ตามที่เห็นใน `types_of` ของ agg.rs ตัวเดียวนี้ตัดสินว่าแถวนั้นนับเป็นอะไร:

```
pub(super) fn types_of(s: &str, role: Option<&str>, kind: Option<&str>,
err: bool) -> [i64; 6] {
    let mut t = [0; 6];
    match (s, role, kind) {
        ("text", Some("user"), _) => t[0] = 1,
        ("text", Some("assistant"), _) => t[1] = 1,
        ("thinking", _, _) => t[2] = 1,
        ("tools", _, Some("call")) => t[3] = 1,
        ("tools", _, Some("result")) => {
            t[4] = 1;
            t[5] = err as i64;
        }
        _ => {}
    }
    t
}
```

อ่านแล้วได้กฎตามนี้ [source]: ข้อความมี `role` แต่ไม่มี `kind`; tool มี `kind` (`call` หรือ `result`) แต่ไม่มี `role`; thinking ไม่ขึ้นกับ role. error คือ result ที่ `err` จริง จึงถูกนับใน results ด้วย.



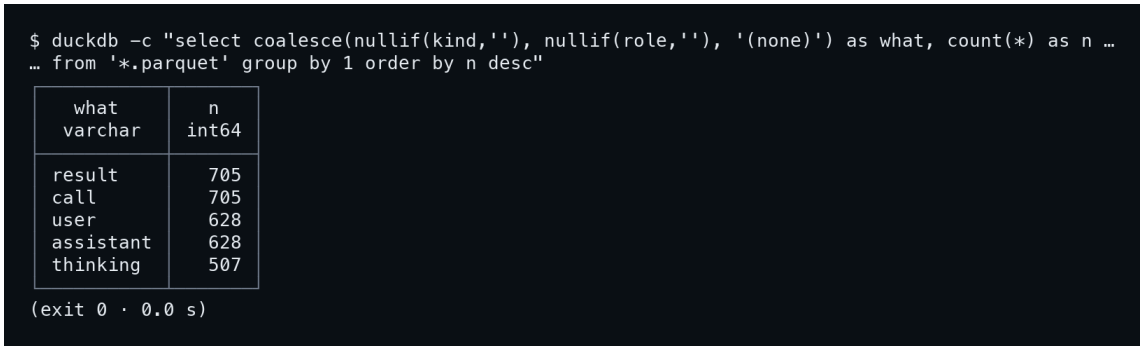

รูปที่ 66: แผนภาพ ONE TRANSCRIPT LINE BECOMES ROWS: response msg_A สามบรรทัด thinking, text, tool_use กลายเป็นสามแถว token อยู่แถว thinking แถวเดียว

ภาพนี้เห็นสิ่งที่โค้ดข้างบนไม่ได้บอกตรง ๆ คือ สามบรรทัดของ msg_A กลายเป็นสามแถว แต่ token อยู่แถว thinking แถวเดียว อีกสองแถวเป็น NULL และสำเนาจากอีก home พับเข้าแถวเดิม

### ขั้นที่ 2 — เทียบกับตัวเลขบนจอ

รันคำสั่งนับแถวตามชนิดที่ AGENTS.md แนะนำ แล้วดูว่าแต่ละบรรทัดตรงกับ `types_of`.

```
duckdb -c "select coalesce(nullif(kind,''), nullif(role,''), '(none)') \
  as what, count(*) as n from '*.parquet' group by 1 order by n desc"
```




รูปที่ 67: ผล duckdb นับแถวตามชนิด result 705, call 705, user 628, assistant 628, thinking 507

ภาพนี้ซ้ำจากบทที่ 6

**[capture 13]** result 705, call 705, assistant 628, user 628, thinking 507 รวม 3,173 แถว เท่ากับตัวเลขบนหน้า Smart Scan. call กับ result เท่ากันเพราะ demo จับคู่ครบทุกตัว. thinking มี 507 แถวแม้บางแถวข้อความว่าง ตรงกับที่ Claude Code รุ่นใหม่เขียน thinking เป็นแค่ signature (แถวยังอยู่ `text` ว่าง).

บรรทัด assistant เดียวที่มี thinking, text, tool_use สามบล็อกจึงได้สามแถว. ถ้าเป็นข้อความที่ส่งต่อกันหลายเครื่อง สำเนาของบรรทัดเดียวกันถูกรวมเป็นแถวเดียว โดยกุญแจคือ hash `blake2b` แปดไบต์ของตัวบรรทัด ( `engine::key` ) และ `sources` เก็บทุกสำเนา.

### ขั้นที่ 3 — ดูคอลัมน์ของไฟล์เดือน

อยากรู้ว่าแถวมีคอลัมน์อะไรบ้าง ให้ `describe` ไฟล์เดือนได้เลย

```
duckdb -c "describe select * from '2026-10.parquet'"
```




รูปที่ 68: ผล describe ไฟล์ 2026-10.parquet ตารางชื่อคอลัมน์กับชนิด 24 แถว

ไฟล์เดือนมี **24 คอลัมน์** ทั้ง `role`, `kind`, `red`, `sources`, `msg_id` และคอลัมน์ token สี่ตัว `out_tokens`, `in_tokens`, `cache_read`, `cache_write` เป็น BIGINT ส่วน `machines` กับ `sources` เป็น VARCHAR[]

## 10.5 token เก็บบนแถวเดียวต่อ msg_id

### ขั้นที่ 1 — ดูว่า merge เก็บ token ที่ไหน

เพราะ response เดียวแตกเป็นหลายบรรทัดที่แปะ usage ซ้ำ ถ้าบวกตรง ๆ จะนับเกินหลายเท่า. ตอน merge relic-scan จึงเก็บ token ไว้ที่แถวแรกของแต่ละ `msg_id` เท่านั้น แถวอื่นเป็น NULL. ข้อความจาก parse/merge.rs ตรงนี้:

```
    // one API response is several rows (one per content block, and every
copy of its lines): its output tokens stay on
    // the first of them only, the largest any copy recorded, so
sum(out_tokens) counts each response once
    let mut tokens: HashMap<&str, (usize, [Option<i64>; 4])> =
HashMap::new();
    for &i in &order {
        if let (Some(id), true) = (cands[i].msg_id.as_deref(),
cands[i].tok[0].is_some()) {
            let e = tokens.entry(id).or_insert((i, cands[i].tok));
```

```
            for k in 0..4 {
                e.1[k] = e.1[k].max(cands[i].tok[k]);
            }
        }
    }
```

สองจุดที่ต้องจำ: **ใช้ค่ามากสุดที่สำเนาไหนบันทึกไว้** ( `max` ), และ `count(distinct msg_id)` คือจำนวนครั้งที่เรียก API.

## ขั้นที่ 2 — นับ call และ token ต่อเดือนต่อโมเดล

```
duckdb -c "select left(ts_th,7) as month, model, \
  count(distinct msg_id) as calls, sum(out_tokens) as out_tokens \
  from '*.parquet' where model is not null \
  group by all order by month, calls desc limit 12"
```

```
$ duckdb -c "select left(ts_th,7) as month, model, count(distinct msg_id) as calls, …
… sum(out_tokens) as out_tokens from '*.parquet' where model is not null group by all order by …
… month, calls desc limit 12"
```

| month varchar | model varchar | calls int64 | out_tokens int128 |
|---|---|---|---|
| 2026-07 | claude-sonnet-5-5 | 52 | 70967 |
| 2026-07 | claude-haiku-5-5 | 45 | 50486 |
| 2026-07 | claude-opus-5-5 | 32 | 43302 |
| 2026-07 | glm-5.3 | 10 | 4925 |
| 2026-07 | gpt-6-astra | 0 | NULL |
| 2026-08 | claude-sonnet-5-5 | 52 | 65519 |
| 2026-08 | claude-haiku-5-5 | 35 | 48510 |
| 2026-08 | claude-opus-5-5 | 31 | 30756 |
| 2026-08 | glm-5.3 | 9 | 4371 |
| 2026-08 | gpt-6-astra | 0 | NULL |
| 2026-09 | claude-opus-5-5 | 61 | 75068 |
| 2026-09 | claude-haiku-5-5 | 52 | 63565 |

```
  12 rows                                  4 columns
(exit 0 · 0.0 s)
```

รูปที่ 69: ผล duckdb ตารางเดือน โมเดล calls และ out_tokens สิบสองแถว เริ่มที่ 2026-07 claude-sonnet-5-5

ภาพนี้ซ้ำจากบทที่ 6

**[capture 16]** เดือน 2026-07: claude-sonnet-5-5 มี 52 calls, 70967 out_tokens; claude-haiku-5-5 45 calls; claude-opus-5-5 32 calls; glm-5.3 10 calls, 4925. ส่วน `gpt-6-astra` ได้ 0 calls กับ `NULL` ตามที่อธิบายในหัวข้อ 10.2.

### ขั้นที่ 3 — รวมขึ้นระดับ session

ฝั่ง `agg.rs` มี struct `Usage` ที่บวก `out`, `inp`, `cache_read`, `cache_write` กับนับคำตอบต่อโมเดล. จากนั้นเขียนลง `meta/sessions.parquet` เป็น `models`, `model_msgs`, `in_tokens`, `out_tokens`, `cache_read_tokens`, `cache_write_tokens` [source].

```
duckdb -c "select sid, models, model_msgs, in_tokens, out_tokens, \
  cache_read_tokens from 'meta/sessions.parquet' \
  where out_tokens > 0 order by out_tokens desc limit 4"
```

```
$ duckdb -c "select sid, models, model_msgs, in_tokens, out_tokens, cache_read_tokens from …
… 'meta/sessions.parquet' where out_tokens > 0 order by out_tokens desc limit 4"
```

| sid<br>varchar | models<br>varchar[] | model_msgs<br>int64[] | in_tokens<br>int64 | out_tokens<br>int64 | cache_read_tokens<br>int64 |
|---|---|---|---|---|---|
| 95f11992 | [claude-haiku-5-5] | [7] | 121 | 11285 | 335324 |
| 28394ccd | [claude-sonnet-5-5] | [6] | 135 | 11250 | 246245 |
| 99aab31b | [claude-opus-5-5] | [6] | 129 | 10476 | 298801 |
| a9593467 | [claude-sonnet-5-5] | [7] | 171 | 10399 | 540510 |

```
(exit 0 · 0.0 s)
```

รูปที่ 70: ผล duckdb ตาราง session สี่แถว models, model_msgs, in_tokens, out_tokens, cache_read_tokens

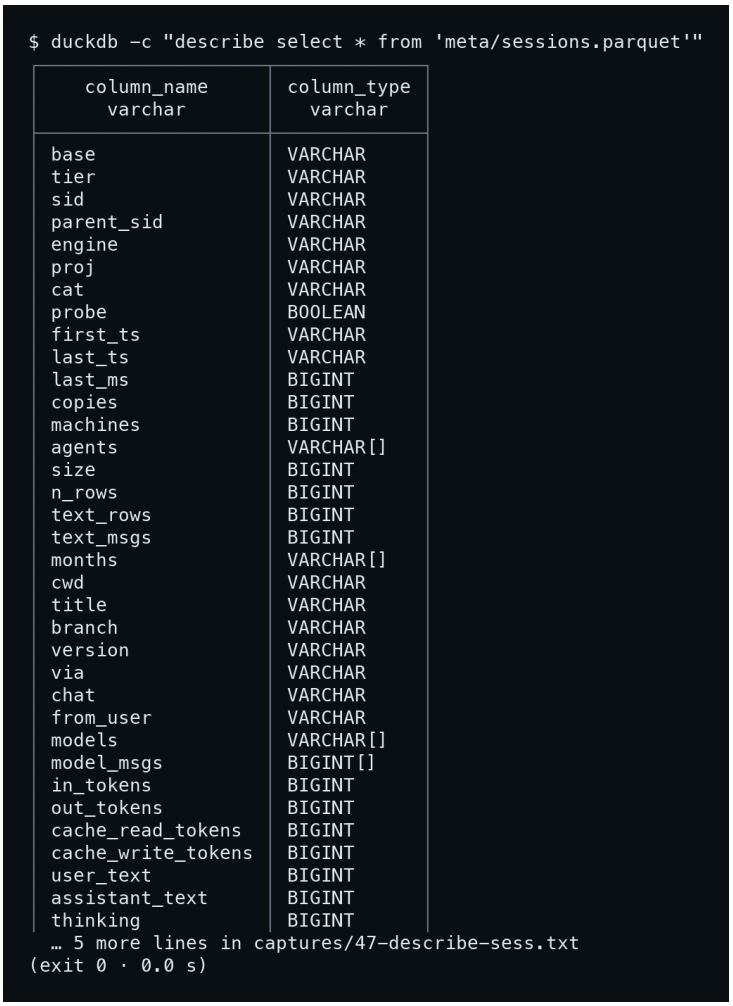
ภาพนี้ซ้ำจากบทที่ 6

[capture 17] session `95f11992` ใช้ `claude-haiku-5-5` ตอบ 7 ครั้ง ได้ `in_tokens` 121, `out_tokens` 11285, `cache_read_tokens` 335324. ลำดับ `models` เรียงจากโมเดลที่ตอบมากที่สุด (`ranked()` ใน agg.rs).

### ขั้นที่ 4 — ดูคอลัมน์ของ sessions

คอลัมน์ที่ขั้นก่อนเรียกใช้มาจากไฟล์ `meta/sessions.parquet` ซึ่ง `describe` ได้เหมือนกัน

```
duckdb -c "describe select * from 'meta/sessions.parquet'"
```




รูปที่ 71: ผล describe ไฟล์ meta/sessions.parquet ตารางชื่อคอลัมน์กับชนิด 38 แถว

ไฟล์นี้มี **38 คอลัมน์** รวม `models` กับ `model_msgs` ที่เป็น array, `cwd`, `title`, `branch`, `version` และตัวนับ `user_text`, `assistant_text`, `thinking`, `tool_calls`, `tool_results`, `tool_errors`

## 10.6 แยก session ด้วยเส้นทางไฟล์

relic-scan ไม่ได้เดาจากเนื้อใน: ตัดสินจาก **path ของไฟล์** ด้วย regex ชุดเดียวใน `rules.rs`. `tier_of` ตอบว่า main, subagent, workflow-agent หรือ other:

```
pub fn tier_of(path: &str) -> &'static str {
    let r = rx();
    if r.wf.is_match(path) {
        "workflow-agent"
    } else if r.sub.is_match(path) || r.sub_omp.is_match(path) {
        "subagent"
    } else if r.main_uuid.is_match(path) || r.main_rollout.is_match(path)
|| r.main_omp.is_match(path)
```

```
        || (r.main_other.is_match(path) && !r.history.is_match(path)) {
            "main"
        } else {
            "other"
        }
    }
```

ลำดับสำคัญ: เช็ก workflow ก่อน, แล้ว subagent (`/subagents/agent-….jsonl` ของ Claude หรือโฟลเดอร์ `<time>_<uuid>/<Task>.jsonl` ของ omp), แล้วถึง main (ไฟล์ชื่อ uuid, `rollout-…uuid.jsonl`, ไฟล์ omp). `history.jsonl` ของ omx/thclaws/gemini ตกเป็น `other`. subagent มี `parent_sid` ที่ดึงจาก uuid ใน path.

### ขั้นที่ 1 — repo (`cat`) มาจากชื่อโฟลเดอร์โปรเจกต์

Claude เข้ารหัส cwd เป็นชื่อโฟลเดอร์ เช่น `-opt-Code-github-com-laris-co-kite-docs`. `cat_of` ตัดส่วน `github-com-` ออก ตัดชื่อ org ที่รู้จัก ตัดหาง worktree แล้วเหลือ `kite-docs`. engine อื่นไม่มี cwd ใน path จึงได้ชื่อ engine ต่อท้ายว่า `(no cwd in path)`:

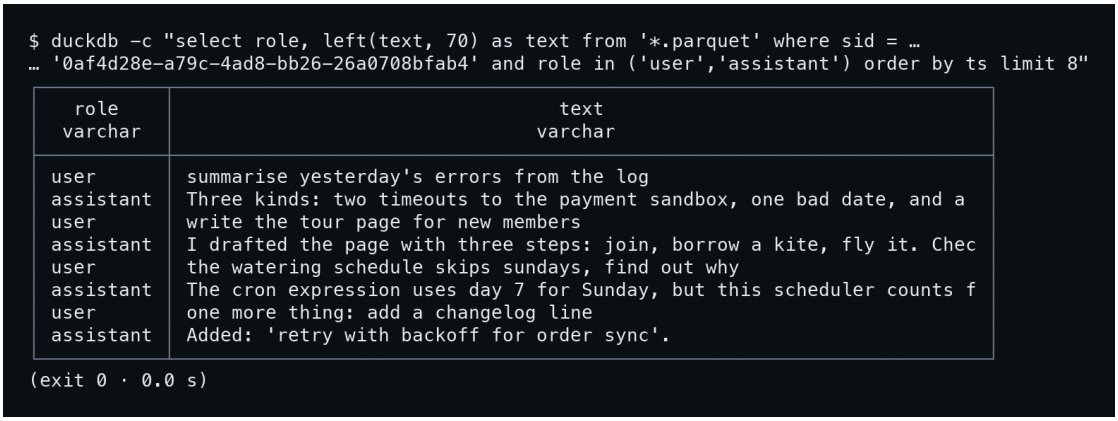
```
pub fn cat_of(engine: &str, proj: Option<&str>) -> String {
    let r = rx();
    if engine != "claude" {
        return format!("{engine} (no cwd in path)");
    }
    let Some(p) = proj.filter(|p| !p.is_empty()) else { return "(no
project dir)".into() };
    let lp = p.to_lowercase();
    if lp.contains("oracle-probe-cwd") {
        "probe (automated)".into()
    } else if lp.contains("github-com-") {
        let x = extract(&r.gh, &lp, 1);
        let x = r.gh_org.replace(&x, "");
        let x = r.gh_wt.replace(&x, "");
        r.dashes.replace_all(&x, "").into_owned()
    } else if lp.contains("-worktrees-") {
        let x = extract(&r.wts, &lp, 1);
        r.wts_wt.replace(&x, "").into_owned()
    } else {
        "home / other".into()
    }
}
```

ข้อควรระวัง [source]: regex ชื่อ org (`soul-brews-studio`, `laris-co`, ...) ฝังในโค้ด. repo ของ org อื่นที่ชื่อ path ขึ้นด้วย `github-com-` จะได้ชื่อยาวติด org ไว้. อีกอย่าง ชื่อโปรเจกต์ที่มีขีดอยู่แล้ว (`kite-docs`) ก็ยังอ่านออกเพราะตัดแค่ขีดหัวท้าย.

### ขั้นที่ 2 — เปิดอ่านบทสนทนาของ session หนึ่ง

พอรู้ว่า `sid` คือ uuid ใน path ก็ดึงบทสนทนาด้วย `sid` ได้เลย:

```
duckdb -c "select role, left(text, 70) as text from '*.parquet' \
  where sid = '0af4d28e-a79c-4ad8-bb26-26a0708bfab4' \
  and role in ('user','assistant') order by ts limit 8"
```




รูปที่ 72: ผล duckdb บทสนทนาแปดแถว user กับ assistant สลับกัน เริ่มที่ summarise yesterday's errors from the log

ภาพนี้ซ้ำจากบทที่ 6

**[capture 18]** แถวแรกเป็น `user` ข้อความ `summarise yesterday's errors from the log` แล้วตามด้วย `assistant` ที่ตอบ สลับกันครบแปดแถว. ผลนี้ยืนยันว่า `role` กับ `text` ที่ relic-scan ถอดจากบรรทัด `user` / `assistant` ตรงกับที่คนพิมพ์จริงใน transcript.

## ข้อสังเกตเรื่องความเชื่อถือได้

ทั้งหมดนี้เป็นการอ่านแบบอ่านอย่างเดียว relic-scan ไม่แก้ transcript. ไฟล์ที่ไม่เข้ารูปทรงข้างบนไม่ทำให้รอบสแกนล้ม: บรรทัดที่ parse ไม่ได้ถูกนับเป็น `bad_json` ใน `Stats` แล้วข้ามไป [source: parse/mod.rs]. ส่วนไฟล์ที่ไม่ใช่ transcript เลยจะติดป้าย verdict ว่า not-transcript ในหน้า Paths แทนที่จะถูกอ่านเป็นแถว.

ถ้าอยากเพิ่มรูปทรงใหม่ (agent ตัวอื่น) ทางสั้นสุดคือดูว่ามัน "คล้าย" ตัวไหนใน `extract` ก่อน: engine ที่เข้าตระกูล `message` ใช้ได้เลย ส่วนที่ห่อด้วย `payload` ต้องดูตัวอย่างของ Codex. ชื่อ field ของ usage ที่ต่างกันแก้จุดเดียวที่ `usage_of`.

## ต่อไป

ตอนนี้รู้แล้วว่าบรรทัดดิบเป็นแถวได้ยังไง บทถัดไปคือโค้ดตัวจริงที่เลือกมาพิมพ์ไว้ท้ายเล่ม ให้เปิดดูตรงกับที่ยกมาในบทนี้.

# ภาคผนวก: โค้ดที่เลือก

ไฟล์ด้านล่างพิมพ์ตามจริงทุกบรรทัด จาก commit `7ae3299` ของ Soul-Brews-Studio/relic4-scan ไม่ย่อ ไม่แก้ให้สวย เลือกเฉพาะส่วนที่บทต่าง ๆ ชี้ไปถึง

> relic-scan เป็นโอเพนซอร์ส (AGPL v3) โค้ดทั้งชุดอยู่ที่ https://github.com/Soul-Brews-Studio/relic4-scan

## src/engine.rs

how one transcript line becomes rows: the Claude, Codex and omp readers, redaction, usage · 526 บรรทัด

```
//! Port of jsonl-truth's line parser: src/relic4/engine.py (extract,
extract_tools, thinking_of, redact,
//! ts_of, stamp, key) as driven by src/relic4/store.py parse_chunk.
Every function mirrors its Python twin;
//! docs/OUTPUT-CHAIN.md lists the rules and verify/understand.py checks
them on real files.

use std::sync::OnceLock;

use chrono::{DateTime, Datelike, Timelike};
use regex::bytes::{Regex, RegexBuilder};
use serde_json::{Map, Value};

// ---------- gjson-like accessors (engine.py gstr / graw / gbool / get)
----------

fn dumps(v: &Value) -> String {
    serde_json::to_string(v).unwrap_or_default() // compact, key order
kept: orjson.dumps(...).decode()
}

/// gjson .String(): strings as is, missing/null as "", anything else as
its JSON.
pub fn gstr(v: Option<&Value>) -> String {
    match v {
        None | Some(Value::Null) => String::new(),
        Some(Value::String(s)) => s.clone(),
        Some(Value::Bool(b)) => if *b { "true".into() } else
{ "false".into() },
        Some(x) => dumps(x),
```

```rust
    }
}

fn graw(v: Option<&Value>) -> String {
    match v {
        None | Some(Value::Null) => String::new(),
        Some(x) => dumps(x),
    }
}

fn gbool(v: Option<&Value>) -> bool {
    match v {
        Some(Value::Bool(b)) => *b,
        Some(Value::String(s)) => s == "true",
        Some(Value::Number(n)) => n.as_f64().is_some_and(|f| f != 0.0),
        _ => false,
    }
}

fn get<'a>(o: Option<&'a Value>, k: &str) -> Option<&'a Value> {
    o.and_then(|x| x.as_object()).and_then(|m| m.get(k))
}

/// Go strings.TrimSpace = Unicode White_Space, the same set as Rust's
char::is_whitespace (not 0x1C-0x1F).
fn trim_go(s: &str) -> &str {
    s.trim_matches(char::is_whitespace)
}

/// Go's builder: a separator only once something non-empty was written.
fn join_nl<I: IntoIterator<Item = String>>(parts: I, sep: &str) -> String
{
    let mut out = String::new();
    for p in parts {
        if !out.is_empty() {
            out.push_str(sep);
        }
        out.push_str(&p);
    }
    out
}

// ---------- time ----------

const MONTHS: [&str; 13] = ["", "January", "February", "March", "April",
"May", "June", "July", "August", "September",
                            "October", "November", "December"];

fn parse_iso(t: &str) -> Option<i64> {
    // Python datetime.fromisoformat, tz-aware only (Go RFC3339 needs a
zone)
    let d = DateTime::parse_from_rfc3339(t)
        .or_else(|_| DateTime::parse_from_str(t, "%Y-%m-%dT%H:%M:
%S%.f%z"))
```

```rust
        .or_else(|_| DateTime::parse_from_str(t, "%Y-%m-%dT%H:%M:%S%z"))
        .or_else(|_| DateTime::parse_from_str(t, "%Y-%m-%dT%H:%M%z"))
        .ok()?;
    Some(d.timestamp_micros().div_euclid(1000)) // (d - EPOCH) // MS:
floor to the millisecond
}

/// Epoch milliseconds, or None. Numbers: seconds (thclaws) or
milliseconds.
pub fn ts_of(r: &Map<String, Value>) -> Option<i64> {
    match r.get("timestamp") {
        Some(Value::Number(n)) => {
            let f = n.as_f64()?;
            Some(if f > 1e12 { n.as_i64().unwrap_or(f as i64) } else { (f
* 1000.0) as i64 })
        }
        Some(Value::String(t)) if t.chars().count() > 10 &&
t.chars().nth(10) == Some('T') => parse_iso(t),
        _ => None,
    }
}

pub struct Stamp {
    pub month: String, // "2026-10" or "undated"
    pub ts: String,    // UTC, 24 chars, Z
    pub ts_th: String, // +07:00
}

pub fn stamp(ms: Option<i64>) -> Stamp {
    let Some(ms) = ms else { return Stamp { month: "undated".into(), ts:
String::new(), ts_th: String::new() } };
    let u = DateTime::from_timestamp_millis(ms).unwrap_or_default();
    let b = DateTime::from_timestamp_millis(ms + 7 * 3600 *
1000).unwrap_or_default();
    let f = format!(".{:03}", ms.rem_euclid(1000));
    let _ = MONTHS;
    Stamp {
        month: format!("{:04}-{:02}", b.year(), b.month()),
        ts: format!("{:04}-{:02}-{:02}T{:02}:{:02}:{:02}{f}Z", u.year(),
u.month(), u.day(), u.hour(), u.minute(), u.second()),
        ts_th: format!("{:04}-{:02}-{:02}T{:02}:{:02}:{:02}{f}+07:00",
b.year(), b.month(), b.day(), b.hour(), b.minute(), b.second()),
    }
}

// ---------- extract ----------

fn content_text(c: Option<&Value>) -> String {
    match c {
        Some(Value::String(s)) => s.clone(),
        Some(Value::Array(a)) => join_nl(
            a.iter().filter(|it| it.is_object() && matches!
(gstr(it.get("type")).as_str(), "text" | "input_text" | "output_text"))
                .map(|it| gstr(it.get("text"))),
```

```
                "\n",
            ),
            _ => String::new(),
        }
    }

    pub struct Extract {
        pub typ: String,
        pub role: String,
        pub sess: String,
        pub ts: Option<i64>,
        pub text: String,
    }

    /// Claude Code, Codex, omp/pi and thclaws shapes -> (type, role,
    session, ts_ms, text).
    pub fn extract(r: &Map<String, Value>) -> Extract {
        let typ = gstr(r.get("type"));
        let ts = ts_of(r);
        let mut sess = gstr(r.get("sessionId"));
        let payload = r.get("payload");
        if typ == "session_meta" {
            sess = gstr(get(payload, "id"));
        } else if typ == "session" || typ == "header" {
            sess = gstr(r.get("id"));
        } else if typ == "summary" {
            let text = gstr(r.get("summary"));
            return Extract { typ, role: String::new(), sess, ts, text };
        }
        let (mut role, mut c) = (String::new(), None);
        if let Some(Value::Object(m)) = r.get("message") {
            role = gstr(m.get("role"));
            c = m.get("content");
        } else if typ == "response_item" && gstr(get(payload, "type")) ==
    "message" {
            role = gstr(get(payload, "role"));
            c = get(payload, "content");
        } else if typ == "user" || typ == "assistant" {
            role = typ.clone();
            c = r.get("content");
        }
        // developer/system text is not conversation; an omp tool result
    (role "toolResult") is already a tool row
        // (extract_tools), so it is not a text row as well
        if role == "developer" || role == "system" || role == "toolResult" {
            return Extract { typ, role, sess, ts, text: String::new() };
        }
        let text = trim_go(&content_text(c)).to_string();
        Extract { typ, role, sess, ts, text }
    }

    /// Which model answered, and the API response a line belongs to. Claude
    Code and omp/pi write them on the assistant
    /// message: Claude `model`, `id`, `usage.output_tokens` (repeated on
```

```rust
every content-block line of one response); omp
/// `model`, `responseId`, `usage.output`. Codex writes no model on its
messages: see turn_model.
#[derive(Default, Debug, PartialEq)]
pub struct Usage {
    pub model: String,
    pub msg_id: String,
    pub out_tokens: Option<i64>,
    /// input tokens and the prompt cache (Claude `input_tokens`,
`cache_read_input_tokens`, `cache_creation_input_tokens`;
    /// omp `input`, `cacheRead`, `cacheWrite`)
    pub in_tokens: Option<i64>,
    pub cache_read: Option<i64>,
    pub cache_write: Option<i64>,
}

pub fn usage_of(r: &Map<String, Value>) -> Usage {
    let Some(Value::Object(m)) = r.get("message") else { return
Usage::default() };
    if gstr(m.get("role")) != "assistant" {
        return Usage::default();
    }
    let id = gstr(m.get("id"));
    let u = m.get("usage");
    Usage {
        model: gstr(m.get("model")),
        msg_id: if id.is_empty() { gstr(m.get("responseId")) } else
{ id },
        out_tokens: get(u, "output_tokens").or_else(|| get(u,
"output")).and_then(Value::as_i64),
        in_tokens: get(u, "input_tokens").or_else(|| get(u,
"input")).and_then(Value::as_i64),
        cache_read: get(u, "cache_read_input_tokens").or_else(|| get(u,
"cacheRead")).and_then(Value::as_i64),
        cache_write: get(u, "cache_creation_input_tokens").or_else(||
get(u, "cacheWrite")).and_then(Value::as_i64),
    }
}

/// What a line says about its session as a whole: where it ran, what it
is called, the branch, the Claude version.
/// Claude writes `cwd`, `gitBranch` and `version` on its lines and the
name as a `custom-title` (or `ai-title`) line;
/// Codex writes `cwd` in the payload of its session_meta and
turn_context lines.
#[derive(Default, Debug, Clone, PartialEq)]
pub struct SessMeta {
    pub cwd: String,
    pub title: String,
    pub branch: String,
    pub version: String,
    /// a message that came in through a channel plugin (Discord,
Telegram, a fleet bridge): the plugin, the room and
    /// the sender of the first such message of the session
```

```
    pub via: String,
    pub chat: String,
    pub from_user: String,
}

pub fn sess_meta_of(r: &Map<String, Value>) -> SessMeta {
    let typ = gstr(r.get("type"));
    let mut cwd = gstr(r.get("cwd"));
    if cwd.is_empty() && matches!(typ.as_str(), "session_meta" |
"turn_context") {
        cwd = gstr(get(r.get("payload"), "cwd"));
    }
    let title = match typ.as_str() {
        "custom-title" => gstr(r.get("customTitle")),
        "ai-title" => gstr(r.get("aiTitle")),
        _ => String::new(),
    };
    SessMeta { cwd, title, branch: gstr(r.get("gitBranch")), version:
gstr(r.get("version")), ..Default::default() }
}

/// The channel a user message came in by: `<channel
source="plugin:discord:discord" chat_id="…" user="…" …>`.
pub fn channel_of(text: &str) -> SessMeta {
    let Some(i) = text.find("<channel source=\"") else { return
SessMeta::default() };
    let tag = &text[i..text[i..].find('>').map(|e| i +
e).unwrap_or(text.len())];
    let attr = |k: &str| -> String {
        let pat = format!(" {k}=\"");
        tag.find(&pat).map(|p| &tag[p + pat.len()..]).and_then(|s|
s.split('"').next()).unwrap_or("").to_string()
    };
    let via = tag["<channel
source=\"".len()..].split('"').next().unwrap_or("").to_string();
    SessMeta { via, chat: attr("chat_id"), from_user:
attr("user"), ..Default::default() }
}

impl SessMeta {
    /// Fold a later line in: the first cwd stays (where the session
began); the others follow the newest line.
    pub fn take(&mut self, x: SessMeta) {
        if self.cwd.is_empty() {
            self.cwd = x.cwd;
        }
        // the first channel message names the session's channel
        if self.via.is_empty() {
            (self.via, self.chat, self.from_user) = (x.via, x.chat,
x.from_user);
        }
        for (a, b) in [(&mut self.title, x.title), (&mut self.branch,
x.branch), (&mut self.version, x.version)] {
            if !b.is_empty() {
```

```
                *a = b;
            }
        }
    }
}

/// Codex names the model once per turn, in a `turn_context` line; it
holds for the turn's rows after it.
pub fn turn_model(r: &Map<String, Value>) -> Option<String> {
    (gstr(r.get("type")) == "turn_context").then(||
gstr(get(r.get("payload"), "model"))).filter(|m| !m.is_empty())
}

const TOOL_HINTS: [&[u8]; 5] = [b"tool_use", b"tool_result", b"_call",
b"toolCall", b"toolResult"];

fn block_text(c: Option<&Value>) -> String {
    match c {
        Some(Value::String(s)) => s.clone(),
        Some(Value::Array(a)) => join_nl(
            a.iter().filter_map(|it| it.as_object()).filter(|o|
o.contains_key("text")).map(|o| gstr(o.get("text"))),
            "\n",
        ),
        other => graw(other),
    }
}

pub struct Tool {
    pub kind: &'static str,
    pub call_id: String,
    pub name: String,
    pub body: String,
    pub err: bool,
}

/// Calls and results, joinable by call_id.
pub fn extract_tools(line: &[u8], r: &Map<String, Value>) -> Vec<Tool> {
    if !TOOL_HINTS.iter().any(|h| memchr::memmem::find(line,
h).is_some()) {
        return vec![];
    }
    let mut out = vec![];
    if gstr(r.get("type")) == "response_item" {
        let p = r.get("payload");
        match gstr(get(p, "type")).as_str() {
            "function_call" => out.push(Tool { kind: "call", call_id:
gstr(get(p, "call_id")), name: gstr(get(p, "name")),
                                                body: gstr(get(p,
"arguments")), err: false }),
            "custom_tool_call" => out.push(Tool { kind: "call", call_id:
gstr(get(p, "call_id")), name: gstr(get(p, "name")),
                                                    body: gstr(get(p,
"input")), err: false }),
```

```
            "function_call_output" | "custom_tool_call_output" => {
                let o = get(p, "output");
                let body = match o {
                    Some(Value::Object(m)) => gstr(m.get("content")),
                    other => gstr(other),
                };
                out.push(Tool { kind: "result", call_id: gstr(get(p,
"call_id")), name: String::new(), body, err: false });
            }
            _ => {}
        }
        return out;
    }
    let m = r.get("message");
    if let Some(Value::Object(mm)) = m {
        if gstr(mm.get("role")) == "toolResult" {
            return vec![Tool { kind: "result", call_id:
gstr(mm.get("toolCallId")), name: gstr(mm.get("toolName")),
                                body: block_text(mm.get("content")), err:
gbool(mm.get("isError")) }];
        }
    }
    let c = match m {
        Some(Value::Object(mm)) if mm.contains_key("content") =>
mm.get("content"),
        _ => r.get("content"),
    };
    let Some(Value::Array(items)) = c else { return out };
    for it in items {
        let o = it.as_object();
        match gstr(get(Some(it), "type")).as_str() {
            "tool_use" => out.push(Tool { kind: "call", call_id:
gstr(get(Some(it), "id")), name: gstr(get(Some(it), "name")),
                                          body: graw(o.and_then(|o|
o.get("input"))), err: false }),
            "toolCall" => out.push(Tool { kind: "call", call_id:
gstr(get(Some(it), "id")), name: gstr(get(Some(it), "name")),
                                          body: graw(o.and_then(|o|
o.get("arguments"))), err: false }),
            "tool_result" => out.push(Tool { kind: "result", call_id:
gstr(get(Some(it), "tool_use_id")), name: String::new(),
                                             body:
block_text(o.and_then(|o| o.get("content"))),
                                             err: gbool(o.and_then(|o|
o.get("is_error"))) }),
            _ => {}
        }
    }
    out
}

/// Readable reasoning: Claude/omp thinking blocks, Codex reasoning
summary (content is encrypted).
/// None: the line has no thinking. Some(""): it thought, but the text
```

```
was not saved (newer Claude Code writes a thinking
/// block as a signature only; a Codex reasoning item may have no
summary), so the row is kept with empty text.
pub fn thinking_of(line: &[u8], r: &Map<String, Value>) -> Option<String>
{
    if memchr::memmem::find(line, b"\"thinking\"").is_none() &&
memchr::memmem::find(line, b"\"reasoning\"").is_none() {
        return None;
    }
    let mut parts: Vec<String> = vec![];
    let mut blocks = 0;
    let mut add = |t: Option<&Value>| {
        let s = gstr(t);
        let s = trim_go(&s);
        if !s.is_empty() {
            parts.push(s.to_string());
        }
    };
    let payload = r.get("payload");
    if gstr(r.get("type")) == "response_item" && gstr(get(payload,
"type")) == "reasoning" {
        for k in ["summary", "content"] {
            if let Some(Value::Array(v)) = get(payload, k) {
                for it in v {
                    add(get(Some(it), "text"));
                }
            }
        }
        return Some(parts.join("\n\n"));
    }
    let c = match r.get("message") {
        Some(Value::Object(mm)) if mm.contains_key("content") =>
mm.get("content"),
        _ => r.get("content"),
    };
    if let Some(Value::Array(items)) = c {
        for it in items {
            if gstr(get(Some(it), "type")) == "thinking" {
                blocks += 1;
                add(it.get("thinking"));
            }
        }
    }
    (blocks > 0).then(|| parts.join("\n\n"))
}

// ---------- line key + JSON parse ----------

/// blake2b(line, digest_size=8) read as a big-endian signed i64
(engine.py:364). The line keeps its "\n".
pub fn key(line: &[u8]) -> i64 {
    let h = blake2b_simd::Params::new().hash_length(8).hash(line);
    i64::from_be_bytes(h.as_bytes().try_into().unwrap())
}
```

```rust
/// engine.py LONE: a lone UTF-16 surrogate escape becomes \ufffd (what Go's
gjson keeps), then parse again.
fn fix_lone_surrogates(line: &[u8]) -> Vec<u8> {
    let hex = |c: u8| c.is_ascii_hexdigit();
    let is_hi = |b: &[u8], i: usize| b.len() >= i + 6 && b[i] == b'\\' &&
b[i + 1] == b'u' && matches!(b[i + 2], b'd' | b'D')
        && matches!(b[i + 3], b'8' | b'9' | b'a' | b'b' | b'A' | b'B') &&
hex(b[i + 4]) && hex(b[i + 5]);
    let is_lo = |b: &[u8], i: usize| b.len() >= i + 6 && b[i] == b'\\' &&
b[i + 1] == b'u' && matches!(b[i + 2], b'd' | b'D')
        && (b[i + 3].is_ascii_hexdigit() && matches!(b[i +
3].to_ascii_lowercase(), b'c'..=b'f')) && hex(b[i + 4]) && hex(b[i + 5]);
    let mut out = Vec::with_capacity(line.len());
    let mut i = 0;
    while i < line.len() {
        let not_escaped = i == 0 || line[i - 1] != b'\\';
        if not_escaped && is_hi(line, i) && !is_lo(line, i + 6) {
            out.extend_from_slice(b"\\ufffd");
            i += 6;
            continue;
        }
        if not_escaped && is_lo(line, i) {
            let prev_hi_tail = i >= 3 && matches!(line[i - 3], b'8' |
b'9' | b'a' | b'b' | b'A' | b'B') && hex(line[i - 2]) && hex(line[i -
1]);
            if !prev_hi_tail {
                out.extend_from_slice(b"\\ufffd");
                i += 6;
                continue;
            }
        }
        out.push(line[i]);
        i += 1;
    }
    out
}

/// The line as a JSON object, {} when it is JSON but not an object, None
when it is not JSON at all.
pub fn parse(line: &[u8]) -> Option<Value> {
    serde_json::from_slice::<Value>(line).ok().or_else(||
serde_json::from_slice::<Value>(&fix_lone_surrogates(line)).ok())
}

// ---------- redaction (engine.py RULES, same order) ----------

struct Rule {
    kind: &'static str,
    hints: &'static [&'static str],
    rx: Regex,
    group: usize,
}
```

```rust
fn rules() -> &'static Vec<Rule> {
    static R: OnceLock<Vec<Rule>> = OnceLock::new();
    R.get_or_init(|| {
        // Python re.ASCII semantics: \b \s \w and (?i) are ASCII-only,
so unicode(false) on bytes
        let b = |p: &str|
RegexBuilder::new(p).unicode(false).build().expect("rule");
        vec![
            Rule { kind: "private-key", hints: &["PRIVATE KEY"], group:
0,
                   rx: b(r"-----BEGIN (?:RSA |EC |OPENSSH |DSA |PGP )?
PRIVATE KEY-----[\s\S]*?(?:-----END (?:RSA |EC |OPENSSH |DSA |PGP )?
PRIVATE KEY-----|\z)") },
            Rule { kind: "anthropic-key", hints: &["sk-ant-"], group: 0,
rx: b(r"sk-ant-[A-Za-z0-9_-]{30,}") },
            Rule { kind: "openai-key", hints: &["sk-"], group: 0, rx:
b(r"\bsk-(?:proj-)?[A-Za-z0-9_-]{32,}") },
            Rule { kind: "aws-key", hints: &["AKIA", "ASIA"], group: 0,
rx: b(r"\b(?:AKIA|ASIA)[0-9A-Z]{16}\b") },
            Rule { kind: "github-token", hints: &["gh", "github_pat_"],
group: 0,
                   rx: b(r"\b(?:gh[pousr]_[A-Za-z0-9]{36,}|github_pat_[A-
Za-z0-9_]{60,})") },
            Rule { kind: "slack-token", hints: &["xox"], group: 0, rx:
b(r"\bxox[abprs]-[A-Za-z0-9-]{10,}") },
            Rule { kind: "google-key", hints: &["AIza"], group: 0, rx:
b(r"\bAIza[0-9A-Za-z_-]{35}") },
            Rule { kind: "jwt", hints: &["eyJ"], group: 0, rx:
b(r"\beyJ[A-Za-z0-9_-]{10,}\.eyJ[A-Za-z0-9_-]{10,}\.[A-Za-z0-9_-]
{10,}") },
            Rule { kind: "bearer", hints: &["earer"], group: 1, rx:
b(r"(?i)\bbearer\s+([A-Za-z0-9._~+/-]{24,})") },
            Rule { kind: "password", hints: &["ass", "ecret", "oken",
"pi_key", "pikey", "API_KEY", "APIKEY"], group: 1,
                   rx: b(r#"(?i)\b(?:password|passwd|secret|api_?key|
token)\s*[=:]\s*['"]?([A-Za-z0-9!#%&*+,./:;=?@^_|~-]{8,})"#) },
        ]
    })
}

/// A password/token value has letters AND digits and entropy >= 2.5
bits/char.
fn secret_like(v: &str) -> bool {
    if v.starts_with(['$', '<', '{', '(']) || v.contains('…') {
        return false;
    }
    if !v.chars().any(char::is_alphabetic) || !v.chars().any(|c|
c.is_numeric()) {
        return false;
    }
    let chars: Vec<char> = v.chars().collect();
    let n = chars.len() as f64;
    let mut counts = std::collections::HashMap::new();
    for c in &chars {
```

```
        *counts.entry(*c).or_insert(0usize) += 1;
    }
    let e: f64 = -counts.values().map(|&k| { let p = k as f64 / n; p *
p.log2() }).sum::<f64>();
    e >= 2.5
}

/// (clean text, sorted kinds joined by ',' or "").
pub fn redact(input: &str) -> (String, String) {
    let mut s = input.to_string();
    let mut kinds: Vec<&str> = vec![];
    for rule in rules() {
        if !rule.hints.iter().any(|h| s.contains(h)) {
            continue;
        }
        let bytes = s.as_bytes();
        let mut out = String::new();
        let mut last = 0usize;
        let mut hit = false;
        for caps in rule.rx.captures_iter(bytes) {
            let Some(m) = caps.get(rule.group) else { continue };
            let (a, z) = (m.start(), m.end());
            if a < last || bytes.get(a) == Some(&b'[') {
                continue; // already redacted by an earlier rule
            }
            let v = &s[a..z];
            if rule.kind == "password" && !secret_like(v) {
                continue;
            }
            out.push_str(&s[last..a]);
            let head: String = v.chars().take(4).collect();
            out.push_str(&format!("[{}:{}…{}]", rule.kind, head,
v.chars().count()));
            last = z;
            hit = true;
        }
        if hit && last > 0 {
            out.push_str(&s[last..]);
            s = out;
            kinds.push(rule.kind);
        }
    }
    kinds.sort();
    (s, kinds.join(","))
}

/// store.engine_of: the engine from the file path.
pub fn engine_of(path: &str) -> &'static str {
    let p = path.to_lowercase();
    if p.contains(".thclaws") {
        "thclaws"
    } else if p.contains(".omx") {
        "omx"
    } else if p.contains("/.pi/") {
```

```
        "pi"
    } else if p.contains(".omp") {
        "omp"
    } else if p.contains("codex") || p.contains("rollout-") {
        "codex"
    } else {
        "claude"
    }
}
```

## src/meta/rules.rs

which file is which session: tier_of, cat_of, proj_of and the path rules · 136 บรรทัด

```
//! The path rules (sql.py MACROS, one for one): tier, engine, repo
(cat), project, parent and home of a file.

use super::*;


pub(super) const UUID: &str = "[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-
f]{4}-[0-9a-f]{12}";

pub(super) struct Rx {
    pub(super) wf: Regex,
    pub(super) sub: Regex,
    pub(super) main_uuid: Regex,
    pub(super) main_rollout: Regex,
    pub(super) main_omp: Regex,
    pub(super) sub_omp: Regex,
    pub(super) parent_omp: Regex,
    pub(super) main_other: Regex,
    pub(super) history: Regex,
    pub(super) proj: Regex,
    pub(super) parent: Regex,
    pub(super) home: Regex,
    pub(super) agent_sid: Regex,
    pub(super) uuid_file: Regex,
    pub(super) gh: Regex,
    pub(super) gh_org: Regex,
    pub(super) gh_wt: Regex,
    pub(super) dashes: Regex,
    pub(super) wts: Regex,
    pub(super) wts_wt: Regex,
}

pub(super) fn rx() -> &'static Rx {
    static R: OnceLock<Rx> = OnceLock::new();
    R.get_or_init(|| {
```

```
        let r = |p: &str| Regex::new(p).expect("meta rule");
        Rx {
            wf: r(r"/subagents/workflows/.*agent-[^/]+\.jsonl$"),
            sub: r(r"/subagents/agent-[^/]+\.jsonl$"),
            main_uuid: r(&format!(r"/{UUID}\.jsonl$")),
            main_rollout: r(&format!(r"rollout-.*{UUID}\.jsonl(\.zst)?
$")),
            main_omp: r(r"/\.(omp|pi)/.*_[0-9a-f-]+\.jsonl$"),
            // omp keeps a subagent's transcript inside its parent
session's folder, named for its task:
            //   sessions/<project>/<time>_<uuid>/<Task>.jsonl  (the
parent is sessions/<project>/<time>_<uuid>.jsonl)
            sub_omp: r(&format!(r"/\.(omp|pi)/.*_{UUID}/[^/]+\.jsonl$")),
            parent_omp: r(&format!(r"/\.(?:omp|pi)/.*_({UUID})/[^/]+
\.jsonl$")),
            main_other: r(r"/\.(omx|thclaws|gemini)/"),
            history: r(r"history\.jsonl$"),
            proj: r(&format!(r"/([^/]+)/(?:{UUID}\.jsonl|{UUID}/
subagents/)")),
            parent: r(&format!(r"/({UUID})/subagents/")),
            home: r(r"^(.*?/\.?(?:claude|codex|omp|omx|pi|thclaws|gemini)
[^/]*)/"),
            agent_sid: r(r"^agent-(.*)\.jsonl$"),
            uuid_file: r(&format!(r"^({UUID})\.jsonl$")),
            gh: r(r".*github-com-(.*)$"),
            gh_org: r(r"^(soul-brews-studio|laris-co|dustboy-pm25|
floodboy-cm|nazt|nat-weerawan)-"),
            gh_wt: r(r"(-worktree-|-wt-|--claude-worktrees-).*$"),
            dashes: r(r"^-+|-+$"),
            wts: r(r".*-worktrees-(.*)$"),
            wts_wt: r(r"(-worktree-|-wt-).*$"),
        }
    })
}

/// DuckDB regexp_extract: group `g` of the first match, "" when there is
none.
pub(super) fn extract(re: &Regex, s: &str, g: usize) -> String {
    re.captures(s).and_then(|c| c.get(g)).map(|m|
m.as_str().to_string()).unwrap_or_default()
}

pub fn tier_of(path: &str) -> &'static str {
    let r = rx();
    if r.wf.is_match(path) {
        "workflow-agent"
    } else if r.sub.is_match(path) || r.sub_omp.is_match(path) {
        "subagent"
    } else if r.main_uuid.is_match(path) || r.main_rollout.is_match(path)
|| r.main_omp.is_match(path)
        || (r.main_other.is_match(path) && !r.history.is_match(path)) {
        "main"
    } else {
        "other"
```

```
    }
}

pub fn engine_of(path: &str) -> &'static str {
    if path.contains("rollout-") || path.contains("/.codex") {
        "codex"
    } else if path.contains("/.omp/") {
        "omp"
    } else if path.contains("/.omx/") {
        "omx"
    } else if path.contains("/.thclaws/") {
        "thclaws"
    } else if path.contains("/.gemini/") {
        "gemini"
    } else if path.contains("/.pi/") {
        "pi"
    } else {
        "claude"
    }
}

pub(super) fn base_of(path: &str) -> &str {
    path.rsplit('/').next().unwrap_or(path)
}

pub(super) fn proj_of(path: &str) -> Option<String> {
    Some(extract(&rx().proj, path, 1)).filter(|s| !s.is_empty())
}

pub(super) fn parent_of(path: &str) -> Option<String> {
    Some(extract(&rx().parent, path, 1)).filter(|s| !s.is_empty())
        .or_else(|| Some(extract(&rx().parent_omp, path, 1)).filter(|s| !
s.is_empty()))
}

pub(super) fn home_of(path: &str) -> String {
    extract(&rx().home, path, 1)
}

pub fn cat_of(engine: &str, proj: Option<&str>) -> String {
    let r = rx();
    if engine != "claude" {
        return format!("{engine} (no cwd in path)");
    }
    let Some(p) = proj.filter(|p| !p.is_empty()) else { return "(no
project dir)".into() };
    let lp = p.to_lowercase();
    if lp.contains("oracle-probe-cwd") {
        "probe (automated)".into()
    } else if lp.contains("github-com-") {
        let x = extract(&r.gh, &lp, 1);
        let x = r.gh_org.replace(&x, "");
        let x = r.gh_wt.replace(&x, "");
        r.dashes.replace_all(&x, "").into_owned()
```

```
    } else if lp.contains("-worktrees-") {
        let x = extract(&r.wts, &lp, 1);
        r.wts_wt.replace(&x, "").into_owned()
    } else {
        "home / other".into()
    }
}
```

## src/meta/agg.rs

the per-month pass that counts rows by type and sums tokens per session, and its cache · 273 บรรทัด

```
//! One pass per month over the rows' sources, and its cache in OUT/meta/
cache/YYYY-MM.tsv.

use super::*;


/// Rows by type, in this order: user text, assistant text, thinking,
tool calls, tool results, tool errors
/// (errors are results with err = true, so they are also counted in
results).
pub const TYPES: [&str; 6] = ["user_text", "assistant_text", "thinking",
"tool_calls", "tool_results", "tool_errors"];

pub(super) fn types_of(s: &str, role: Option<&str>, kind: Option<&str>,
err: bool) -> [i64; 6] {
    let mut t = [0; 6];
    match (s, role, kind) {
        ("text", Some("user"), _) => t[0] = 1,
        ("text", Some("assistant"), _) => t[1] = 1,
        ("thinking", _, _) => t[2] = 1,
        ("tools", _, Some("call")) => t[3] = 1,
        ("tools", _, Some("result")) => {
            t[4] = 1;
            t[5] = err as i64;
        }
        _ => {}
    }
    t
}

#[derive(Default, Clone)]
pub(super) struct Agg {
    pub(super) n_rows: i64,
    pub(super) text_rows: i64,
    pub(super) text_msgs: i64,
    pub(super) by: [i64; 6],
    pub(super) first_ts: Option<String>,
```

```
    pub(super) last_ts: Option<String>,
}

impl Agg {
    pub(super) fn ts(&mut self, ts: Option<&str>) {
        let Some(t) = ts else { return };
        if self.first_ts.as_deref().is_none_or(|f| t < f) {
            self.first_ts = Some(t.to_string());
        }
        if self.last_ts.as_deref().is_none_or(|l| t > l) {
            self.last_ts = Some(t.to_string());
        }
    }
    pub(super) fn add(&mut self, o: &Agg) {
        self.n_rows += o.n_rows;
        self.text_rows += o.text_rows;
        self.text_msgs += o.text_msgs;
        for (a, b) in self.by.iter_mut().zip(o.by) {
            *a += b;
        }
        self.ts(o.first_ts.as_deref());
        self.ts(o.last_ts.as_deref());
    }
}

/// What a session's model answers cost: tokens summed over its API
responses (each once: the month file keeps a
/// response's tokens on one row) and the answers per model.
#[derive(Default, Clone, Debug, PartialEq)]
pub(super) struct Usage {
    pub(super) out: i64,
    pub(super) inp: i64,
    pub(super) cache_read: i64,
    pub(super) cache_write: i64,
    pub(super) models: BTreeMap<String, i64>,
}

impl Usage {
    pub(super) fn add(&mut self, o: &Usage) {
        self.out += o.out;
        self.inp += o.inp;
        self.cache_read += o.cache_read;
        self.cache_write += o.cache_write;
        for (m, n) in &o.models {
            *self.models.entry(m.clone()).or_default() += n;
        }
    }
    /// The models, the most answers first, and their answer counts.
    pub(super) fn ranked(&self) -> (Vec<String>, Vec<i64>) {
        let mut v: Vec<(&String, &i64)> = self.models.iter().collect();
        v.sort_by(|a, b| b.1.cmp(a.1).then(a.0.cmp(b.0)));
        (v.iter().map(|x| x.0.clone()).collect(), v.iter().map(|x|
*x.1).collect())
    }
```

```
}

/// One month's aggregates: per file path, and per session (base, tier).
#[derive(Default)]
pub(super) struct MonthAgg {
    pub(super) files: HashMap<String, Agg>,
    pub(super) sessions: HashMap<(String, String), Agg>,
    pub(super) usage: HashMap<(String, String), Usage>,
}

/// msgkey for a text row is md5(to_json([s, role, ts, text]));
count(DISTINCT msgkey) only needs a stable key of
/// (role, ts, text) with NULL kept apart from any string, so a length-
prefixed blake2b-8 does the same job.
pub(super) fn msg_key(role: Option<&str>, ts: Option<&str>, text:
Option<&str>) -> u64 {
    let mut st = blake2b_simd::Params::new().hash_length(8).to_state();
    for v in [role, ts, text] {
        match v {
            None => {
                st.update(&[0]);
            }
            Some(s) => {
                st.update(&[1]);
                st.update(&(s.len() as u64).to_le_bytes());
                st.update(s.as_bytes());
            }
        }
    }
    u64::from_le_bytes(st.finalize().as_bytes().try_into().unwrap())
}

pub(super) fn str_col<'a>(b: &'a RecordBatch, n: &str) -> &'a StringArray
{
b.column_by_name(n).unwrap().as_any().downcast_ref::<StringArray>().unwrap()
}

pub(super) fn opt(a: &StringArray, i: usize) -> Option<&str> {
    if a.is_null(i) { None } else { Some(a.value(i)) }
}

pub(super) fn month_agg(path: &Path, agent: &str) -> MonthAgg {
    let b =
ParquetRecordBatchReaderBuilder::try_new(fs::File::open(path).unwrap()).unwrap();
    // model and the token columns came later: a month written before
them has none
    let want = ["s", "role", "kind", "err", "ts", "text", "sources",
"model", "msg_id", "out_tokens", "in_tokens", "cache_read",
"cache_write"];
    let idx: Vec<usize> = want.iter().filter_map(|n|
b.schema().index_of(n).ok()).collect();
    let mask = ProjectionMask::roots(b.parquet_schema(), idx);
    let rd =
b.with_projection(mask).with_batch_size(16_384).build().unwrap();
```

```
    let prefix = format!("{agent}:");
    let mut out = MonthAgg::default();
    let mut class: HashMap<String, Option<(String, String)>> =
HashMap::new(); // path -> session key (None = tier other)
    let mut msgs: HashMap<(String, String), HashSet<u64>> =
HashMap::new();
    let mut touched: Vec<(String, String)> = vec![];
    for batch in rd {
        let batch = batch.unwrap();
        let (s, role, ts, text) = (str_col(&batch, "s"), str_col(&batch,
"role"), str_col(&batch, "ts"), str_col(&batch, "text"));
        let kind = str_col(&batch, "kind");
        let err =
batch.column_by_name("err").unwrap().as_any().downcast_ref::<arrow::array::BooleanArray>().u
        let model = batch.column_by_name("model").map(|a|
a.as_any().downcast_ref::<StringArray>().unwrap());
        let msg_id = batch.column_by_name("msg_id").map(|a|
a.as_any().downcast_ref::<StringArray>().unwrap());
        let tok = |n: &str| batch.column_by_name(n).map(|a|
a.as_any().downcast_ref::<Int64Array>().unwrap());
        let (t_out, t_in, t_cr, t_cw) = (tok("out_tokens"),
tok("in_tokens"), tok("cache_read"), tok("cache_write"));
        let src =
batch.column_by_name("sources").unwrap().as_any().downcast_ref::<ListArray>().unwrap();
        let vals =
src.values().as_any().downcast_ref::<StringArray>().unwrap();
        let offs = src.value_offsets();
        for i in 0..batch.num_rows() {
            let is_text = s.value(i) == "text";
            let t = opt(ts, i);
            let ty = types_of(s.value(i), opt(role, i), opt(kind, i), !
err.is_null(i) && err.value(i));
            touched.clear();
            for j in offs[i] as usize..offs[i + 1] as usize {
                let Some(rest) = vals.value(j).strip_prefix(&prefix) else
{ continue };
                let Some((p, _line)) = rest.rsplit_once(':') else
{ continue };
                let fa = out.files.entry(p.to_string()).or_default();
                fa.n_rows += 1;
                fa.text_rows += is_text as i64;
                for (a, b) in fa.by.iter_mut().zip(ty) {
                    *a += b;
                }
                fa.ts(t);
                let k = class.entry(p.to_string()).or_insert_with(|| {
                    let tier = tier_of(p);
                    (tier != "other").then(|| (base_of(p).to_string(),
tier.to_string()))
                });
                if let Some(k) = k
                    && !touched.contains(k) {
                    touched.push(k.clone()); // a row counts once per
session, however many of its copies it holds
```

```
                }
            }
            // an answer: the row that carries its tokens, or an
assistant text row with no response id (Codex)
            let n = |c: Option<&Int64Array>| c.and_then(|a| (!
a.is_null(i)).then(|| a.value(i))).unwrap_or(0);
            let has_tokens = t_out.is_some_and(|a| !a.is_null(i));
            let answer = has_tokens || (is_text && opt(role, i) ==
Some("assistant") && msg_id.is_none_or(|a| a.is_null(i)));
            let mdl = model.and_then(|a| opt(a, i)).filter(|m| !
m.is_empty());
            for k in &touched {
                if has_tokens || answer {
                    let u = out.usage.entry(k.clone()).or_default();
                    u.out += n(t_out);
                    u.inp += n(t_in);
                    u.cache_read += n(t_cr);
                    u.cache_write += n(t_cw);
                    if let (true, Some(m)) = (answer, mdl) {
                        *u.models.entry(m.to_string()).or_default() += 1;
                    }
                }
                let sa = out.sessions.entry(k.clone()).or_default();
                sa.n_rows += 1;
                sa.text_rows += is_text as i64;
                for (a, b) in sa.by.iter_mut().zip(ty) {
                    *a += b;
                }
                sa.ts(t);
                if is_text {
msgs.entry(k.clone()).or_default().insert(msg_key(opt(role, i), t,
opt(text, i)));
                }
            }
        }
    }
    for (k, m) in msgs {
        out.sessions.get_mut(&k).unwrap().text_msgs = m.len() as i64;
    }
    out
}

// ---------- the month cache: OUT/meta/cache/YYYY-MM.tsv ----------

pub(super) fn stamp_of(p: &Path) -> String {
    let m = fs::metadata(p).unwrap();
    format!("{}\t{}.{}", m.size(), m.mtime(), m.mtime_nsec())
}

// NULL is written as \N: an empty ts is a real value (the undated month
holds rows with ts = "")
pub(super) fn agg_tsv(a: &Agg) -> String {
    let by: Vec<String> = a.by.iter().map(|x| x.to_string()).collect();
    format!("{}\t{}\t{}\t{}\t{}\t{}", a.n_rows, a.text_rows, a.text_msgs,
```

```
by.join("\t"), a.first_ts.as_deref().unwrap_or("\\N"),
            a.last_ts.as_deref().unwrap_or("\\N"))
}

pub(super) fn agg_from(c: &[&str]) -> Agg {
    let n = |i: usize| c[i].parse::<i64>().unwrap_or(0);
    let t = |i: usize| Some(c[i].to_string()).filter(|s| s != "\\N");
    Agg { n_rows: n(0), text_rows: n(1), text_msgs: n(2), by:
std::array::from_fn(|j| n(3 + j)), first_ts: t(9), last_ts: t(10) }
}

/// The cache format and the rules it was counted with: bump it when a
path rule changes what a session is (#4: omp
/// subagents are sessions), so every month is counted again once.
const CACHE_V: &str = "#5";

pub(super) fn save_cache(p: &Path, stamp: &str, agent: &str, m:
&MonthAgg) {
    let tmp = p.with_extension("tsv.part");
    let mut w = std::io::BufWriter::new(fs::File::create(&tmp).unwrap());
    writeln!(w, "{CACHE_V}\t{stamp}\t{agent}").unwrap();
    for (f, a) in &m.files {
        writeln!(w, "F\t{f}\t{}", agg_tsv(a)).unwrap();
    }
    for ((base, tier), a) in &m.sessions {
        writeln!(w, "S\t{base}\t{tier}\t{}", agg_tsv(a)).unwrap();
    }
    for ((base, tier), u) in &m.usage {
        let models = serde_json::to_string(&u.models).unwrap();
        writeln!(w, "U\t{base}\t{tier}\t{}\t{}\t{}\t{}\t{models}", u.out,
u.inp, u.cache_read, u.cache_write).unwrap();
    }
    w.flush().unwrap();
    drop(w);
    fs::rename(tmp, p).unwrap();
}

/// The cached aggregates, or None when the cache was made from another
version of the month file.
pub(super) fn load_cache(p: &Path, stamp: &str, agent: &str) ->
Option<MonthAgg> {
    let mut lines =
BufReader::new(fs::File::open(p).ok()?).lines().map_while(Result::ok);
    if lines.next()? != format!("{CACHE_V}\t{stamp}\t{agent}") {
        return None;
    }
    let mut m = MonthAgg::default();
    for l in lines {
        let c: Vec<&str> = l.split('\t').collect();
        match c[0] {
            "F" if c.len() == 13 => {
                m.files.insert(c[1].to_string(), agg_from(&c[2..]));
            }
            "S" if c.len() == 14 => {
```

```
                m.sessions.insert((c[1].to_string(), c[2].to_string()),
agg_from(&c[3..]));
            }
            "U" if c.len() == 8 => {
                let n = |i: usize| c[i].parse::<i64>().unwrap_or(0);
                let models = serde_json::from_str(c[7]).ok()?;
                m.usage.insert((c[1].to_string(), c[2].to_string()),
Usage { out: n(3), inp: n(4), cache_read: n(5), cache_write: n(6),
models });
            }
            _ => return None,
        }
    }
    Some(m)
}
```

## src/ops/text.rs

the text view of `relic-scan ops` · 74 บรรทัด

```
//! The snapshot as text lines, for `relic-scan ops` and `--watch`.

use serde_json::Value;

pub fn dur(s: i64) -> String {
    if s >= 3600 { format!("{}:{:02}:{:02}", s / 3600, s / 60 % 60, s %
60) } else { format!("{}:{:02}", s / 60, s % 60) }
}

pub fn render(v: &Value) -> Vec<String> {
    let c = &v["chain"];
    let mut o = vec![];
    match c["state"].as_str() {
        Some("running") => {
            let (el, eta) = (c["elapsed_s"].as_i64().unwrap_or(0),
c["eta_s"].as_f64());
            o.push(format!("CHAIN {}  RUNNING  {} elapsed{}",
c["machine"].as_str().unwrap_or(""), dur(el),
                match (c["typical_total_s"].as_f64(), eta) {
                    (Some(t), Some(e)) if e > 0.0 => format!(" · typical
{} · about {} left", dur(t as i64), dur(e as i64)),
                    (Some(t), _) => format!(" · typical {}, over it now",
dur(t as i64)),
                    _ => String::new() }));
        }
        Some("idle") => o.push(format!("CHAIN {}  idle  last pass {} took
{} · next in about {}", c["machine"].as_str().unwrap_or(""),
            c["started"].as_str().unwrap_or("")[11..].to_string(),
dur(c["took_s"].as_i64().unwrap_or(0)),
dur(c["next_in_s"].as_i64().unwrap_or(0)))),
        _ => o.push("CHAIN  no pass logged yet (".to_string() +
```

```
v["log"].as_str().unwrap_or("") + ")"),
    }
    for s in c["stages"].as_array().into_iter().flatten() {
        let mark = match s["state"].as_str() { Some("done") => "✓",
Some("running") => "●", _ => "·" };
        let t = s["secs"].as_f64().map(|x| format!("{x:.0}
s")).unwrap_or_default();
        let ty = s["typical_s"].as_f64().map(|x| format!("(typical {x:.0}
s)")).unwrap_or_default();
        o.push(format!("  {mark} {:<18} {:>7} {:<18} {}",
s["name"].as_str().unwrap_or(""), t, ty,
s["detail"].as_str().unwrap_or("")));
    }
    let p = v["processes"].as_array().cloned().unwrap_or_default();
    o.push(String::new());
    o.push(if p.is_empty() { "PROCESSES  none running".into() } else
{ "PROCESSES".into() });
    for x in &p {
        o.push(format!("  {:<22} {:<8} {:>9}  {:>5.0}% cpu  {:>6.0} MB",
x["what"].as_str().unwrap_or(""), x["user"].as_str().unwrap_or(""),
x["elapsed"].as_str().unwrap_or(""),
            x["cpu"].as_f64().unwrap_or(0.0),
x["rss_mb"].as_f64().unwrap_or(0.0)));
    }
    let h = &v["hub"];
    o.push(String::new());
    if h["ok"].as_bool() == Some(true) {
        o.push(format!("HUB  load {:.1} of {} cpus · {} job(s) running",
h["load"][0].as_f64().unwrap_or(0.0), h["cpus"],
h["running"].as_array().map_or(0, |a| a.len())));
        for m in h["machines"].as_array().into_iter().flatten() {
            o.push(format!("  {:<20} last push {:>6} ago  {}",
m["machine"].as_str().unwrap_or(""),
dur(m["push_age_s"].as_i64().unwrap_or(0)),
m["sync"].as_str().unwrap_or("").chars().take(60).collect::<String>()));
        }
        for w in h["warnings"].as_array().into_iter().flatten() {
            o.push(format!("  ! {}", w.as_str().unwrap_or("")));
        }
    } else {
        o.push(format!("HUB  not read ({})",
h["error"].as_str().unwrap_or("")));
    }
    let s = &v["s3"];
    if s["running"].as_bool() == Some(true) {
        o.push(String::new());
        o.push(format!("S3  {} {}/{}", s["job"].as_str().unwrap_or(""),
s["done"], s["total"]));
    }
    o
}

/// `render` for a terminal: the word HUB is a link to the hub, and the
last line points at the page's Activity view.
```

```
/// `on` false leaves the escapes out (a pipe, NO_COLOR); the page line
stays as plain text.
pub fn render_linked(v: &Value, base: &str, hub: &str, on: bool) ->
Vec<String> {
    use crate::links::{link, view_url};
    let mut o = render(v);
    if hub != "off" && v["hub"]["ok"].as_bool() == Some(true) {
        if let Some(l) = o.iter_mut().find(|l| l.starts_with("HUB")) {
            *l = format!("{}{}", link(on, hub, "HUB"), &l[3..]);
        }
    }
    let page = view_url(base, "ops");
    o.push(String::new());
    o.push(format!("page  {}", link(on, &page, &page)));
    o
}
```